UI BUILDER: AN AUTOMATIC CODE GENERATION TOOL FOR DESIGN & IMPLEMENTATION OF UI.

Project Id: 2022 - 070

Final Report

Keshawa Ekanayake (IT19150758)
Sakthivel Sanjay (IT19158228)
Kasuni Navodya (IT19144986)
Kavindi Gimshani (IT19150826)

B.Sc. (Hons) Degree in Information Technology

Department of Computer Systems and Software Engineering

Sri Lanka Institute of
Information Technology - Sri Lanka

January 2022

UI BUILDER: AN AUTOMATIC CODE GENERATION TOOL FOR DESIGN & IMPLEMENTATION OF UI.

Project Id: 2022 - 070

Final Report

Keshawa Ekanayake (IT19150758)
Sakthivel Sanjay (IT19158228)
Kasuni Navodya (IT19144986)
Kavindi Gimshani (IT19150826)

B.Sc. (Hons) Degree in Information Technology

Department of Computer Systems and Software Engineering

Sri Lanka Institute of

Information Technology - Sri Lanka

January 2022

TABLE OF CONTENTS

DECLARATION OF CANDIDATE AND SUPERVISOR i
ACKNOWLEDGEMENT ii
ABSTRACT iii
LIST OF FIGURES iv
LIST OF TABLES vi
LIST OF ABBREVATIONS vii
LIST OF APPENDICES vii
1. INTRODUCTION 1
1.1 Background Study. 1
1.2 Literature review. 4
1.3 Research Gap. 11
1.4 Research Problem. 14
2. OBJECTIVES. 15
2.1. Main Objective. 15
2.2. Specific Objective. 16
3. METHODOLOGY. 17
3.1. System Overview Diagram. 19
3.2. System Description. 20
3.3. Development Process. 23
3.3.1 UI generator 23
3.3.2 Frontend Code Generator - Web 27
3.3.3 Frontend Code Generator - Mobile 32
3.3.4 Backend Code Generator 34
3.4. Testing. 44
3.4.1 UI generator - Testing 44
3.4.2 Web Frontend Generator - Testing 48
3.4.3 Mobile Frontend Generator - Testing 54
3.4.4 Backend & Database Generator - Testing 60
3.5. Software Development Life Cycle (SDLC). 70
3.6. Gantt Chart. 70
3.7. Work Breakdown Structure. 71
3.8. Commercialization Aspects of the Products 71
4. RESULTS & DISCUSSION 71
4.1 Results 71
4.1.1 UI Generator 71

4.1.2 Web Frontend Generator. 73
4.1.3 Mobile Frontend Generator 75
4.1.4 Backend & Database Generator 77
4.2 Research Findings 83
4.3 Discussions 85
4.3.1 UI Generator. 85
4.3.2 Web Frontend Generator. 86
4.3.3 Mobile Frontend Generator 87
4.3.4 Backend & Database Generator 88
5. SUMMARY OF EACH ESTUDENT'S CONTRIBUTIONS 89
6. CONCLUSION 91
REFERENCE LIST. 92
APPENDICES 95

DECLARATION OF CANDIDATE AND SUPERVISOR

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Ekanayake K.L.W	IT19150758	keshawa
Sakthivel.S	IT19158228	Sanjay
H.M. Kasuni Navodya	IT19144986	Kasuni
D. P. Kavindi Gimshani	IT19150826	Kavindi

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

Name of supervisor: Dr. Nuwan Kodagoda

Name of co-supervisor: Ms. Kushnara Suriyawansa

…… ……..

Signature of the supervisor: Date

(Dr. Nuwan Kodagoda)

…… ……

Signature of the co-supervisor: Date

(Ms. Kushnara Suriyawansa)

ACKNOWLEDGEMENT

We would like to thank Dr. Nuwan Kodagoda and Ms. Kushnara Suriyawansa, our research supervisors for their guidance, encouragement and patience which helped us to complete our research project successfully. We truly need to thank Sri Lankan Institute of Information Technology (SLIIT) for their excellence in creating numerous success stories and helping us show our talents by conducting this research project. All facilities and workshops conducted by the CDAP team at SLIIT helped to understand and successfully complete each phase of research progress.

We would also like to extend our thanks to our colleagues at SLIIT who took their valuable time to fill the survey conducted for gathering requirements and understanding real needs of different users. We need to emphasize the support within our research project team for standing together in all situations. Special thanks to our team leader for guiding us and monitoring our progress throughout the research.

Finally, we wish to thank our parents for their support and encouragement throughout our studies.

Keshawa Ekanayake, Sanjay Sakthivel, Kasuni Navodya, Kavindi Gimshani
Department of Computer Science and Software Engineering
Faculty of Computing
Sri Lanka Institute of Information Technology (SLIIT)

ABSTRACT

With the advancement of technology, the growth of software applications has rapidly increased. Both small-scaled businesses and large-scale corporations prefer software applications for achieving their business objectives such as marketing, automation, management, etc. With changing business requirements, the software application had to scale in the direction of adding new features or improving its non-functional requirements. In many aspects, scaling results in increasing the code size and complexity of the application. The question arises on how to improve the software development process to withstand changing business requirements. Automatic Code Generation in Automatic Programming is a suitable solution. This paper introduces a new tool "UI Builder" which can convert user interface (UI) sketches to code. The use of a code generator increases the productivity and efficiency of the organization. The tool satisfies different needs of developers using its enhanced features and easily understandable user flow. With critical experimentation, the effectiveness of the tool is compared with manual implementation to highlight the benefits and advantages. A bug-free, structured, and properly organized code was generated using the tool. The Model used in the tool achieves 91.0% accuracy and 93.0% validation accuracy. The generated code showed 92.0% improvement in average development time and 87.5% improvement in code size.

Keywords—Software application, Automatic Code Generation (ACG), Automatic Programming, Code generator.

LIST OF FIGURES

Figure 3.1 Input/Output - component 01 17
Figure 3.2 Input/Output - component 02 17
Figure 3.3 Input/Output - component 03 18
Figure 3.4 Input/Output - component 04 18
Figure 3.5 System Overview Diagram 19
Figure 3.6 Sample rough sketches from the survey. 24
Figure 3.7 Sample dataset of the survey 24
Figure 3.8 Trained Dataset 24
Figure 3.9 Tested Dataset 25
Figure 3.10 Convert rough sketche to XML file 25
Figure 3.11 UI Generator process 26
Figure 3.12 Provided system template styles 29
Figure 3.13 Template Style Form 29
Figure 3.14 JSON structure 30
Figure 3.15 Web code generate process 31
Figure 3.16 How system reads the UI elements from the XML file 33
Figure 3.17 The color themes available in the system 34
Figure 3.18 Conversion of XML file to Refined XML File 36
Figure 3.19 Refined XML file mapping on table structure 36
Figure 3.20 Data read from Refined XML and displayed in data grid 38
Figure 3.21 Table creation process 40
Figure 3.22 Code used to generate ‘INSERT’ CRUD operation 42
Figure 3.23 Code segment responsible for ‘POST’ code generation 43
Figure 3.24 SCRUM Hierarchy and Agile Methodology 70
Figure 3.25 Gantt Chart 71
Figure 3.26 Work Breakdown Structure 72
Figure 4.1 standard UI - result 01 74
Figure 4.2 XML file - result 02 75
Figure 4.3 Formatted JSON file - result 1.1 76
Figure 4.4 Downloaded Frontend Code - result 1.2 77
Figure 4.5 Downloaded Result UI - result 1.3 77

Figure 4.6 Formatted JSON file - result 2.1 78
Figure 4.7 Downloaded Frontend Code - result 2.2 78
Figure 4.8 Downloaded Result UI - result 2.3 79
Figure 4.9 Flutter code as txt - result 01 80
Figure 4.10 Flutter Code displayed in the system - result 02 81
Figure 4.11 Sample refined XML file - result 01 82
Figure 4.12 Evaluation metric accuracy 90

LIST OF TABLES

Table 1.1 Summary of previous research. 9
Table 1.2 Concepts and basic features of different existing tools. 12
Table 1.3 Comparison with different existing tools 13
Table 3.1 Sub-components and their respective outcomes 22
Table 3.2 UI specification details included in the XML File 28
Table 3.3 CRUD Code generation steps 41
Table 3.4 Unit Testing - component 01 45
Table 3.5 Integration Testing - component 01 48
Table 3.6 Unit Testing - component 02 49
Table 3.7 Integration Testing - component 02 53
Table 3.8 Unit Testing - component 03 55
Table 3.9 Integration Testing - component 03 58
Table 3.10 Unit Testing - component 04 61
Table 3.11 Integration Testing - component 04 67
Table 4.1 Generated CRUD operation code samples 82
Table 4.2 Generated REST API Code samples 84
Table 4.3 Evaluation Metric Results - component 02 91
Table 4.4 Evaluation Metric Results - component 03 92
Table 4.5 Evaluation Metric Results - component 04 93
Table 5.1 Individual Contribution 94

LIST OF ABBREVATIONS

ACG	-	Automatic Code Generation.
SLDC	-	Software Development Life Cycle.
DND	-	Drag and Drop.
BL	-	Business Logic.
XML	-	eXtensible Markup Language.
JSON	-	JavaScript Object Notation.
UI	-	User Interface.
UX	-	User Experience.
ADC	-	Automatic Database Creator.
REST	-	Representational Stateful Transfer.
ML	-	Machine Learning.
HTML	-	Hyper Text Markup Language.
API	-	Application Programming Interface.
JS	-	JavaScript.
MDA	-	Model Driven Architecture.
UML	-	Unified Modeling Language.
CIM	-	Computation Independent Model.
PIM	-	Platform Independent Model.
PSM	-	Platform Specific Model.
MERN	-	MongoDB, Express, ReactJS, NodeJS.

LIST OF APPENDICES

Appendix - A Results from selected user group (UI Generator) 100
Appendix - B Results from selected user group (Web Frontend Generator) 103
Appendix - C Results from selected user group (Mobile Frontend Generator) 106
Appendix - D Results from selected user group (Backend Generator) 108
Appendix - E UI Generator Interfaces 111
Appendix - F Web Frontend Generator Interfaces 112
Appendix - G Mobile Frontend Generator Interfaces 114
Appendix - H Backend & Database Generator Interfaces 116
Appendix - I Plagiarism Report 118

1.INTRODUCTION

1.1 Background Study.

Automatic Code Generation (ACG) plays an important role in software engineering process which introduces new ways and trends in software development process [2]. ACG marks to be the most trending topic among software developers who involve in continuous software development. Making the Software Development Life cycle (SDLC) process flexible, platform independent and auto-generated code with globally accepted standards is proposed by the Object Management Group (OMG) [3].

In recent times several approaches have been put forward in the direction of ACG. For example, Converting UML diagrams to functional code using forward engineering [4][5]. The DND approach [6] which made a considerable impact among the software engineering community. To get some broader understanding on the ACG concept, DND approach can be highlighted. In DND approach the users were given the opportunity to design UI on their own by dragging and dropping UI elements. In addition to the highlighted concept several other research focus on generating a fully functional frontend and backend code using different techniques [7].

Our research stands unique from other previous work, in a way the auto-generated code is embedded with business logic (BL). According to observation and walkthrough conducted among the existing ACG tools [1] a common feature was lacking for developers. The generated code resulted from these tools were code templates with classes, on which attributes and methods are bind. Software developers' expectation was far away from the resulting code from ACG tools. Need of a fully functional code with less involvement of developers, make the debugging process easier, following proper coding standards, understandable comments and following best practices of coding [2] are some of the highlighting expectations from developers on an ACG tool.

The problem of not having a proper application for building UI templates is the designers and the software engineers have their own design ideas, but they are unable to design a proper UI template to their own preferred ideas. As an example, the designers make a rough paper sketch for their UI templates before going to design

using any technology. After completing the rough sketch design the designer/software engineer needs to design using any technology (react, ionic, android etc.). The problem is the designers, and the software engineers, must spend more time completing this task (developing the UIs (User Interface)). They must code many code lines to align a single text box or some image. Therefore, designing a UI is a huge task in ICT field.

Getting a user preferred UI template in a very short period is a huge advantage for the designers. Simply the users need to give the rough sketch of the UI to the application and the application converts that rough sketch to a beautiful UI design. There is a unique feature that is the users can select the best UI from the UI templates that are provided by the application.

The web frontend code generator acts as an inner component of this ACG tool. Converting XML metadata uploaded as input details into JSON data, executing code using those JSON details and adding template styles are some of the highlight functions of the component. Front-end database maintenance, code validations, best practices are some of the supporting features of the component to deliver quality front-end code.

Even though people can gain the knowledge from varies resources, the mobile application development is a time-consuming task. On average an application takes from three to nine months for the development according to the complexity of the application. This amount of time is very critical for the small business owners, managers, and startup companies as they can be gained new customers through releasing their own mobile application for the customers as well as themselves. This research suggests the frontend code generator for mobile applications. Even though the people do not have the knowledge on coding the applications the application can be developed which saves the cost and time. Also, even the developers have the knowledge and the experience, the time can be saved that they must spend on developing the frontend of the application.

Some projects require developers or software engineers to design user interfaces in a short period of time. In those cases, the developers must create the user interfaces in a short period of time. Therefore, for those situations the UI template generator is an especially important application for those tasks. Obtaining a user-favored UI template

in a short period of time is a huge benefit for designers. Simply provide a rough sketch of the UI to the application, and the application will convert that rough sketch into a beautiful UI design. There is a unique feature in that users can choose the best UI from the UI templates provided by the application.

The web front-end code generator component's primary goal is to implement the features mentioned above in our UI Builder tool. Along with tool improvements, reusability and efficiency of frontend code are considered to maintain user-friendliness and simplicity. This internal component is critical for supporting the tool's main purpose by capturing the attention of developers during the web front-end code generation process. The main feature that distinguishes this component from other tools is that it is implemented using React, a popular and high-performance front-end library among developers. Furthermore, this web frontend generator allows developers to select user-favorite template designs. Some internal modules include XML Uploader, XML2JSON Converter, Template Style Selection, and Frontend Code Downloader.

1.2 Literature review.

Optimizing and increasing the efficiency of software development process is the main objective of software developers and designers. Much research has been done in the direction of making the software development more efficient. ACG is one the active research area followed by many researchers, which they suggest in new approaches to make the code generation process efficient and satisfy real expectations of developers. I collected several research papers and deeply analyzed them to get a clear understanding on what approaches researchers have used in code generation in their work, what they have missed or suggested as future work and finally what area in ACG needs more research to build an intelligent code generation tool.

In 2005, Sun, Zhengxing, and Jing Liu. in their research [9] explained concepts of sketch-based informal user interface for graphic computing, which can be characterized by two properties: stroke-based input and perceptual processing of strokes. A sketch-based graphics input prototype system designed for creative brainstorming in conceptual design was introduced by conducting the research.

The research paper published by M. Usman, A. Nadeem and T. -h. Kim on 2008 [10], introduced the UJECTOR tool which can generate executable java code from UML diagrams such as class diagram, sequence diagram and activity diagram. It combines both structural and behavioural features of an object-oriented (OO) application. This tool consists of 3 main components namely XMIParser, Code Generator (JavaFilesCreator), and Code Merger. XMIParser captures the UML models XMI as input and then generates java code files. Code Merger Component takes isolated Java code from the code generator and replaces it with activity diagram references in class method codes. Finally, the java file creator obtains the complete java code as input and executes the java files in the defined locations.

In 2009, M. Tubishat, I. Alsmadi and M. Al-Kabi published their research paper [11] which clearly explained the process of converting a UI design to XML file. The researcher strongly believed on the need to save and preserve UI specifications which includes both state and structure for later communication and modifications in requirements, design, and implementation. The approach had the capability to capture only structure related UI information while ignoring the controls present in UI. Performing a regression testing when the requirements on UI changed was a

highlighting concept with their research. They concluded their research by explaining the impact of XML file with state changes in UI and the importance of software testing when the UI requirements are changes or modified. They expected to develop an adapter to deal with different XML formats since their approach worked only with the standard XML format.

In 2010, Kinzel, J in the research work [12] introduced an approach to generate as many of the generic components as possible, to allow developers to concentrate on optimizing the user interface and the backend connectivity. The procedure involves in steps such as creating the development project using an application template, definition of the data model and the required functions, generation of elements for the provision of data and functions, individual implementation of the user interface and the backend connectivity. An XML file holds the information used to generate the data model (entities, relationship, and methods). The generation mechanism is implemented in Java using Velocity. Individual templates are used for producing each of the required program elements, code, configuration files, etc. These templates are stored in simple text-files. Each project defines which components are generated by using a configuration file. The target platform is independent of this generation mechanism such as Java, and C#.

In the year 2016, Sunil D Rathod published the work on ACG with business logic by capturing the attributes from UI via XML file [13]. The tools resulted from the research was named as 'Rapid Project Builder'. The tool has the capability of auto-generating the code based on the UI specifications stored as an XML Meta file. Generating code in multiple languages which included C++, java, etc. and use of Drag and Drop (DND) approach were some added features with the proposed tool. The DND approach [14] which was more popular among UI designers for its ability to allow users to drag and drop UI components for designing an UI. The proposed tool also had the ability to auto-generate database with tables and validation codes. As a conclusion, the researchers were able to suggest an efficient way to make the software development process simple, efficient, and robust using approaches of ACG.

The research paper published by M. Ko and Y. B. Park in 2017 [15] proposed method to support automatic code generation through UML modelling and design guide. The proposed model consists of 3 parts. They are HIPI Design Guide, modelling tool, and

Code Generator. ***HIPI Design Guide*** – Guide examples for applying HIPI Map-Reduce. ***Modelling tool*** – Generate model. ***Code Generator*** – Generate source code based on the generated model. The researchers designed a model with structural and behavioral viewpoints through UML and gave a design guide for modelling. And they proposed a method to generate source code based on the created model. As a conclusion, the code reusability and code design quality has been improved by using these model-based development methods, and due to the automatic source code, the development time and errors have also been reduced.

The research conducted by Chowdhury, Anirban in 2019 [16], explained the problem related to the rough sketches drawn by pen or a pencil. The aim of this research paper is to design and develop UI stencil to help mobile UI designers to create quick, easy, and visually pleasing paper prototypes. For this research, a total of three UI stencil prototypes were designed and developed for usability testing to ensure the user acceptance of UI stencil. As an outcome, it was able to observe users utilizing comparatively less time for UI screens using 'Stencil-3' than the time taken for hand drawings of the same UI screens.

In 2019 the research work by B. Aşıroğlu et al [17] explained the process of generating HTML codes from mockup images by using ML techniques. The main aim of this research to automatically generate HTML codes for the given mockup UI image was achieved by using computer vision techniques and deep learning techniques. The system had 96 % method accuracy and 73 % validation accuracy. Images provided from Microsoft AI Lab was used as the dataset for training the models in the system. HTML builder script was used to generate HTML codes for the objects identified on the mockup UI. Object detection, object cropping, object recognition and HTML builder script were the main four consecutive principal stages of the research. The introduced system had the capability of generating frontend HTML code while it lacks the ability to generate backend code along with CRUD operations. This was a major feature lacking in their system which many businesses needed both frontend and backend code to be automatically generated from a code generator tool or system. As a conclusion they explained about another HTML code generator algorithm they are working on which is an improved version of the algorithm explained in this paper.

In 2020 Sermuga Pandian and Vinoth Pandian in their research paper [18], expressed the ideas on converting low fidelity UI sketches to higher fidelity UI sketches. This research paper highlighted the use of AI for UI components identification. The tool "MetaMorph" was introduced in this research which detects the constituent UI elements of low fidelity sketches. DNN UI element detector was used in identification of UI components. The researchers collected dataset that contains 6,785 hand drawn sketches of 21 UI elements, 201 hand drawn low fidelity sketches, and 125,000 synthetically generated low fidelity sketches. This research paper explained concepts such as taxonomy of UI elements, dataset, user satisfaction and design recommendations. This solution was engineered to classify a limited number of UI elements. Data imbalance is said to be addressed in our future work by collecting more UI element sketches digitally. MetaMorph was trained using 125,000 synthetically generated low fidelity sketches.

This research conducted by Pereira, Carlos Eduardo, Fábio Fagundes Lima, and João Pedro Granja [19] explained the process of transformation between UML models and meta models through the creation of canonical data structure. The creation of a metamodel to store the data that can be extracted from UML diagrams was proposed. The target platforms were Android, iOS and the development was done by using Java and Objective-C. The process begins with the XML generation from UML Models. Then XML is passed into the canonical structure which is done by a specific process where its entities are designed to support a particular modeling tool exportation format. Then it goes through semantic analysis process. It verifies if there are errors such as entities that will be mapped into a file with the same name and absolute path, attributes inside classes with the same name, variables which are invoked and not defined and inadequately defined types of data. The final phase is the generation modules for Android and iPhone. The problem with this approach is the dependency on Android and iOS. And generation of two different codes for the two platforms Android and iOS using Java and Objective-C respectively was considered as a drawback of the tool.

In the year 2020, a template-based code generator for web application was by Elec, Turk & Sci, Comp & Uyanik, Burak & Şahin, Veysel [20]. This paper identifies the problems faced by businesses from small scale to large scale in their need to improve the software development process of web applications. The introduced code generator

was integrated with real-life web application. Experiments were conducted to prove the effective integration of the introduced code generator with real-life web application. Table design, table creation, stored procedure generation, HTML template selection, plugin generation and finally HTML and backend code generations are major capabilities of the introduced tool. As conclusion of the research, explanation about the design, development, integration, and workflow process were described briefly. As closing quotes, the authors suggested in use of code generators the developer productivity gets increased.

In 2021 M. d'Amorim, R. Abreu and C. Mello in their research paper [21] explained the process of converting visual sketch into code. The tool was named 'Visual Sketching' that uses computer vision algorithms and ML techniques to convert visual sketch to code. The approach used in this research mostly fit for data scientists, but it is said that the tool can be practiced in any domain. The researchers found the lack of research in use of images to compile code snippets as an immediate gap that can be filled. They focused on this idea throughout the research paper by introducing new approaches and displaying the higher level of accuracy in generated code. As a conclusion they briefly described the capabilities of the tool under different conditions along with future research work. Improving the efficiency and accuracy of the generated code by fine tuning their model and approaches was the main challenge given to future researchers in this paper.

The online journal [22] explained the process of pre-programmed web page implementation for mockup images. The use of hand-drawn mock-ups as input and obtain structured HTML code as an outcome. Here the REMAUI algorithm [23] is used to recognize GUI components such as buttons and text boxes. MYSQL database is used to store uploaded images as input. The concepts used in this tool can be categorized as object detection and pruning (cropping), object recognition and HTML Builder. According to this journal, they were able to get 90% accuracy in generated java code. They also managed to identify the colors of the buttons correctly. But the inability to select template styles according to user preference was a drawback of this tool.

Table 1. 1 Summary of previous research.

Reference			Description
	Year	Topic	
[9]	2005	Informal user interface for graphical computing.	Expressed the ideas about a concept of sketch-based informal user interface for graphic computing, which can be characterized by two properties: stroke-based input and perceptual processing of strokes.
[10]	2008	UJECTOR: A Tool for Executable Code Generation from UML Models.	This automatically generated Java source code according to the given UML diagram. It combines both structural and behavioral features pf an object-oriented (OO) application.
[11]	2009	Using XML files to document UI applications.	This paper explains the process and need of storing UI specification in an XML file. The stored XML file is useful for future changes and modifications in UI.
[12]	2010	A model driven approach to build high effective and ergonomic mobile business applications.	Here code generation is done using Java and C# and elements are produced using individual templates.
[13]	2016	Automatic code generation with business logic by capturing attributes from user interface via XML.	The tool named 'Rapid Project Builder' that can generate code in user preferred language. It gives the approach of business embedded code but at a very high level which the approach needs to be enhanced and optimized to obtain the best results from the tool.
[15]	2017	Automatic Generation of UML Model-Based Image Processing Source Code in Hadoop Platform.	The Hadoop Map-Reduce platform is used to generate the source code. It basically focuses on design quality and the ability to reuse code.
[16]	2019	Design and development of a stencil for mobile user interface (UI) design.	Explain the problems related to the rough sketches drawn by pen or a pencil. The aim of this research paper is to design and develop UI stencil to help mobile UI designers to create quick, easy and visually pleasing paper prototypes.
[17]	2019	Automatic HTML code generation from Mock-up images using ML techniques	This system has the capability to convert UI mockups to HTML code. Object detection, object recognition and java file creator are the major components used in code generation.

[18]	2020	MetaMorph: AI Assistance to Transform Lo-Fi Sketches to Higher Fidelities.	AI (Artificial Intelligence) can be used to identify the UI sketches. “MetaMorph” was introduced in this research and that tool is used to detect the constituent UI elements of low fidelity sketches.
[19]	-	Code Generation based on Uml Model Transformation.	The transformation between UML models and meta models through the creation of canonical data structure.
[20]	2020	A template-based code generator for web application.	This code generator was capable enough to be integrated with real-life web applications. It generated HTML codes for the given UI in the application. The design, development, integration, and workflow were elaborated in this work.
[21]	2020	Visual Sketching: From Image Sketches to Code.	This tool has the capability to convert visual images to executable code by using machine learning models. Different approaches were used on each section of the tool for obtaining the highest accuracy such as document image segregation, text recognition, etc.
[22]	2020	pre-programmed web page implementation for mockup image	This ACG tool generates HTML source code using mock-up images. The techniques used here are REMAUI algorithm and Image Processing. Also uses bootstrap framework for styles.

1.3 Research Gap.

With all the evidence and research papers described in the previous section we can understand that active research is being done in the direction of ACG to propose a solution which can improve the software development process. With the existing tools, still software designers and developers face challenges to apply the available tools for their production environments. The main cause for minimum applicability of the tools, lies on the restrictions and limitations imposed with growing expectations of the software developers and changing requirements of the businesses. Most of the existing tools [1] focus on generating frontend code with their own specified language but in real scenarios both software developers and business expect the fully functional code with proper data flow storage and storage. In our solution, we are planning to implement an intelligent tool with new features and capabilities that can satisfy both software developers and business at its maximum.

In our proposed tool we can identify all the UI components in rough sketches. As well our tool can generate professional UI templates according to the rough sketches whereas the existing tools mainly focusing one specific UI component. With our presented 'UI Builder' solution, the component has a simple, efficient, and understandable web front-end code execution and provides a user-friendly and downloadable UI result compared to other web frontend code generations. The diversity of mobile devices raised the demand for the cross-platform development which the same code could be used to develop the application on multiple platforms. The frontend code generator – mobile module suggests generating the code for a cross-platform mobile application while considering the coding standards. The backend code generator component extends its abilities with a flexible and manageable database, showing the real use of generating RESTful web services and CRUD code to software developers and organizations by providing evidence and impact compared to the traditional software development process.

Table 1. 2 Concepts and basic features of different existing tools.

	Tools			
	Pix2Code [23]	Mobile business Application [12]	Rapid Project Builder [13]	Our solution – UI Builder
Basic concept	Automatically generate code from the given GUI screenshot image. Used the concept of generating English textual description for a given scene photography.	A model driven approach to build high effective and ergonomic mobile business applications.	Designers allowed to create UI designs; developers generate code according to their preferred technology.	Convert UI sketch to UI design preferred by user with fully functional frontend (web, mobile) and backend code.
Features	- Computer vision model to understand GUI screenshot. - Identification of UI components. - Language model for code generation. - No need of human labelled data.	- Code generation using Java and C# - Use individual templates to produce elements.	- DND approach for UI design. - Language specific code generation. - Object identification and isolation. - BL embedded code with restrictions.	- Object detection. - Easily understandable continuous integration pipeline. - BL embedded code generation and flexible database with proper validations. - Web and mobile frontend generators with user-preferred template design selection. - Support for cross platform development.
Limitations	- Model with few parameters and trained with small datasets. - HTML code generated. - Absence of backend code generation.	- Platform independent which means not available for cross-platform development.	- Absence of flexible and manageable database. - Difficulty in understanding of continuous integration pipeline. - Difficulty in understanding the generated code.	- System integration is quite complex.

Table 1. 3 Comparison with different existing tools.

Aspects	**Tool**			
	Pix2Code	**Mobile Business Application**	**Rapid Project Builder**	**Our solution (UI Builder)**
Accuracy in generated code. [Considered to satisfy only if its 80%]	✓	✓	✗	✓
Time utilized for entire code generation process.	✗	✓	✗	✓
BL embedded backend code.	✗	✗	✓	✓
Manageable database for data storage.	✗	✗	✗	✓
Adhering to best practices during code generation.	✓	✗	✗	✓
User preferences.	✗	✗	✗	✓
Usability and applicability.	✗	✗	✗	✓
Suitable for cross-platform	✗	✗	✗	✓
Responsive mobile development.	✗	✗	✗	✓
User Preferred template style selection. (web/mobile)	✗	✗	✗	✓

1.4 Research Problem.

The available ACG tools as mentioned in previous sections extends its features and capabilities to improve the software development process and make it efficient. With the limitations and restrictions on software developers and designers these tools find it difficult to get applied on real production environments of software projects. With the literature review, it was able to understand the cause for the above-mentioned fact.

The main research problem can be explained with respect to each component as follows,

1. Process of converting UI sketches to standard UI and document UI specifications on XML document.
2. Generate web frontend code using pre-defined code snippets.
3. Generate mobile frontend code for cross platform applications with real time review.
4. Generate backend code with database connectivity encouraging code reuse and use of predefined code snippets.

The above listed problems were further broken down into sub problems to achieve in it an efficient way.

- How the UI components in UI sketch is segmented and identified.
- Process of documenting UI.
- The process of converting XML to JSON
- Injecting style sheet into generating code.
- The process of converting XML to Flutter
- Customize the styles according to user preferences
- Flexible and easily manageable database with proper validation.
- Use of API's created using RESTful web services. [This widely improves the usability and code-reusability]

Note: A survey is being circulated among different user groups to collect more precise and clear requirements. Please find the analysis of the survey in the appendices section which is attached at the end of this report.

2. OBJECTIVES.

With the detailed elaboration of literature survey, it is possible to quote that active research is being done in the direction of ACG in software engineering. Our solution tends to introduce new approach and different methodologies in code generation with the main objective of increasing the efficiency of the software development process. To be precise, our main objective lies on the introduction of an intelligent ACG tool with extended capabilities and satisfying real needs of developers and business.

The 4 major objectives of UI builder can be listed as follows.

1. Object recognition, image enhancement and optimization of UI screenshot.
2. Web front end code generation using MERN-Stack Technologies.
3. Mobile front end code generation for cross platform application.
4. Backend code & database generator using forward engineering systems.

2.1. Main Objective.

The main objective can be summarized as the introduction of a new code generator tool which can convert UI sketches into standard UI and generate fully functional frontend and backend code. The absence of backend code generation acts a gap in existing code generator tool which is filled with our tool ‘UI Builder’. Meanwhile the UI generator component which resides inside the tool is expected to identify majority of the UI components using an improved machine learning algorithm. Mobile and web frontend code generation options will be provided for end users which can make the tool platform independent.

Like the functional aspects, the tool is expected to be efficient, accurate and robust in its process and execution. Increasing the user interaction and adding enhanced features is the main approach followed throughout the development of the tool. It is also planned to deploy the tool in real production environment of an organization and prove the advantages of using code generators for software development.

2.2. Specific Objective.

The main objective can be broken down into sub objectives in a way to achieve it in a regular manner. The sub objectives can be listed as follows.

- Clearly identifying the drawings in the rough sketch.
- Identifying the components in the sketch which are partially completed.
- Producing UI templates and XML metafile.
- Generate web frontend code according to UI / UX concepts.
- Follow proper coding standards and best practices in generated code.
- Focus on selecting user-preferred technologies for frontend implementation.
- Concern for code reusability, comments and code refactor.
- Customization of the provided code.
- Provide a better UI frontend code generation tool.
- Performance, accuracy, and user friendliness of the tool.
- Understandable continuous integration (CI) pipeline for backend code generation process.
- The texture of real handwritten code imposed on the generated backend code.
- Making the code generation process accurate and efficient.

3. METHODOLOGY.

As for the entire system, our research has four major components. The first component is UI generator component. The input of the component is a rough sketch which is drawn by the user. A standard UI will be generated as the result of the component. It will be converted into XML file for the mobile and web frontend code generator components. To start up this flow (converting rough sketches to UI templates) we have done a survey to collect sample UI rough elements. As for the next step we trained the model from the data set and finally we tested the model using the testing dataset. The following Figure 3.1 illustrates the input and output of UI generator component.

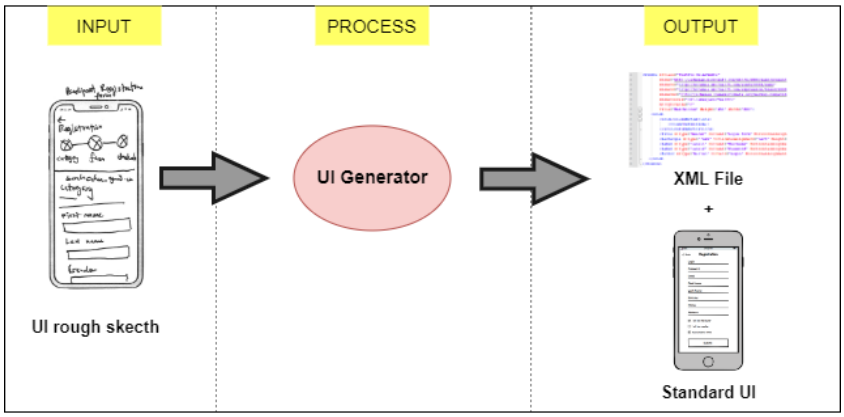


Figure 3.1 Input/Output - component 01

The web frontend code generator, which is the second sub-components of the UI Builder tool, is important for frontend code implementation in web applications. It uses the XML file generated from UI generator component as the input and generates the web frontend code as the output (Figure 3.2). The UI specification stored on XML meta file contains the names and alignments details of titles, labels, buttons related to a UI template form.

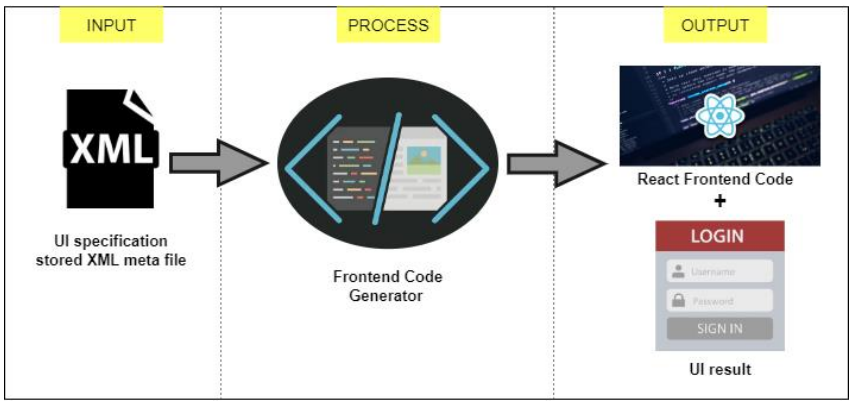


Figure 3.2 Input/Output - component 02

The third component is mobile frontend code generator. The XML file generated from the UI generator component will be used as the input of this component. As the output, it generates the mobile frontend code for cross-platform applications (Figure 3.3).

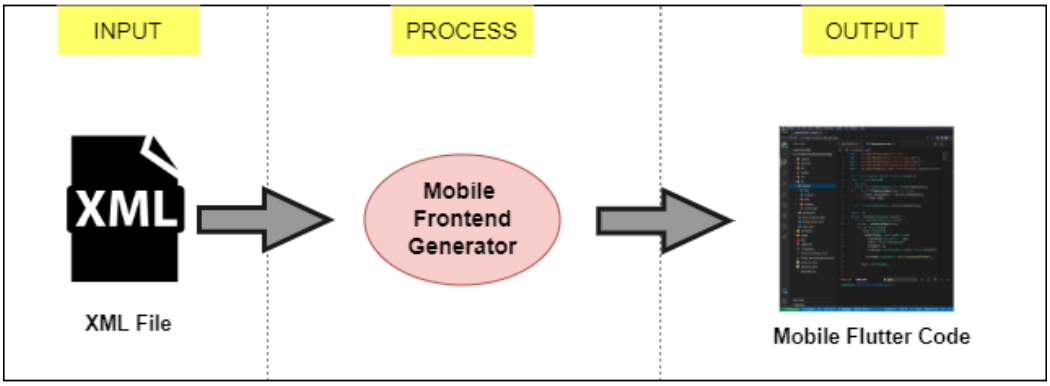


Figure 3.3 Input/Output - component 03

Backend code and database generator is the fourth component of the UI Builder. It receives the XML file from the UI generator component. It generates backend code and database table as the output. This tool itself has hidden complexity in constructing UI and generating a fully functional code with database tables. The diagram (Figure 3.4) helps the reader to get some more deeper understanding on each input and output of individual components in the tool.

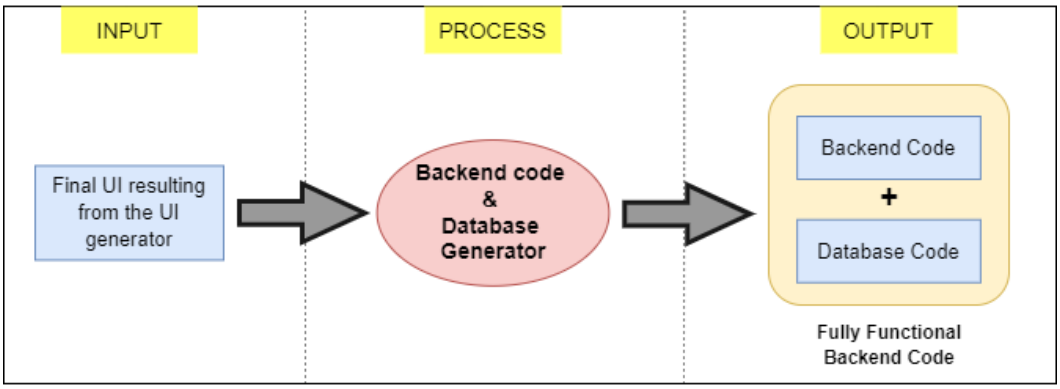
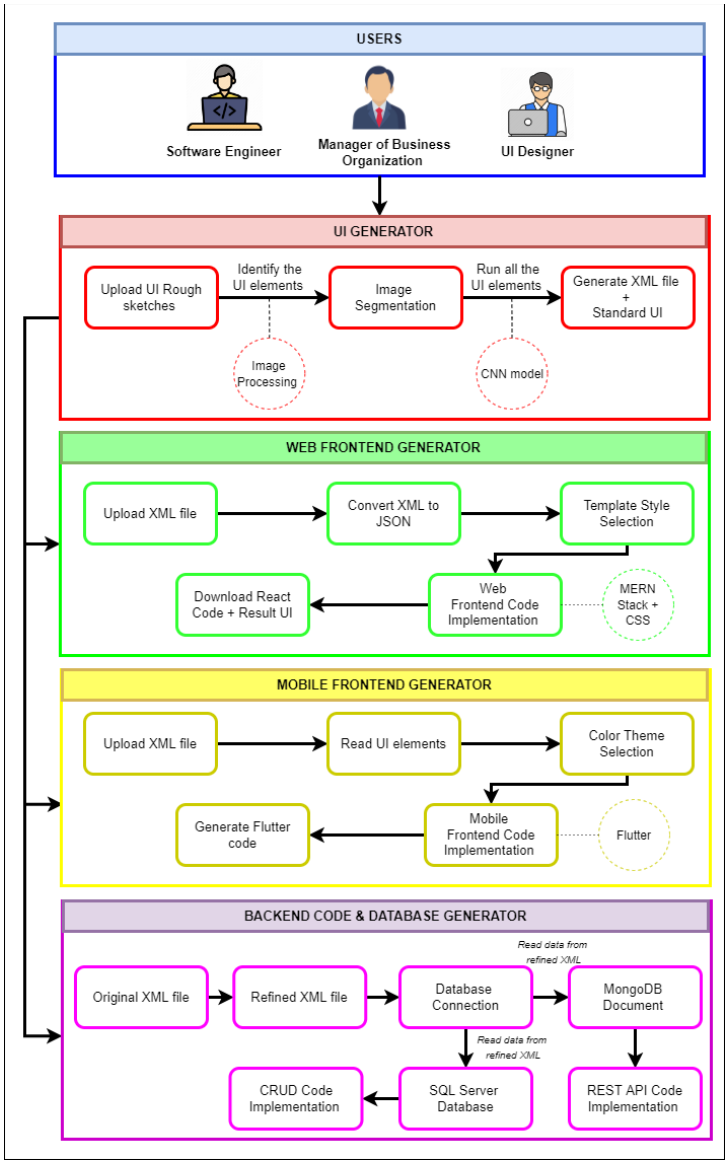


Figure 3.4 Input/Output - component 04

3.1. System Overview Diagram.



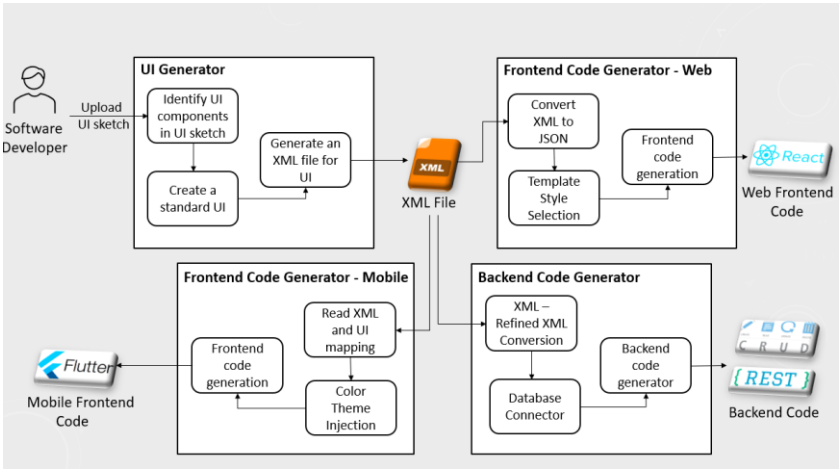


Figure 3. 5 System Overview Diagram

3.2. System Description.

The functionalities of the four components can be explained by providing the key idea on each of the inner modules. Since the input and output of the components are clearly explained in the previous sections, it is simple and easy to deliver the inner logic on the modules. As the initial step UI rough sketches must be converted into professional UI templates and produce an XML meta file for the use of other components. The second step will generate web front end code according to the XML meta file, as the third step this system will generate front code for the mobile UI. Finally, backend for both web and mobile UIs and a database according to the business function of the templates will be created.

In UI generator component the main goal is to convert the rough sketches to professional web and mobile UI templates. In addition to that an XML metafile will be created for both web and mobile UI templates. A survey was conducted to collect sample UI rough elements for this process. The next step involved in training the model from the data set which was collected and finally tested the data set using a CNN model and generate the XML file. For the UI elements extraction, the Image segmentation in OpenCV was used and Google Vision API was used to identify the labels in the rough sketches.

There are four subcomponents in web frontend code generator component. They are XML to JSON convertor, Template style selection, frontend code generator and frontend code downloader. Through XML to JSON convertor, the uploaded XML file will be formatted, and the details will be converted into JSON. Template styles or user preferred styles will be selected relevant to the JSON details by the template style selection module. Web frontend code will be implemented according to the JSON details and the styles. Finally using frontend code downloader module, frontend code and result UI could be downloaded. MERN Stack technology is used for the web frontend implementation.

Mobile frontend code generator consists of three subcomponents. They could be listed as read UI elements, color theme selection and code generation. The procedure of the function starts when the XML file was generated from the previous UI Generator Component. Then this function reads the UI elements from the XML file. The identified UI elements will be converted to an executable Flutter code through this

component. After that, the system allows the user to select the color theme for the interface. Otherwise, user has the option of going with the default color theme. Then the system converts the UI elements to the executable Flutter code.

Backend code generator starts with the XML to Refined XML convertor which is responsible to create the Refined XML file which describes the table structure by reading and mapping data from the uploaded original XML file. The resulting Refined XML file is thereafter used by the other modules throughout the backend code generation process. This acts as a common module to both the CRUD code generator and REST API code generator modules to generate code for specified business operation. The SQL Database connector is responsible to establish connection with the database created in MS-SQL Server. Once a successful connection is established the table creation module will be able to create a table in the connected database. The Refined XML file is read and processed to display the table structure to developers. The displayed table structure can be modified by the developer where the modifications are validated and then saved on the Refined XML file. Once the table is created on the database using the table creation module, the CRUD Code generation module becomes active. Depending on the table name CRUD operation names are auto populated. Developer can simply check on checkboxes to generate CRUD code for relevant business operations. A similar approach is used in the REST API Code generator. This module follows a sequential pathway. First the MongoDB URL and port needs to be specified by the developer. Then the model class is generated by reading the Refined XML file through a pre-defined template. Following the model class generation, the route class and the main backend JS file is generated. The generated code can be viewed by developer at any stage (Example: The model class can be viewed individually) of the code generation process. The generated CRUD or REST API code can be downloaded to personal machines which provides the ability to customize and adapt the code to different business organizations.

Table 3. 1 Sub-components and their respective outcomes

UI Generator	
Sub-component	**Outcome**
UI rough sketch element extraction by Image Segmentation in OpenCV	Identifying the UI elements in the rough sheet.
XML generator	Generated XML schema.
Standard UI generation	Standard UI.
Web Frontend Generator	
Sub-component	**Outcome**
XML to JSON Convertor	Formatted JSON File
Template Style Selection	Template Styles of the UI elements
Frontend Code Generator Module	Generated frontend code + UI result
Code & UI Downloader	Generated frontend code + UI result
Mobile Frontend Generator	
Sub-component	**Outcome**
Read UI elements	Retrieve UI elements details
Select color theme	Color theme of the UI elements
Generate code	Generated Flutter code
Backend & Database Generator	
Sub-component	**Outcome**
XML to Refined XML Convertor.	Refined XML File.
SQL Database Creator.	Established of SQL Database connection.
Table Creation Module.	Table created on connected Database.
CRUD Code Generator.	CRUD Code for selected business operations.
REST API Code Generator.	REST API Code for selected business operations.

3.3. Development Process.

3.3.1 UI Generator.

All the implemented interfaces related to the component are shown below [Appendix E].

- Identifying the UI elements in the rough sketch

At the initial step for this research partition the proposed tool needs to identify the UI elements in the rough sheets. To have that process our tool needs to have a proper strategy to identify the UI elements in the rough sketches. For this purpose, I am going to use Image Segmentation in OpenCV and Google Vision API for that process. In this phrase this research component needs to identify the image contrast, image value and image noise. Not only that, in this phrase the tool needs to be identified the UI elements that are partially represent in the rough sketch.

- **The process of the generating standard UIs according to rough sketches can be listed as follows,**
  - ✓ User needs to upload rough sketch to the application. – ***Only work by the user.***
  - ✓ Identifying the UI elements in the rough sketches using a CNN model.
  - ✓ Process the UI elements using an image processing technique and develops an XML document.
  - ✓ Map the rough UI elements to standard UIs.
  - ✓ Design the standard UI.

- **User needs to upload sketch to the application**

  This is the main unit of this research component. In this phrase all the identified UI elements mapped to corresponding wireframe components. To initiate the process, the user needs to upload the rough sketches to the application. Users need to draw rough sketches by hand, or any other tool and those rough sketches need to be uploaded to the application. Examples for the rough sketches given as below,

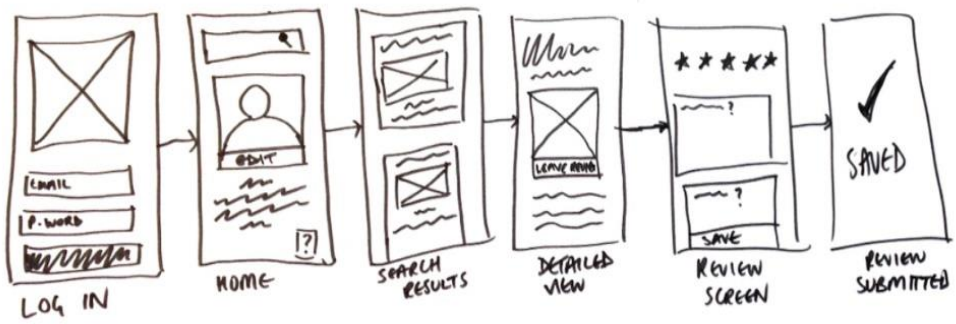


Figure 3.6 Sample rough sketches from the survey.

- Identifying the UI elements in the rough sketches using a CNN model.

To identify the UI rough elements in the rough sketches we have created a model using python TensorFlow and we have collected a dataset from a survey relevant for this research component. After collecting the dataset for this process, the model was trained by the sample dataset and tested the dataset.

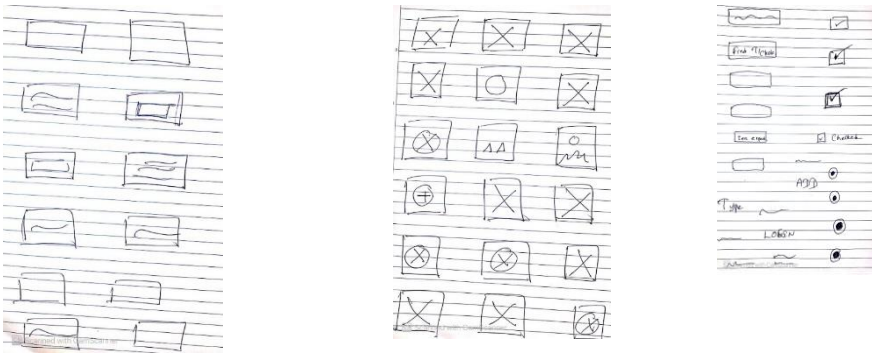


Figure 3.7 Sample dataset of the survey.

- Train the model by the dataset. (CNN Model)

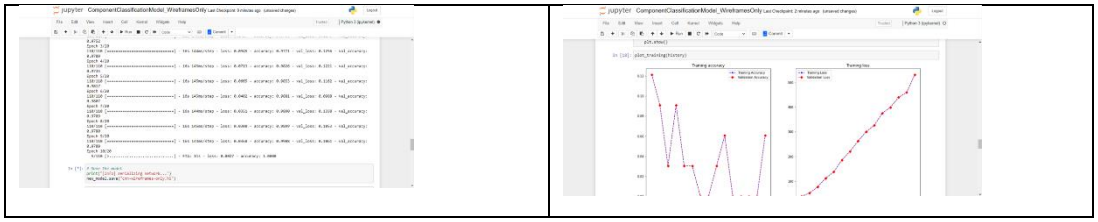


Figure 3.8 Trained Dataset

- Test the dataset

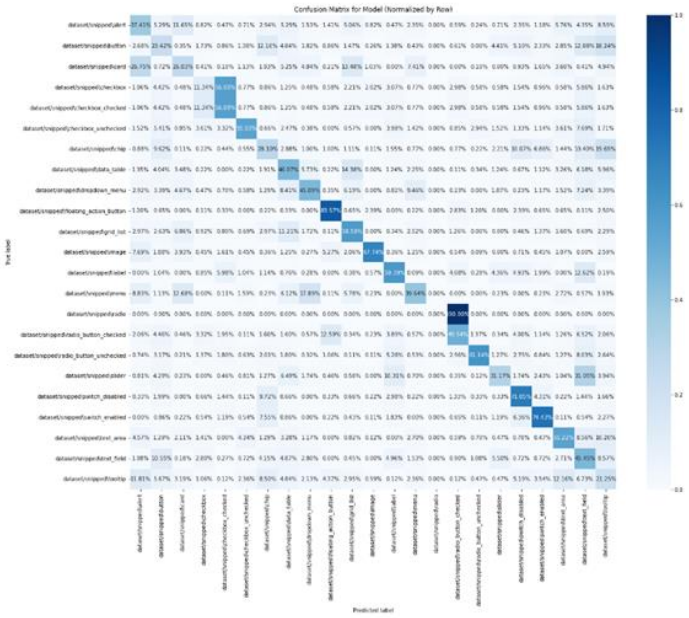


Figure 3.9 Tested Dataset

- Map identified UI elements to user data.

Once the UI elements in the rough is identified and separated using the created CNN model the group elements are displayed using a label. For an example if the UI sketch contains labels, headings, textbox, etc. Then the identified UI elements tag will be set accordingly. This in turn decides the fields in user data form from which the data needs to be obtained. The margin and alignment of identified UI component is determined from the data entered by user. This provides the facility for the users to change templates and visualize various UI alternatives. The below diagram shows the sample UI elements on rough sketch from which user data is obtained.

- Develops an XML document for the rough sketches.

Once the rough sketch is uploaded to the application and the UI components are segregated there after the standard UI generation process begins. During this process when generating frontend for each identified UI element in the rough sketch the XML generation is also injected. This ensures that each UI element identified from rough sketch is properly mapped to its corresponding XML tag. For example, the below table list down the sample mapping of UI element and the XML tag.



Figure 3.10 Convert rough sketches to XML file

- Map the rough UI elements to standard UIs.

This is the tricky point of this research component. Mapping the UI rough sketches to standard UI elements process is done in this phrase. From the resulted XML document, we mapped the rough UI elements to standard UIs. This process is done by an algorithm.

- Design the standard UI.

This is the final phase of this research component. In this phrase the users can have the standard UI for a particular UI rough sketch. The final output of this research component is to result an XML document and a standard UI for a particular rough sketch.

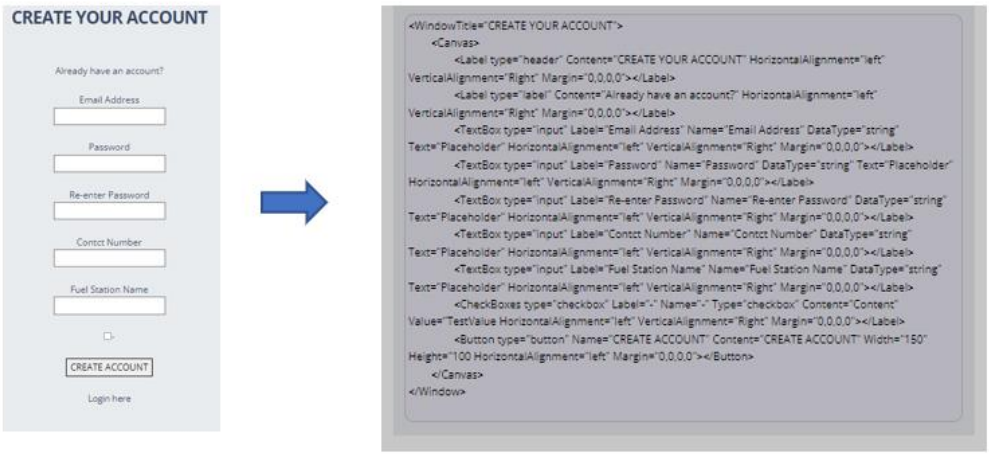


Figure 3.11 UI Generator process

3.3.2 Frontend Code Generator - Web.

All the implemented interfaces related to the component are shown below [Appendix F].

The web frontend code generator component is develop using 4 major sub-components.

1. XML to JSON Convertor
2. Template Style Selection
3. Frontend Code Generator
4. Frontend Code Downloader

- Convert XML to JSON

First, the user uploads the relevant UI specification stored XML file to the system using browse or drag and drop options. Then the uploaded XML details are automatically displayed by the system for the user's convenience. Then the XML details can be converted into JSON details through a button. Finally, the JSON details are automatically displayed to the user.

Special considerations in sub-component 1 implementation

1. Only files with .xml extension can be uploaded to the system.
2. Other buttons are disabled until the relevant XML file is uploaded to the system.
3. After all the tasks of sub-component 1 are done successfully, user can go to the next steps.

Why convert XML file details to JSON details?

The technologies we use for this web frontend generator implementation are MERN stack technologies. The popular React frontend library among software developers and designers is directly used for the frontend implementation. XML meta details are not closely related to React implementation. But JSON format details are closely related to React. Therefore, the XML meta details received as input are converted to JSON format for the convenience of future frontend implementation

- Template Style Selection

The function of the second sub-component is to select the template styles or the desired styles for the code implementation related to the JSON details selected by the user. According to the literature survey we talked about earlier, the existing tools do not allow to select template styles according to the user's choice. The user must get the default styles provided by the system. Therefore, this step is a unique part of our UI Builder tool.

With this sub-component, the user has complete freedom to choose the system-provided template designs or styles as per his/her preference. The selected template styles are automatically stored in the database after clicking the 'submit' button.

The content of the style form that the user can choose are given below.

Table 3. 2 UI specification details included in the XML File

Field	Style Type
Form	• Background Color Styles • Border Color Styles • Other Border Styles
Header	• Font Size Styles • Bold/UnBold Styles • Font Color Styles
Label	• Font Size Styles • Font Color Styles
Input Field	• Border Color Styles • Other Border Styles • Border Radius Styles
Button	• Background Color Styles • Font Color Styles • Border Radius Styles

Special features of sub-component 2 implementation

1. The provided templates can be enlarged and viewed by the system.
2. Template styles provided by the system can be added.
3. Fields can be updated according to user preferred styles.
4. It is not possible to navigate to the next steps until the style selection is complete.

- Web Frontend Code Generator

This is the most important sub-component of this web frontend generator component. Here the code implementation is done with the help of formatted JSON details and template styles submitted earlier. The advantage is the ability for user to generate and view frontend code just by clicking one button.

How does the internal implementation process work?

When using formatted JSON details for code implementation, attention is primarily focused on 2 main points. That is about JSON structure and content widgets. Here the JSON structure can be received in many ways such as Grid, Stack Panel, Canvas, so first it is checked by a condition to prevent errors at the beginning of the implementation.

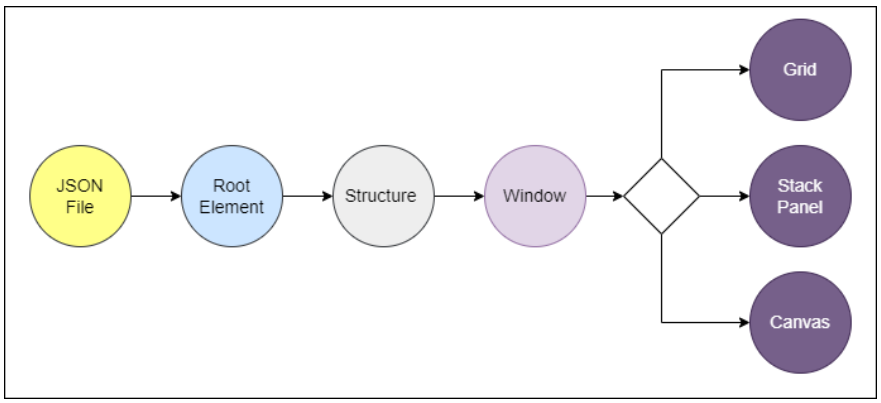


Figure 3.14 JSON structure

The second thing to consider is what content widgets are. Here we use only the widgets introduced by the system in our UI Builder tool, so all those related widgets are listed as a loop. Examples of widgets used in the application are given below.

1. Label List
2. Button List
3. Checkbox List
4. Textbox List
5. Checkbox Label List

The frontend code implementation is done after taking the above mentioned 2 points into consideration. Here the template styles stored in the database are embedded as inline styles to the same generated code. A clear understanding of the entire implementation can be obtained from the diagram below.

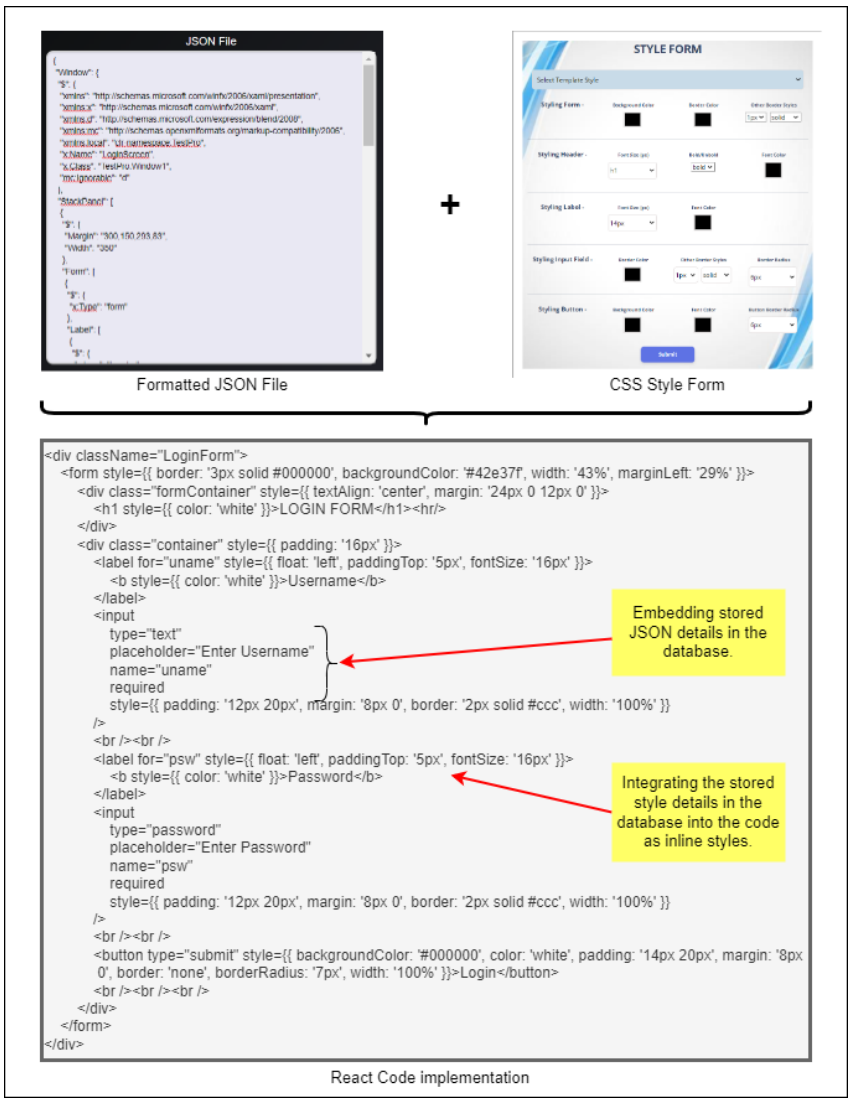


Figure 3.15 Web code generate process

One of the main objectives of generating our 'UI Builder' tool is to produce a unique ACG tool that is easy for the user to use and more than the existing tools. Taking that into consideration, we have introduced many features that are convenient for the user in this web frontend implementation process. Among them, one of the main features is that when the frontend code is generated, the system automatically displays the UI result related to that code. Thus, the user can get a clear understanding about the output.

- Frontend Code and Result UI Downloader

This is the last sub-component of the web frontend generator. According to the background survey conducted by us earlier, an important point found when studying the user needs is that the generated frontend code should be able to be downloaded. Considering the user needs, this sub-component allows user to download both the frontend code and the result UI template that they generate. Here, the frontend code can be easily downloaded as a java file and the result UI can also be downloaded to the local machine with the .png extension.

Special features of the entire web frontend development process

- Auto generated web frontend code.
- Allow template styles modifications.
- Simple and error-free implementation.
- Proper Coding Standards.
- Proper Validations
    - ✓ Display of success and error toast messages.
    - ✓ Validated button navigations.
- User friendly and attractive interfaces.

3.3.3 Frontend Code Generator - Mobile.

All the implemented interfaces related to the component are shown below [Appendix G].

- Read UI Elements

When the user inputs the UI sketch, it is converted to an XML file. The mentioned XML file will be the input for this component. The XML file consists of the details of UI elements. As the initial step the components reads and extracts the details of the XML file which has the UI elements details. The extracted details of the elements will be displayed in a data grid view to the user as shown in Figure 3.16. The process is happening with the help of the Net Language-Integrated Query library which contains the classes for LINQ to XML [16]. The XML file contains all the details of UI elements, but the user can only see the name and the type of the UI element. As the LINQ library [17] allows to read the attributes of each element separately [20], the system can filter the necessary attributes and display them.

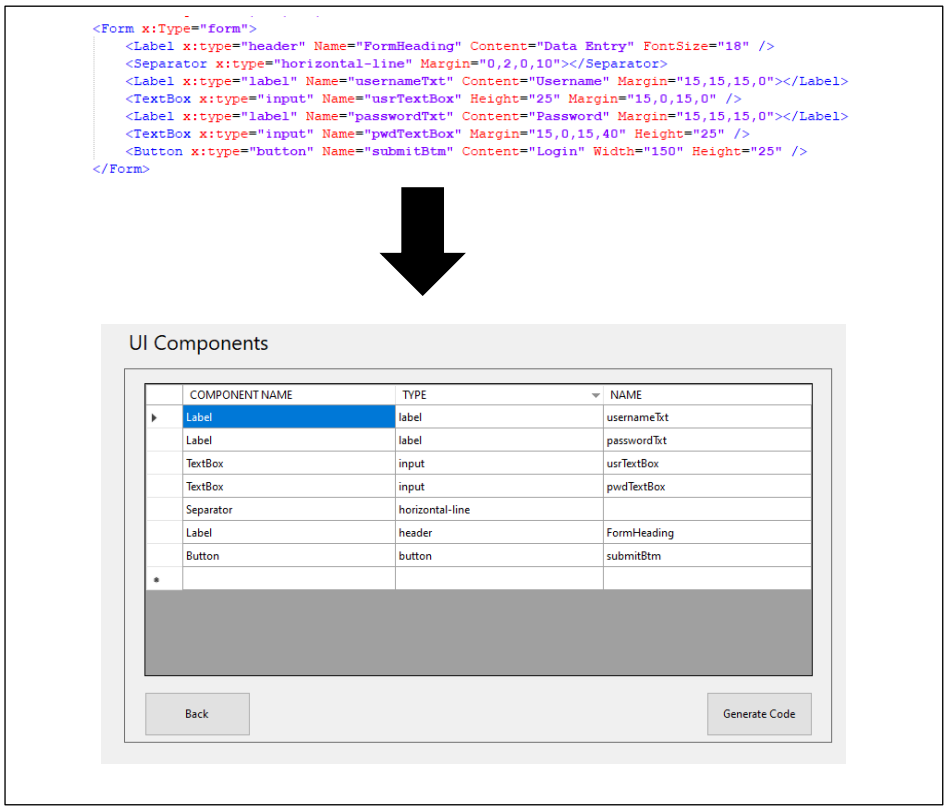


Figure 3.16 How system reads the UI elements from the XML file

The first node, as the first node of the XML file it will be noted as a form if the interface is a form. If the first node is the form node, then the system displays the child nodes of the form in the data grid view in the system interface after omitting the form node. Else all the nodes will be displayed on the screen without omitting any node.

Once the XML file is uploaded to the component, it is first read and processed to verify and validate its format. If the format is valid and according to the requirements of the component, then the user is allowed to move forward with the mobile front-end code generation process. Each tag in the XML file is mapped to its corresponding UI element which is later used for the code generation process. Developers get the opportunity to upload the XML file obtained from UI generator component or create their own XML file adhering to the guild lines specified by the front-end mode component.

The component also handles the reusability of generated code and helps developers to organize the output obtained from the component. For example, all the generated mobile front-end code is stored in the user's local machine on the path which they prefer the mobile front-end code generator folder should be created. The component is also capable of reusing the same folder whenever the user tries to generate code in future.

- Color Theme

The color theme for the interface that the user will be expecting can be chosen by the user. The user is allowed to choose one of the themes which was given by the system. The themes are pre-defined and stored in the system. Each theme consists of the colors for specific UI elements. It costs only one button click to select the preferred color theme by the user for a specific interface.

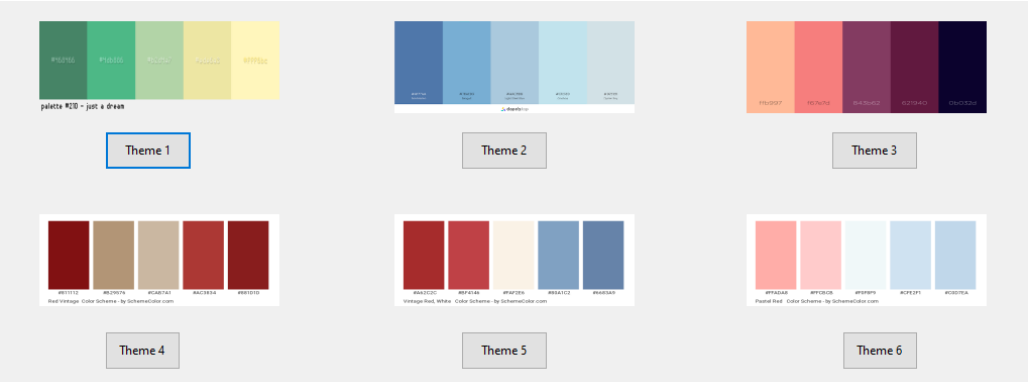


Figure 3.17 The color themes available in the syste

The following diagram shows the color of the component according to each color theme.



Figure 1 – How colors are mapped with each UI component

- Code Generation

As the last step of the procedure of mobile frontend code generation, Flutter code will be developed according to the color theme that user has selected for the interface. The code for each element is stored in the system. During this step, UI elements will be mapped with the code segments. To complete the generation of the whole interface, the system will go through each element which was identified during the initial step. The system will extract the data which is related to the color theme that the user has previously selected and match them with the UI elements in the user interface.

The user will be able to view the generated code in the system itself, and a txt file containing the code will be generated as well. Therefore, users can view the code through the system as well. The generated code will be a formatted code with the correct alignments and no errors will be found in the code. The generated code can be executed using any application or an environment.

3.3.4 Backend Code Generator.

All the implemented interfaces related to the component are shown below [Appendix H].

The backend code generator component is build using 5 major sub-components. They can be listed as follows:

1. XML to Refined XML Convertor.
2. SQL Database Connector.
3. Table creation module.
4. CRUD code generator.
5. REST API code generator.

- XML to Refined XML Convertor.

This module in backend code generator components plays a significant role in proper functioning of all other modules. The primary goal of this module lies on constructing the refined XML file for the uploaded original XML file. The uploaded original XML file contains UI specifications which includes, UI components and their relative positions. But backend code generator component is focused on determining the table structure on which the data needs to be stored, therefore the original XML file obtained need to be processed before moving forward in the backend code generation process. The following list outline the process of creating refined XML file from the uploaded original XML file.

1. User (developer) uploads the original XML file.
2. Module verifies and process the uploaded XML file.
3. Once processing is completed, the data is displayed in a data grid for the user.
4. Then the user provides the desired table name and path to store the Refined XML file.
5. The module checks the path and then the Refined XML file is created and displayed. Parallelly the Refined XML is stored in the specified path.

The above steps give a clear idea in the process followed inside the module. Using the above idea, it's able to understand the approach used in the conversion process. Once the XML file is uploaded, the implemented code read the entire XML file and map XML tags to column name and their attributes to datatypes and constraints. Mapping

between the original XML and Refined XML can be explained through an example. Consider “Customer Signup UI” which is documented in XML file. Once the XML file is uploaded to the backend code generator component reads and creates the refined XML file. Figure 3.18 shows the resulting refined XML file and Figure 3.19 shows a sample mapping done on the “Textbox” UI component.

NOTE: The XML file is more simplified to deliver the concept easily but in real scenario more complex XML files are used.

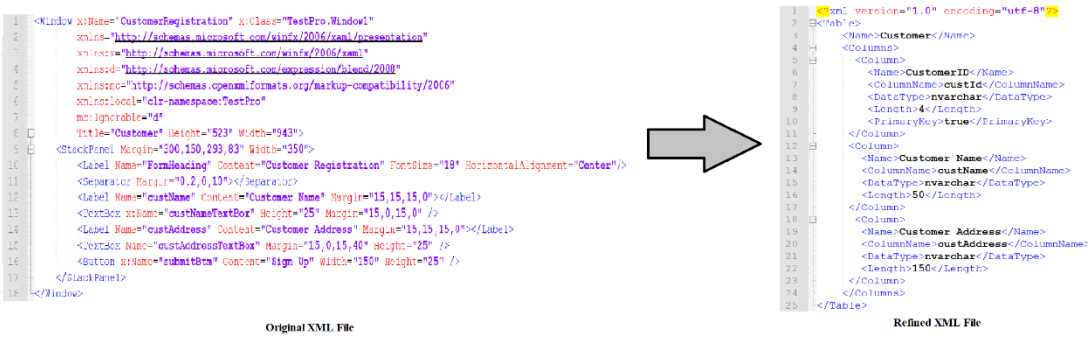


Figure 3.18 Conversion of XML file to Refined XML File.

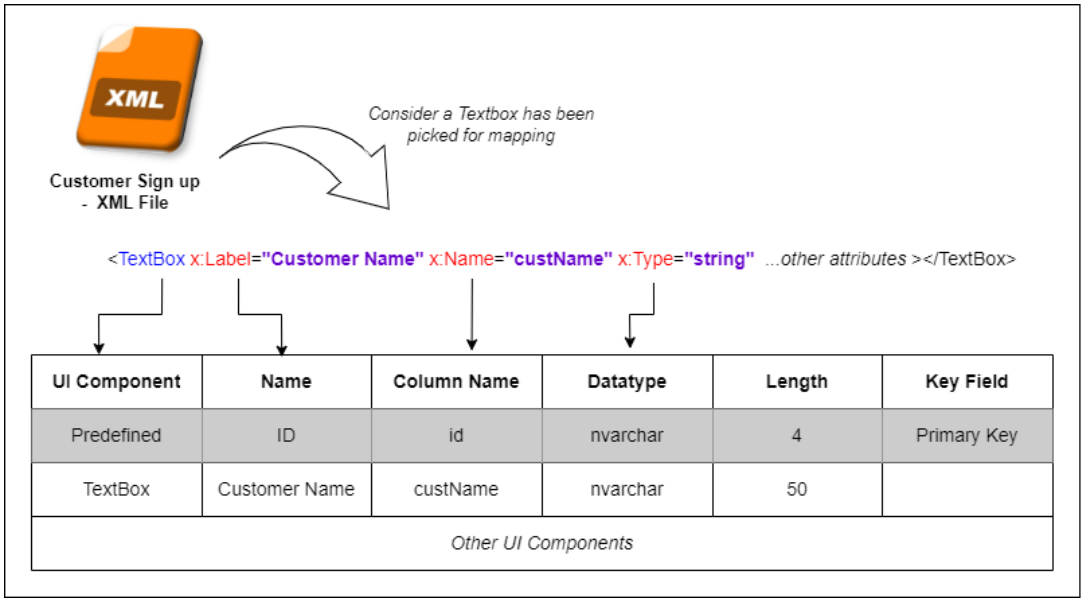


Figure 3.19 Refined XML file mapping on table structure

- SQL Database Connector.

With the creation of Refined XML file, the SQL Database Connector module becomes active. The responsibility of the module is establishing a successful connection with the SQL database. The user (developers) utilizes features of this module when they intend to generate CRUD code. The capabilities of the modules can be listed as follows:

1. Generate SQL Connection string.
2. Test database connection with generated/provided connection string.
3. Verify and establish connection with the database along with a session.

❖ Generate SQL Connection string.

The SQL Server connection string can be generated or manually provided by the user. The application provides the feature of generating a connection URL by selecting the database that the user wishes to establish a connection. The generated connection URL is displayed to the user and thereafter it can be used to connect to the database.

❖ Test database connection with generated or provided connection string.

The entered or generated connection string is tested against certain validation rules. The server's name, database name and the integrated security options are mainly checked with the connection string. Once the verification is completed the application tries to establish a connection with the database and provides the status of database connection to user.

❖ Verify and establish connection with the database along with a session.

Once the database is connected with the application, the database name is displayed for user confirmation. Parallelly the table creation module becomes active, and the user is allowed to create a table on the connected database.

- Table Creation Module.

This module is comparatively complex and performs various functions during the table creation process. The primary goal of this module lies on creating table on the connected database. The Refined XML file acts as the input for this component on

which the table structure is explained. Before explaining different subunits in the module, it is important to understand the activities performed at high-level. The following list shows the important activities performed in the module.

- Read the Refined XML file and display the table structure in data grid view. The following Figure 9 shows the table structure displayed for "Customer Sign up UI" considered in previous examples.

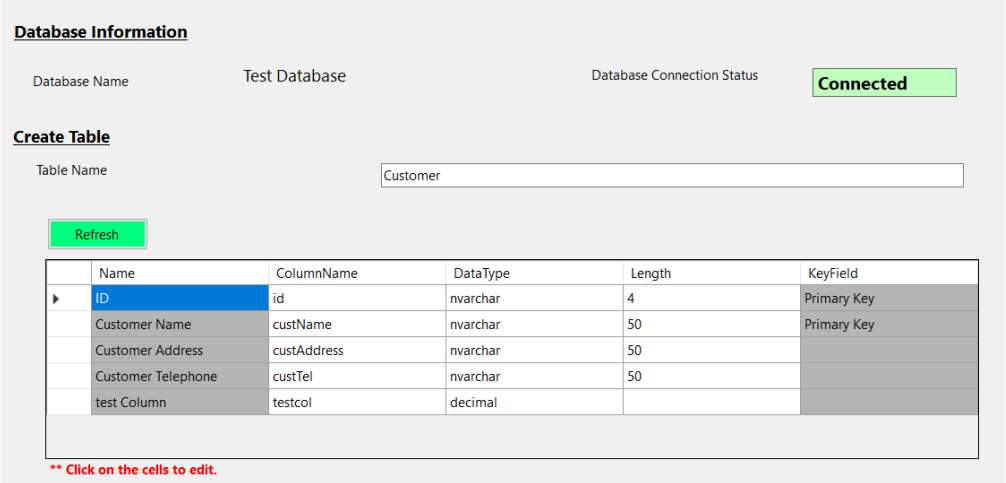


Figure 3.20 Data read from Refined XML and displayed in data grid.

- The table structure can be modified or changed by the user (developer).
- Modifications are verified and validated.
- Other supporting features which are needed during table creation is provided to users.
- User can create the table and the SQL query used for table creation is displayed.

The 'supporting features' keyword used in the previous paragraph can be categorized as follows:

1. Modify table columns [Add / Update / Remove].
2. Add new keys or modify existing keys.
3. Drop tables in Database.

- Modify table columns.

After construction of the refined XML file, the developer can modify or change the structure of refined XML file ***only*** during the table creation process. The table structure displayed on the data grid can be directly modified by the user according to business requirements. The Column Name, Datatype and length columns can be modified by the user while the Name and Key field columns are read-only. These read-only fields can be modified using other supporting features whenever needed. Each modification made by the developer on the data grid must be validated before updating the refined XML file. Removing existing columns and adding new columns (Foreign key/normal column) is the unique feature of this module.

- Add new keys or modify existing keys.

Composite keys and foreign keys are mainly focused on the subunit. The default primary key on the refined XML file can be replaced with a new key using this logic. A validation exists on unnecessary removal of existing keys which can directly break the table structure. The default primary key can be replaced only in the presence of a composite key in the table structure.

- Drop tables in Database.

Dropping tables in the connected database is made possible with this functionality provided from the subunit. Since this is a destructive change, the execution of this requires higher privileges and user confirmation to successfully drop a table from the database. Foreign key references included in the table is an important validation aspect fired during this process.

The below shown diagram (Figure 3.21) clearly explains the process of table creation process along with the subunits and user processes.

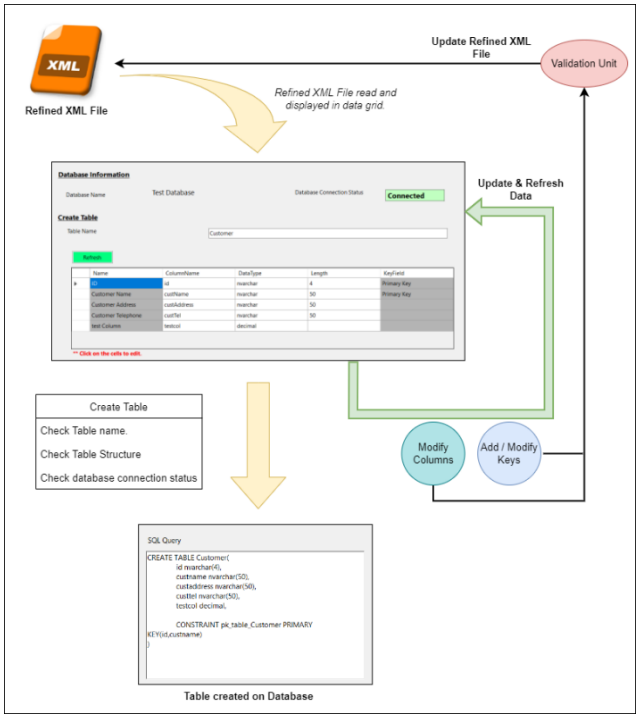


Figure 21. Table creation process

- CRUD Code Generator.

The module becomes active after execution of table creation module. The CRUD operation name is auto populated depending on the table name. Here the table name is checked against two scenarios. First, the table name on Refined XML file is checked and then the table name created on the database is checked.

The following table (Table 3.3) provides a stepwise explanation on the process followed in the CRUD code generator module.

Table 3.3 CRUD Code generation steps.

STEP 01	Read refined XML file and store the field name and datatypes in an array.
STEP 02	Construct method signatures and parameters as required by the CRUD operation code template.
STEP 03	Inject the field name and data types into the pre-defined template. (Ensure the code is properly organized and structured)
STEP 04	Generate comments needed for the code.
STEP 05	Merge comments with generated code and display to user.

The following code segment (Figure 3.22) from the application is provided to get a better understanding on the usage of pre-defined template used in the code generation process.

```
public String buildAddMethod(String QueryFieldsNames, String insertQuery, String methodParam)
{
    String methodSignature = "public void " + addOpName + "(" + methodParam + ") {\n";

    String statement = "\t" + "try{" +  "\n";
    statement = statement + "\t\t" + "SqlConnection con = new SqlConnection(@" + connString + ");" + "\n";
    statement = statement + "\t\t" + "SqlCommand cmd;" + "\n\n";
    statement = statement + "\t\t" + "String sql = " + insertQuery + "\n";
    statement = statement + "\t\t" + "con.Open();" + "\n";
    statement = statement + "\t\t" + "cmd = new SqlCommand(sql, con);" + "\n\n";

    String[] splitfields = QueryFieldsNames.Split(',');

    for (int i = 0; i < splitfields.Length; i++)
    {
        String val = '"' + splitfields[i] + '"';
        statement = statement + "\t\t" + "cmd.Parameters.AddWithValue(" + val + ", " + fields.ElementAt(i) + ");" + "\n";
    }

    statement = statement + "\n\t\t" + "cmd.ExecuteNonQuery();" + "\n";
    statement = statement + "\n\t\t" + "Console.Write('Date Added Successfully!');" + "\n\n";
    statement = statement + "\t" + "} catch(Exception ex) {" + "\n";
    statement = statement + "\t\t" + "Console.Write(" + '"' + "Exception Occurred!" + '"' + ");" + "\n";
    statement = statement + "\t" + "}" + "\n";

    methodSignature = methodSignature + statement;

    methodSignature = methodSignature + "\n" + "}" + "\n";

    return methodSignature;

}
```

Figure 3.22 Code used to generate 'INSERT' CRUD operation.

- REST API Code Generator.

This module is responsible to handle the REST API code generation process. The module is built using the following subunits where each one of them work together to deliver a complete code.

1. MongoDB Connector.
2. Model class Generator.
3. Route Class Generator.
4. Main Backend JS Generator.

❖ MongoDB Connector.

This subunit focuses on establishing connection with MongoDB Cluster. The MongoDB connection URL and the port are expected as user inputs. Once the user (developer) enters the MongoDB connection URL the connection is tested before moving forward in the code generation process.

❖ Model class Generator.

Once the connection with the database is established the model class which is a definition of schema on which the collection (MongoDB table) is created. The list of fields and their relevant datatypes are auto filled in the model class by reading the refined XML file.

❖ Route Class Generator.

Following the model class generation, the route needs to be generated. Comparing to other tasks in the REST API code generation process, the route class code generation is complex.

The Figure 3.23 shows the code segments used to create “POST” route for HTTP POST method.

```
public String constructPOSTRoute()
{
    String routeName = "/add" + generateREST_API_Code.tableName;

    String methodSignature = "router.route(" + "'" + routeName + "').post(async (req, res) => {" + "\n";

    String statement = "\t" + "if(req.body) {" + "\n\n";
    statement = statement + "\t\t" + "const " + generateREST_API_Code.tableName + " = new " + generateREST_API_Code.modelClassName + "(req.body);" + "\n";
    statement = statement + "\t\t" + "await " + generateREST_API_Code.tableName + ".save()" + "\n";
    statement = statement + "\t\t\t" + ".then(data => {" + "\n";
    statement = statement + "\t\t\t\t" + "res.status(200).send({data: data});" + "\n";
    statement = statement + "\t\t\t" + "}).catch(error => {" + "\n";
    statement = statement + "\t\t\t\t" + "res.status(500).send({error: error});" + "\n";
    statement = statement + "\t\t\t" + "});" + "\n";
    statement = statement + "\t" + "}" + "\n";

    methodSignature = methodSignature + statement;
    methodSignature = methodSignature + "});" + "\n\n";

    return methodSignature;
}
```

Figure 3 Code segment responsible for ‘POST’ code generation

❖ Main Backend JS Generator.

The REST API code generation process completes with generation of the main backend JS file. The generated route class is called in the main JS file, which makes it possible to use the created REST web services. The MongoDB connection URL and port provided at the beginning of the code generation process is called in the relevant place of the code. The necessary libraries are imported, and the entire code is organized and structured.

Once the code is organized, it is checked for code syntax and proper comments are added in the necessary code section. Once the code is fully generated, it is displayed to the developer and downloaded into the project folder.

3.4. Testing

3.4.1 UI Generator – Testing

Table 3. 4 Unit Testing - component 01

Uploading rough sketches to application	
Test Case ID	01
Pre-requirements	User has a rough UI sketch image.
Description	After uploading the image by the user, it should be displayed to the user.
Test Input	Rough UI sketch image.
Test Procedure	1. Upload the UI sketch file by browsing or dragging. 2. Click the 'Upload' button
Expected Outcome	The uploaded UI sketch will be displayed on the application.
Validations	✓ Before uploading the UI sketch, all the buttons on the front page are disabled. ✓ Checking that the uploaded file extension is correct. (Only png, jpg files are valid.) ✓ Only one file can be uploaded at a time. (Cannot upload multiple files.)
Test Output	The uploaded UI file was successfully displayed.
Test Result	PASS
Identifying UI element Module	
Test Case ID	02
Pre-requirements	UI sketch is needed to upload, and the model should be trained and tested properly.
Description	Model should be capable to identify the UI elements in the rough sketch.
Test Input	None
Test Procedure	Click ‘Submit’ button.
Expected Outcome	Properly identified the UI components in the rough sketch.

Validations	✓ Check the backend of this tool to ensure that whether UI elements are identifying or not.
Test Output	The UI components of the rough sketch are identified clearly by the model.
Test Result	PASS
Convert UI elements to an XML file module	
Test Case ID	03
Pre-requirements	UI elements on the rough sketches are needed to be identified by the model.
Description	Converting the rough UI elements to an XML file.
Test Input	UI Sketch
Test Procedure	1) Click the 'Generate' button
Expected Outcome	A properly arranged XML file
Validations	✓ The UI sketch need to be uploaded to the system before this test.
Test Output	Generate an XML file.
Test Result	PASS
Checking the accuracy of the resulted XML file.	
Test Case ID	04
Pre-requirements	• There should be a prepared correct XML file relevant for a rough sketch.
Description	Model should be capable to identify the UI elements in the rough sketch and the algorithm created to generate the XML files is cable to generate a correct XML file for a particular rough UI.
Test Input	Rough sketch. .
Test Procedure	1. Click "Generate" button.
Expected Outcome	Accurate XML file for uploaded UI sketch.
Validations	✓ Check the backend of this tool is working and the model is running.
Test Output	Accurate XML file for uploaded UI sketch.
Test Result	PASS
Checking the dynamically change of User Input form fields according to different UIs.	

Test Case ID	05
Pre-requirements	There should be different UIs. (Rough Sketches)
Description	Testing whether the User Input form fields dynamic nature. (Changing the User Input fields)
Test Input	Different UIs (Rough Sketches)
Test Procedure	Upload different UIs in different approaches. (Different times)
Expected Outcome	Dynamically changed User input form.
Validations	UI components need to be identified to display the user input form.
Test Output	Dynamically changed User input form.
Test Result	PASS

Table 3. 5 Integration Testing - component 01

Identifying UI element Module	
Test Case ID	01
Pre-requirements	The UI sketch needed to be uploaded to the system.
Description	Clearly identifying whether the model identifies the UI components in the rough sketch.
Test Input	Rough UI sketch (Image)
Test Procedure	1. Upload the UI sketch by browsing or dragging. 2. Click the 'Submit UI Sketch button. 3. Automatically display uploaded UI Sketch Image details in a text area.
Expected Outcome	Identified UI elements.
Test Output	Identified UI elements.
Test Result	PASS
Convert UI elements to an XML file module	
Test Case ID	02
Pre-requirements	UI elements on the rough sketches are needed to be identified by the model.
Description	Converting the rough UI elements to an XML file.
Test Input	UI Sketch
Test Procedure	1. Click the ‘Convert to XML file’ button
Expected Outcome	Generate an XML file.
Test Output	Generate an XML file.
Test Result	PASS
Test Output	Downloading the zip file containing web Frontend code and result UI.
Test Result	PASS

3.4.2 Web frontend Generator – Testing

Table 3. 6 Unit Testing - component 02

XML to JSON Convertor Module	
Test Case ID	01
Pre-requirements	Original XML file obtained from UI Generator Component.
Description	After uploading the original XML file, it should be converted into a formatted JSON file.
Test Input	Original XML file
Test Procedure	1. Upload the XML file by browsing or dragging. 2. Click the 'Submit File' button. 3. Automatically display uploaded XML details in a text area. 4. Click the 'Convert XML to JSON' button. 5. The converted formatted JSON automatically displayed in another text area.
Expected Outcome	The uploaded XML file should be converted to formatted JSON file.
Validations	✓ Before uploading the XML file, all the buttons on the front page are disabled. ✓ Checking that the uploaded file extension is correct. (Only XML files are valid.) ✓ Only one file can be uploaded at a time. (Cannot upload multiple files.) ✓ Automatically displayed XML file details and formatted JSON details are read only.
Test Output	The uploaded XML file was successfully converted to formatted JSON file.
Test Result	PASS
Template Style Selection Module	
Test Case ID	02
Pre-requirements	Formatted JSON details should be stored in the database.
Description	Store the style details in the database by selecting the styles user prefer and submitting the from.

Test Input	Style form fields.
Test Procedure	1. Select the Color boxes, drop down fields in the style form. 2. Click the 'Submit' button.
Expected Outcome	Add style form details and those details should be stored in the database.
Validations	✓ All details in the style form are required. ✓ If the styles related to a field are not selected, the default style will be auto submitted.
Test Output	When the styles form was submitted, the style details were automatically stored in the database.
Test Result	PASS
Web Frontend Generator Module	
Test Case ID	03
Pre-requirements	JSON details and style details should be stored in the database.
Description	Generate Frontend Code for web applications.
Test Input	None
Test Procedure	1. Click the 'Generate Code' button. 2. View the generated React code.
Expected Outcome	After clicking the button, the generated code will be displayed in a text area.
Validations	✓ Disabling the text area field before clicking the button. ✓ Provide read/write option to update code details.
Test Output	After clicking the button, the generated front-end-code will be successfully displayed.
Test Result	PASS
Code and result UI downloader Module	
Test Case ID	04
Pre-requirements	The React frontend code should be generated.
Description	Download the generated web frontend code and result UI image together.
Test Input	None
Test Procedure	1. Click the ‘Download Frontend Code’ button. 2. Download the web frontend code and resulting image as a zip file.

Expected Outcome	Downloading the generated frontend code and result UI image.
Validations	✓ Checking whether the data connection is stable.
Test Output	The generated frontend code and result UI image was successfully downloaded.
Test Result	PASS

Table 3. 7 Integration Testing - component 02

Template Style selection Module	
Test Case ID	01
Pre-requirements	Original XML file obtained from UI Generator Component.
Description	Select styles from style fields according to the JSON details stored in the database.
Test Input	Original XML file
Test Procedure	1. Upload the XML file by browsing or dragging. 2. Click the 'Submit File' button. 3. Automatically display uploaded XML details in a text area. 4. Click the 'Convert XML to JSON' button. 5. The converted formatted JSON automatically displayed in another text area. 6. Click the 'Submit JSON File' button. 7. Store JSON details in database. 8. Select template styles based on those JSON details. 9. Submit the style form.
Expected Outcome	Formatted JSON details are stored in the database. User selected template styles are stored in the database.
Test Output	Formatted JSON details are stored in the database. User selected template styles are stored in the database.
Test Result	PASS
Frontend Code Generator Module	
Test Case ID	02
Pre-requirements	Original XML file obtained from UI Generator Component.
Description	Generating the web Frontend-Code with the help of stored JSON details and Style details.
Test Input	Original XML file

Test Procedure	1. Follow same steps in Test case UIB01 (1 to 9) 2. Click the 'Generate Code' button. 3. Display JSON details and inline CSS embedded frontend code. 4. Automatically display the result UI related to the frontend code.
Expected Outcome	Displaying the web Frontend code and result UI.
Test Output	Displaying the web Frontend code and result UI.
Test Result	PASS
	Code and Result UI Downloader Module
Test Case ID	03
Pre-requirements	Original XML file obtained from UI Generator Component.
Description	Downloading JSON details and styles embedded frontend code and result UI.
Test Input	Original XML file
Test Procedure	1. Follow same steps in Test case UIB02 (1 to 4) 2. Click the 'Download Frontend Code' button. 3. Download the web frontend code and resulting image as a zip file.
Expected Outcome	Downloading the zip file containing web Frontend code and result UI.
Test Output	Downloading the zip file containing web Frontend code and result UI.
Test Result	PASS

3.4.3 Mobile Fronted Generator – Testing

Table 3. 8 Unit Testing - component 03

Upload XML file to the system	
Test case ID	**01**
Pre-requirements	XML file received from UI Generator Component.
Description	When the XML file is generated from UI Generator Component it can be uploaded to the system.
Test Input	XML file from UI Generator Component
Test Procedure	1. Click on upload XML file button. 2. Select the XML file. 3. Upload the file.
Expected Outcome	The system should display the alert "file is uploaded."
Validations	• After clicking on the alert message, the system should navigate the view UI components interface.
Test Output	The system displayed the alert "file is uploaded."
Test Result	PASS
Read UI elements in a form.	
Test case ID	**02**
Pre-requirements	XML file should be uploaded to the system.
Description	UI elements in XML file relevant to a form will be read and displayed to the user.
Test Input	None
Test Procedure	1. Click on 'ok' button on 'file uploaded' alert.
Expected Outcome	The UI components in XML file should be displayed in a data grid view.
Validations	• The form element should be omitted by the system.
Test Output	The UI components in XML file displayed in a data grid view.
Test Result	PASS
Color theme selection	
Test case ID	**03**
Pre-requirements	XML file should be uploaded to the system.
Description	Select a color theme for the user interface
Test Input	None
Test Procedure	1. Navigate to color theme interface.

	2. Select a theme for the interface
Expected Outcome	Code should be generated according to the theme that selected.
Validations	None
Test Output	Code was generated according to the theme that selected.
Test Result	PASS
Generate the Flutter code for a form interface.	
Test case ID	**04**
Pre-requirements	XML file should be uploaded.
Description	Generate frontend Flutter code for a form user interface.
Test Input	None.
Test Procedure	1. View components in the data grid view. 2. Select a color theme. 3. Click on 'Code Generation' button.
Expected Outcome	Code should be generated for the uploaded UI.
Validations	None
Test Output	Code generated for the uploaded UI.
Test Result	PASS

Table 3. 9 Integration Testing - component 03

Upload XML file and theme selection.	
Test Case ID	01
Pre-requirements	XML file should be uploaded to the component.
Description	The uploaded XML file should be read and displayed on data grid; user should be able to select the theme.
Test Input	XML file.
Test Procedure	1. Click on upload XML file button. 2. Select the XML file. 3. Upload the file. 4. Navigate to color theme interface. 5. Select a theme for the interface
Expected Outcome	1. Uploaded XML file should be read and displayed on the data grid. 2. All XML tags should be correctly mapped to the corresponding UI components. 3. User should be able to see the available color theme.

	4. User should be able to select one color theme.
Test Output	1. Uploaded XML file is read and displayed on the data grid. 2. All XML tags were correctly mapped to the corresponding UI components. 3. User able to see the available color theme. 4. User was able to select one color theme.
Test Result	PASS
Generate flutter code for selected theme.	
Test Case ID	02
Pre-requirements	XML should be uploaded, and the theme should be selected.
Description	Obtaining the flutter frontend code for uploaded XML file.
Test Input	None
Test Procedure	1. Click the “Code Generation” button. 2. View the generated frontend code. 3. Run the generated code using emulator or mobile device.
Expected Outcome	Generated frontend code. UI displayed on emulator or mobile device.
Test Output	Generated frontend code. UI displayed on emulator or mobile device.
Test Result	PASS

3.4.4 Backend & Database Generator – Testing

Table 3. 10 Unit Testing - component 04

XML to Refined XML Convertor Module	
Test case ID	01
Pre-requirements	Original XML file obtained from UI Generator Component.
Description	Once the original XML file is uploaded, it should be converted to Refined XML file and stored in specified project folder.
Test Input	Original XML File
Test Procedure	1. Upload the original XML file by clicking on "Upload XML File" button. 2. Display the components on XML file on data grid. 3. Provide the name for table and refined XML file 4. Check the folder path to store the refined XML file. 5. Display the created refined XML file once "Convert XML" is clicked.
Expected Outcome	The uploaded XML file should be converted to Refined XML file and stored on specified project folder.
Validations	• Once XML file uploaded successfully the buttons should be disabled. • Table name, refined XML file name are required fields. • The provided project folder path should be valid. • Displayed original XML and refined XML are read-only.
Test Output	The uploaded XML file was converted to Refined XML file and stored on specified project folder.
Test Result	PASS
SQL Database Connector	
Test case ID	**02**
Pre-requirements	Refined XML should be created and stored on project folder. Database on MS-SQL Server should be created.
Description	Connect to SQL Server Database by providing or generation the SQL connection string.
Test Input	SQL Server connection string. (No input if generated)
Test Procedure	1. Provide the SQL Server Connection string. • Enter connection string manually.

	• Generate the connection string by selecting the Database name. 1. Test the connection by clicking on “Test Connection” button.
Expected Outcome	The selected database should be connected with the module successfully and the connected database name should be displayed to user.
Validations	• Testing the connection is mandatory. • Connection string is required field. • Database name displayed after successful connection is read-only.
Test Output	The selected database connected with the module successfully and the connected database name displayed to user.
Test Result	PASS
Table Creation Module.	
Test case ID	**03**
Pre-requirements	Refined XML should be created and stored on project folder. Database on MS-SQL Server should be connected.
Description	Create table on connected SQL Server Database.
Test Input	None
Test Procedure	1. Table structure is displayed in data grid which included column name, data types and key fields along with table name. (Read automatically from created Refined XML File. 2. Click on “Create Table” button. 3. Table created on database and SQL query used to create the table is displayed.
Expected Outcome	Table should be created on the database and SQL query displayed to the user.

Validations	• If the displayed data from refined XML is modified, it should be validated to create table. • All user modifications should be verified and saved on refined XML file. • Table name is a required field. • Duplicate table/ existing database table cannot be recreated. • Column API name and key field are read-only.
Test Output	Table created on the database and SQL query displayed to the user.
Test Result	PASS
CRUD Code Generator	
Test case ID	**04**
Pre-requirements	Refined XML File, Connected Database, Table created in SQL Server Database.
Description	Generate CRUD Code for selected CRUD operations.
Test Input	Check on CRUD operations checkbox.
Test Procedure	4. CRUD operation name is auto filled depending on the table name. 5. Able to check on relevant checkboxes for which code needs to be generated. 6. Click on “Generate Code” button. 7. View the generated code.
Expected Outcome	Code should be generated for selected CRUD operation and stored as .cs file in the project folder.
Validations	• Minimum of 1 CRUD operation need to be checked to generate code. • Modification to CRUD operation name need to be validated before generating code. • Code regeneration is possible only when code was generated already.
Test Output	Code generated for selected CRUD operation and stored as .cs file in the project folder.
Test Result	PASS
REST API Code Generator.	
Test case ID	**05**

Pre-requirements	Refined XML file, MongoDB URL details and model class.
Description	Generate route class code and main backend JS file.
Test Input	Check on HTTP methods checkbox.
Test Procedure	1. Mark the checkboxes for HTTP methods for which the code needs to be generated. 2. Click on “Generate Code” button. 3. View the generated code. 4. Check the main backend JS file name and click on “Generate Code”. 5. View the generated main backend JS file.
Expected Outcome	The route class code should be generated and stored on the project folder. The main backend JS file should be generated and stored on the project folder.
Validations	• Minimum of 1 HTTP method checkbox needs to be selected to generate code. • Auto filled endpoint URL must be validated before save. • The main backend JS file name cannot be empty.
Test Output	Route class code generated and stored on project folder. Main backend JS code generated and stored on project folder.
Test Result	PASS.

Table 3. 11 Integration Testing - component 04

Generate CRUD Code.	
Test case ID	**01**
Pre-requirements	Original XML file obtained from UI Generator component.
Description	Generate CRUD code for uploaded XML file, with database connectivity and database table.
Test Input	Original XML File.
Test Procedure	1. Upload the original XML file by clicking on "Upload XML File" button. 2. Uploaded XML file read and processes. Data displayed on data grid. 3. Enter table name, refined XML file name and specify path to store project deliverables. 4. Click on "Convert XML", which results in creation of refined XML file. 5. Connect to SQL Server Database by providing or generating connection string. 6. Once connection is established database name is displayed. 7. Refined XML file read, and data displayed on data grid. 8. Click on "Create Table". Validations are checked if displayed data is manually modified. 9. Table created on database and SQL query used for table creation is displayed. 10. Select the CRUD operation for which the code needs to be generated. 11. Click on "Generate Code" button. 12. View the generated code.
Expected Outcome	• Refined XML File created and stored on project folder. • Table created on connected SQL Server Database. • Generated CRUD code stored on project folder.
Test Output	• Refined XML File created and stored on project folder. • Table created on connected SQL Server Database. • Generated CRUD code stored on project folder.
Test Result	PASS.

Generate REST API Code.	
Test case ID	**02**
Pre-requirements	Original XML file obtained from UI Generator component.
Description	Generate REST API code for uploaded XML file, with database connectivity and database table.
Test Input	Original XML File.
Test Procedure	1. Followed same steps in Test case 01 (1 to 4) 2. Enter and save MongoDB details. 3. Generate Model class code and view. 4. Select HTTP methods for which code need to be generated. 5. Generate route class code and view. 6. Generate main backend JS file and view.
Expected Outcome	• Refined XML File created and stored on project folder. • Generated REST API code stored on project folder.
Test Output	• Refined XML and REST API Code generated.
Test Result	PASS

3.4.5 System Testing

Upload rough sketch and obtain standard UI, XML, frontend code web and mobile along with backend code.	
Test case ID	**01**
Pre-requirements	Rough sketch should be drawn and stored in format of .jpg or .png
Description	Starting from rough sketch process of obtaining standard UI, XML, frontend code web and mobile along with backend code.
Test Input	Rough sketch.
Test Procedure	1. Upload the rough sketch to UI generator component. 2. Click on submit button, the model should run on background and finally "Predicted successfully!!" message should be displayed. 3. The predicted UI elements should be displayed and the user data form with its fields should be displayed accordingly. 4. Then click on "Submit" button which in turn the data should be saved to the database. 5. Finally, the generated UI and XML file should be displayed. 6. Upload the XML file to frontend code generator web. 7. Obtain the corresponding JSON file. 8. Select the template for UI along with the desired style selection. 9. Generated frontend code should be displayed along with the frontend web UI. 10. Download generated frontend code along with standard UI to the local machine. 11. Upload the XML file to frontend code generator mobile. 12. The XML file should be read and displayed on the data grid. 13. Allow the user to select the theme. 14. Depending on the theme selected generate the flutter code.

	15. Run the generated code on an emulator or mobile device. 16. Upload the XML file to backend code generator component. 17. The XML file should be read and displayed on the data grid along with the refined XML file name, path, and the table name. 18. Connect to the database by providing the connection URL. 19. Generate CRUD code and REST API code depending on the preference. 20. Generated backend code should be saved on the project folder.
Expected Outcome	• Standard UI • XML file with UI specification. • Generated web frontend code. • Generated web frontend UI. • Generated mobile frontend code. • Generated web frontend UI. • Refined XML File created and stored on project folder. • Generated REST API or CRUD code stored on project folder.
Test Output	• Standard UI • XML file with UI specification. • Generated web frontend code. • Generated web frontend UI. • Generated mobile frontend code. • Generated web frontend UI. • Refined XML File created and stored on project folder. • Generated REST API or CRUD code stored on project folder.
Test Result	PASS

3.5. Software Development Life Cycle (SDLC).

Like other software development projects, the research project should be well planned and organized to come up with the desired product/outcome at the end. With our experience in different modules in previous semester we believe following an agile methodology would make our development process organized and well-structured. With said that, business requirements may change time to time or there can be a modification in functionalities. These changes or modification should not affect the project development process. Agile SCRUM methodology fits perfectly to these kind of projects with its ceremonies and principles. Now the readers might get a question, why did we choose agile SCRUM methodology for the project? The main reasons are as follows.

1. Agile SCRUM methodology adds flexibility and can respond to changes during the project development.
2. Sprints in the agile SCRUM methodology helps to properly breakdown the task and deliver working products or outcomes at each sprint.
3. Able to observe the work completed, work in progress and the work that needs to be done clearly with their prioritization.
4. The team consists of 6 members, which we have an added advantage of playing the SCRUM roles easily.
5. SCRUM ceremonies help in establishing a good communication between the team and helps to keep the entire team to be aware of the project status.

The above-mentioned points outline the key reasons while there are several other reasons why we chose agile methodology for our project. The figure 3.4.1 shows how we planned to play the SCRUM roles.

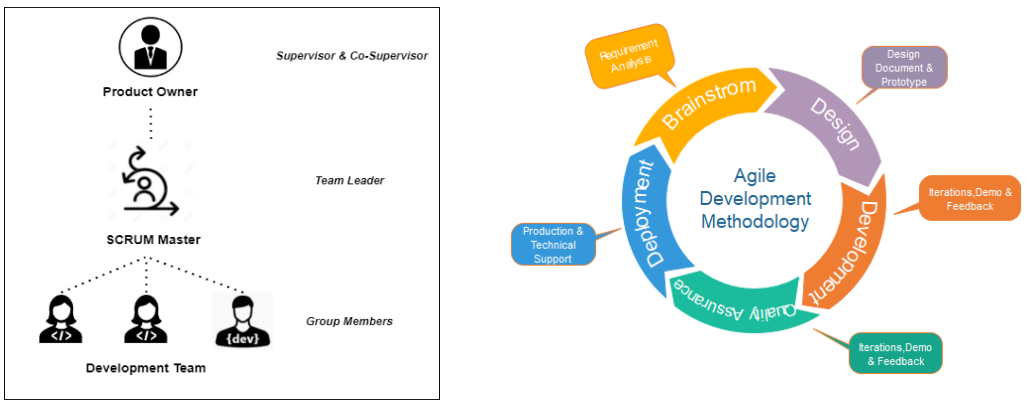


Figure 3. 24 SCRUM Hierarchy and Agile Methodology

3.6. Gantt Chart.

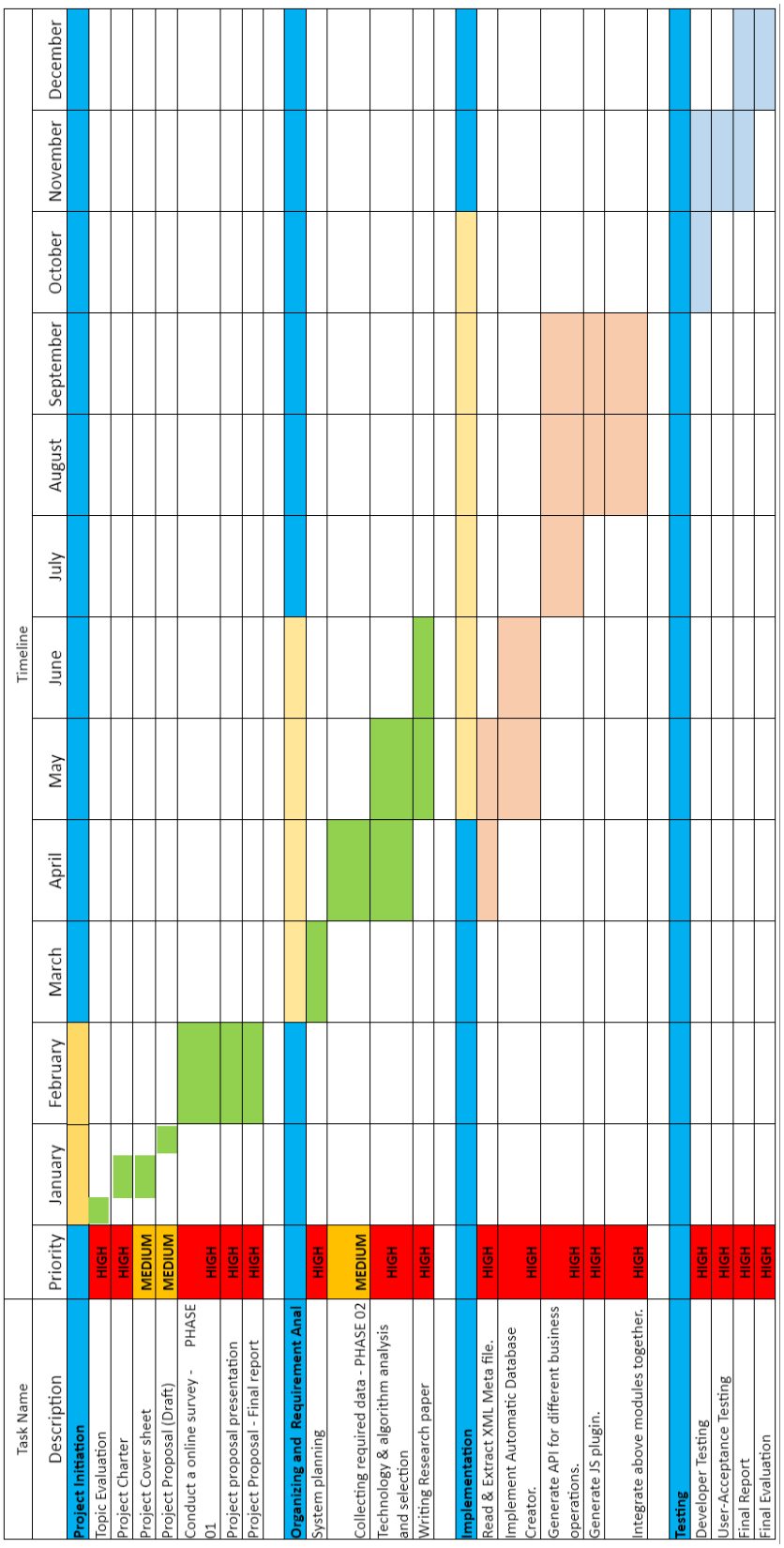


Figure 3. 25 Gantt Chart

3.7. Work Breakdown Structure.

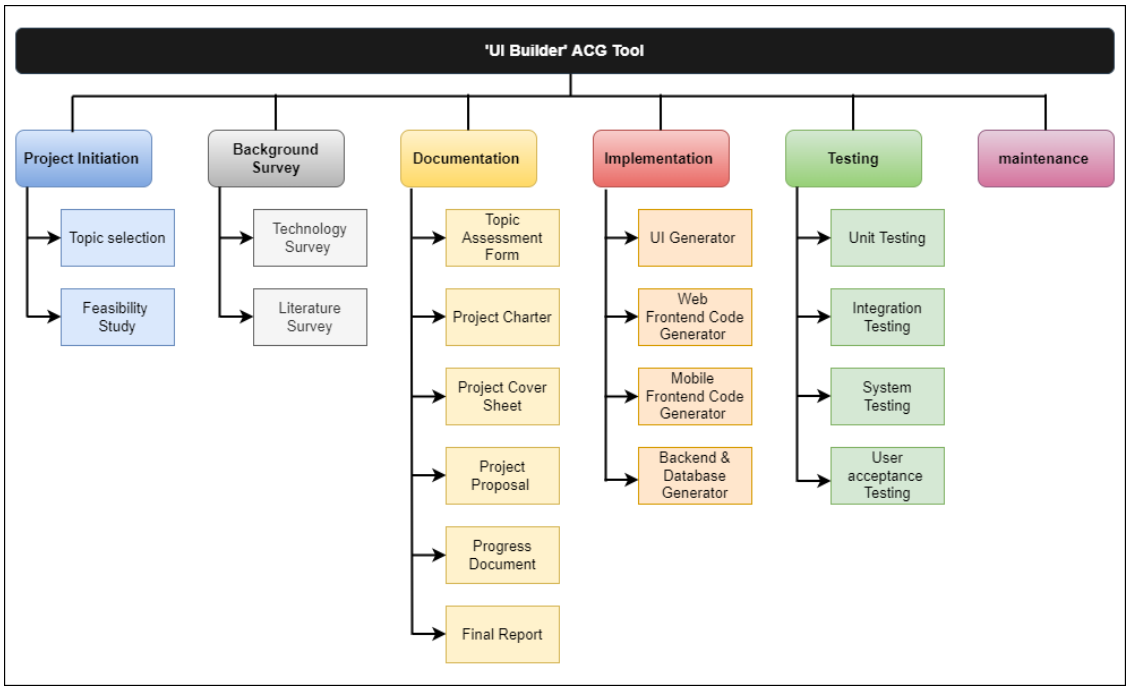


Figure 3. 26 Work Breakdown Structure

3.8. Commercialization Aspects of the Products

With the explanation made upon the backend code and database generator component, it is required to outline the business value with the successful implementation and launch to the market. The business potential is listed below trying to be specific for the component meanwhile covering some general aspects and values of the entire 'UI Builder' tool. This helps to stress the need and value from conducting the research and delivering a product upon completion.

- Time consumed for providing a working software product in software engineering projects get reduced.
- Able to make the SDLC process flexible, manageable and adapt to changing requirements from business.
- Incorporating organizational standards and requirements into the generated code help business to fight competition and achieve their goals easily.
- Decrease overhead of developers, facilitating them to test and debug code easily.
- Able to formulate the concept of 'Anyone Can Code' with the successful completion of the product.
- Executives, software engineering managers and wide range of users from other profession lacking technology knowledge get opportunity to design and develop applications by themselves with their own preferred customization.
- Decrease in cost of software product development while increase in user satisfaction.

4. RESULTS & DISCUSSION

4.1 Results

This section describes the results / outcomes after the development process of all the components of “UI Builder” tool. The results can be mainly classified into main and other supporting results. Key outcomes are the achievement of our primary goal of the tool, while other supporting outcomes explain new ways in which new features and processes can be associated with the tool as required by the business users.

4.1.1 UI Generator

There are plenty of results of this research component. Software developers and UI/UX engineers can do the designs of the UIs in very short period. Not only that the software development process goes smoothly by having this tool. The major results of this component are, the developers and UI/UX engineers can convert the rough sketches to standard UIs in very short period. It was possible to provide a genuine benefit to the UI builder tool once the UI generator component was developed and integrated with business needs. The outcomes can primarily be divided into crucial and other underwrite outcomes. The accomplishment of our key component goal is crucial outcome. Other underwrite outcomes discuss the novel connections that can be made between the component and new features and procedures as the business sees fit.

Result 01

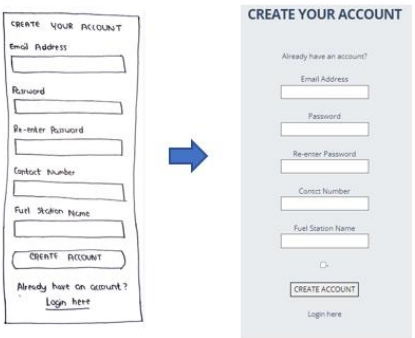


Figure 4. 1 standard UI - result 01

The above figure briefly explains about the process of this research component. Initially the users (developers, UI/UX designers) need to upload their preferred rough

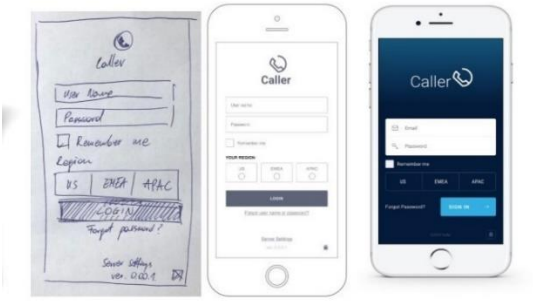
sketch to the application and finally the application generates standard UI. Not only that the XML file will be generated for the other members to continue their research partitions. The sample output of that process is shown below:

Result 02

```
<WindowTitle="CREATE YOUR ACCOUNT">
    <Canvas>
        <Label type="header" Content="CREATE YOUR ACCOUNT" HorizontalAlignment="left"
VerticalAlignment="Right" Margin="0,0,0,0"></Label>
        <Label type="label" Content="Already have an account?" HorizontalAlignment="left"
VerticalAlignment="Right" Margin="0,0,0,0"></Label>
        <TextBox type="input" Label="Email Address" Name="Email Address" DataType="string"
Text="Placeholder" HorizontalAlignment="left" VerticalAlignment="Right" Margin="0,0,0,0"></Label>
        <TextBox type="input" Label="Password" Name="Password" DataType="string" Text="Placeholder"
HorizontalAlignment="left" VerticalAlignment="Right" Margin="0,0,0,0"></Label>
        <TextBox type="input" Label="Re-enter Password" Name="Re-enter Password" DataType="string"
Text="Placeholder" HorizontalAlignment="left" VerticalAlignment="Right" Margin="0,0,0,0"></Label>
        <TextBox type="input" Label="Contct Number" Name="Contct Number" DataType="string"
Text="Placeholder" HorizontalAlignment="left" VerticalAlignment="Right" Margin="0,0,0,0"></Label>
        <TextBox type="input" Label="Fuel Station Name" Name="Fuel Station Name" DataType="string"
Text="Placeholder" HorizontalAlignment="left" VerticalAlignment="Right" Margin="0,0,0,0"></Label>
        <CheckBoxes type="checkbox" Label="-" Name="-" Type="checkbox" Content="Content"
Value="TestValue HorizontalAlignment="left" VerticalAlignment="Right" Margin="0,0,0,0"></Label>
        <Button type="button" Name="CREATE ACCOUNT" Content="CREATE ACCOUNT" Width="150"
Height="100 HorizontalAlignment="left" Margin="0,0,0,0"></Button>
    </Canvas>
</Window>
```

Figure 4. 2 XML file - result 02

At initially the users (developers, UI/UX designers) need to upload their preferred rough sketch to the application and finally the application generates beautiful UI. Not only that the XML file will be generated for the other members to continue their research partitions.



4.1.2 Web Frontend Generator

The major and supporting results obtained related to the Web Frontend code generator component can be described as follows.

Results obtained from a simple input XML file (Login Form)

Result 1.1 – Formatted JSON File

As we discussed in the development process, the XML file received as input is converted into a JSON file for ease of implementation on the web front. This JSON file contains the UI specification details. A special feature here is that the details of the XML file are correctly formatted while converting the XML file to JSON file.



Figure 4. 3 Formatted JSON file - result 1.1

Result 1.2 – React Web Frontend Code

The main result of this component is the development process of web frontend code. The JSON details and template styles selected by the user are embedded in this code.

The special features of this generated code are:

- Refactored and commented frontend code.
- Simple and easily understandable code.
- Follow proper coding standards (naming conventions, error handling)
- The code is flawlessly organized and well structured.

```
<div className="LoginForm">
    <form style={{ border: '3px solid #000000', backgroundColor: '#42e37f', width: '43%', marginLeft: '29%' }}>
        <div class="formContainer" style={{ textAlign: 'center', margin: '24px 0 12px 0' }}>
            <h1 style={{ color: 'white' }}>LOGIN FORM</h1><hr/>
        </div>
        <div class="container" style={{ padding: '16px' }}>
            <label for="uname" style={{ float: 'left', paddingTop: '5px', fontSize: '16px' }}>
                <b style={{ color: 'white' }}>Username</b>
            </label>
            <input
                type="text"
                placeholder="Enter Username"
                name="uname"
                required
                style={{ padding: '12px 20px', margin: '8px 0', border: '2px solid #ccc', width: '100%' }}
            />
            <br /><br />
            <label for="psw" style={{ float: 'left', paddingTop: '5px', fontSize: '16px' }}>
                <b style={{ color: 'white' }}>Password</b>
            </label>
            <input
                type="password"
                placeholder="Enter Password"
                name="psw"
                required
                style={{ padding: '12px 20px', margin: '8px 0', border: '2px solid #ccc', width: '100%' }}
            />
            <br /><br />
            <button type="submit" style={{ backgroundColor: '#000000', color: 'white', padding: '14px 20px',
             margin: '8px 0', border: 'none', borderRadius: '7px', width: '100%' }}>Login</button>
            <br /><br /><br />
        </div>
    </form>
</div>
```

Figure 4. 4 Downloaded Frontend Code - result 1.2

Result 1.3 – Result UI

In addition to the generated code in this component development process, the other result is the result UI. This result UI is automatically displayed by the system. The advantage is that the user can immediately view the UI result related to the generated code.

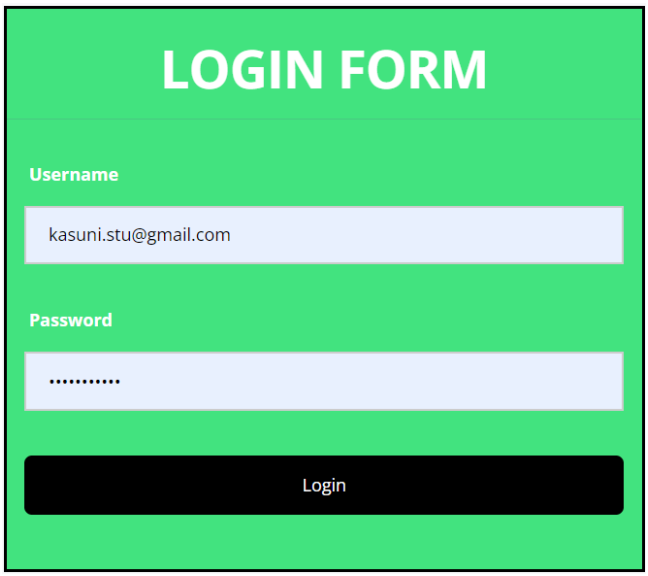


Figure 4. 5 Downloaded Result UI - result 1.3

Results obtained from a complex input XML file (Sign Up Form)

Result 2.1 – Formatted JSON File

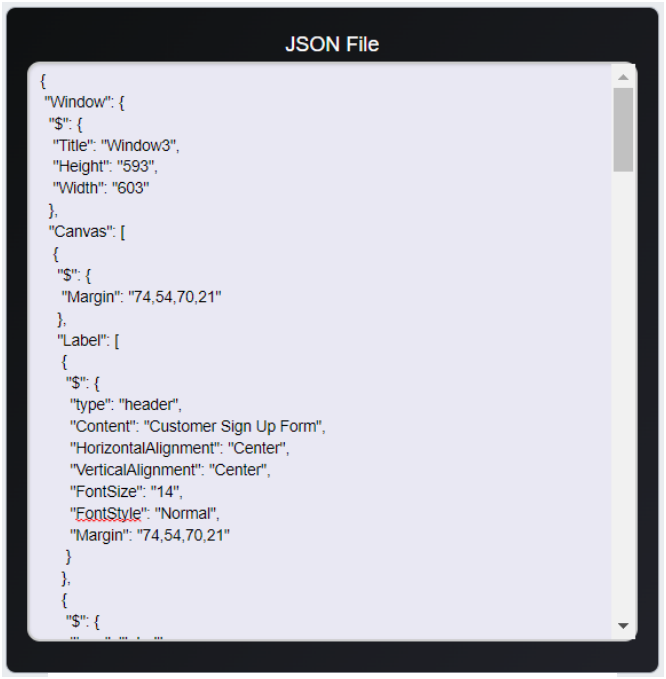


Figure 4. 6 Formatted JSON file - result 2.1

Result 2.2 – React Web Frontend Code

```
<div class='Window3;'>
    <form style='border: undefined undefined undefined; width: 45%; margin-left: -15px; background-color: undefined;'>
        <div class='formContainer' style='text-align: center; margin: 24px 0 12px 0;'>
            <h1 style='color: undefined; horizontal-align: Center; margin: 74,54,70,21;'>Customer Sign Up Form</h1><hr/>
        </div>
        <div class='container' style='padding: 16px;'>
            <br />
            <label style=' font-size: undefined;'>
                <b style='color: undefined; font-family: Garamond; font-size: 30px;'>***** FASHIONZ *****</b>
            </label>
            <br />
            <label style=' font-size: undefined;'>
                <b style='color: undefined; font-family: Garamond; font-size: 30px;'>~~~ One stop for your design choice ~~~</b>
            </label>
            <br />
            <br/> <p style='color: white;'>Customer Name</p>
            <input
                type="text"
                placeholder="Enter Customer Name"
                name="Customer Name"
                required
                style='padding: 12px 20px; margin: 161,129,0,0; color: white; width: 100%; border: undefined undefined undefined;
            />
            <br /><br />
            <br/> <p style='color: white;'>Customer Address</p>
            <input
                type="text"
                placeholder="Enter Customer Address"
                name="Customer Address"
                required
                style='padding: 12px 20px; margin: 161,129,0,0; color: white; width: 100%; border: undefined undefined undefined;
            />
            <br /><br />
            <br/> <p style='color: white;'>Customer Telephone</p>
            <input
                type="text"
                placeholder="Enter Customer Telephone"
```

Figure 4. 7 Downloaded Frontend Code - result 2.2

Result 2.3 – Result UI

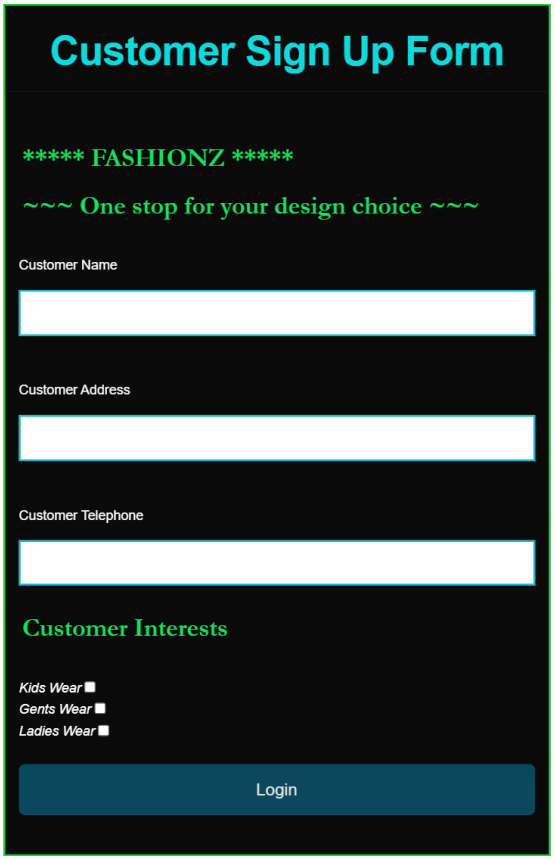


Figure 4. 8 Downloaded Result UI - result 2.3

4.1.3 Mobile Frontend Generator

As the development process is completed of the mobile fronted code generator component, the advantages and the uses of the component could be clearly pointed. The most important result of the components could be called as the initial goal of the component which was recognized during the proposal procedure. The purpose of the mobile frontend code generator component is to allow the users to receive a mobile frontend code without coding the interface by hand.

Result 01 – Flutter Code as txt file

Mobile frontend code generator component generated a text file with the Flutter code for the uploaded user interface and stores in user's device as well. As the code should be customizable for the user, this component allows them to choose their preferred color theme.

```
import 'package:flutter/material.dart';

void main() {
        runApp( MyApp());
}

class MyApp extends StatelessWidget {
        Widget build(BuildContext context) {
        return MaterialApp(
                home: Scaffold(
                                backgroundColor: Color(0xFFc1e3ed),
                        appBar: AppBar(
                                title: const Text('Welcome to My App'),
                        ),
                        body: MyCustomForm(),
                        ),
        );
 }}

// Create a Form widget.
class MyCustomForm extends StatefulWidget{
        MyCustomFormState createState() {
                return MyCustomFormState();
        }
}

class MyCustomFormState extends State<MyCustomForm> {
        //a global key that uniquely identifies the Form widget and allows validation of the form
        final _formKey = GlobalKey<FormState>();
        Widget build(BuildContext context) {
         return Form(
                key: _formKey,
                child: Center( child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
```

Figure 4. 9 Flutter code as txt - result 01

Result 02 – Flutter Code displayed in the system

As shown in the development process section, this component gives the mobile code in Flutter language as the output. The output can be displayed in the system and user can view the code from the system as shown in the figure 16.

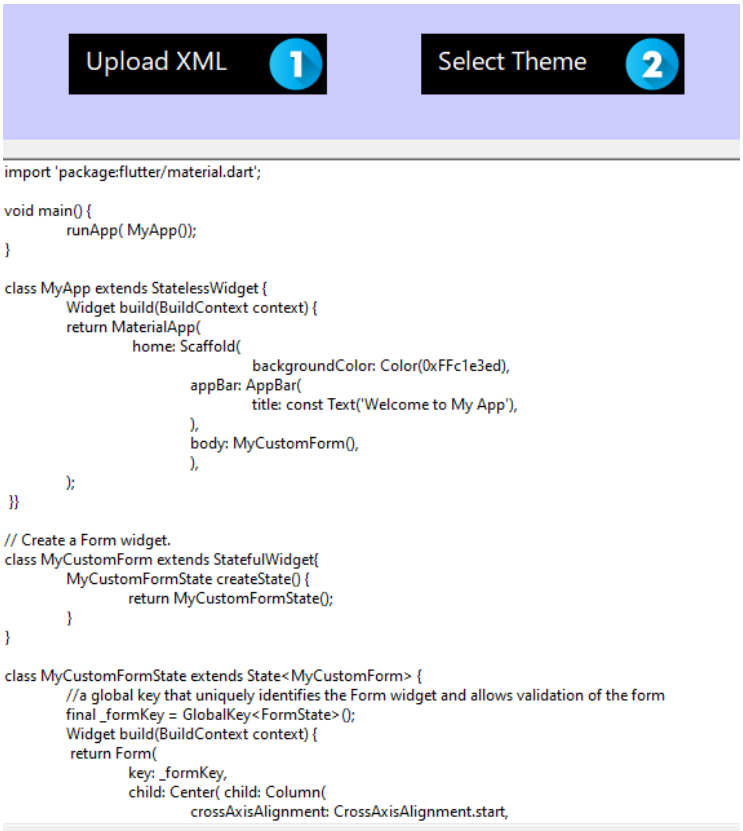


Figure 4. 10 Flutter Code displayed in the system - result 02

4.1.4 Backend & Database Generator

The major and supporting results obtained related to the Backend and Database generator component can be described as follows.

Result 01 – Refined XML File

An outcome obtained from the XML to the refined XML convertor module which describes the schema (table) structure. The XML file focuses on carefully recording the modifications on structure and storing them for future need. The following Figure 18 shows a sample refined XML file constructed from the component.

An example Scenario: Consider the developer uploads XML file which describes the customer sign-up user interface.

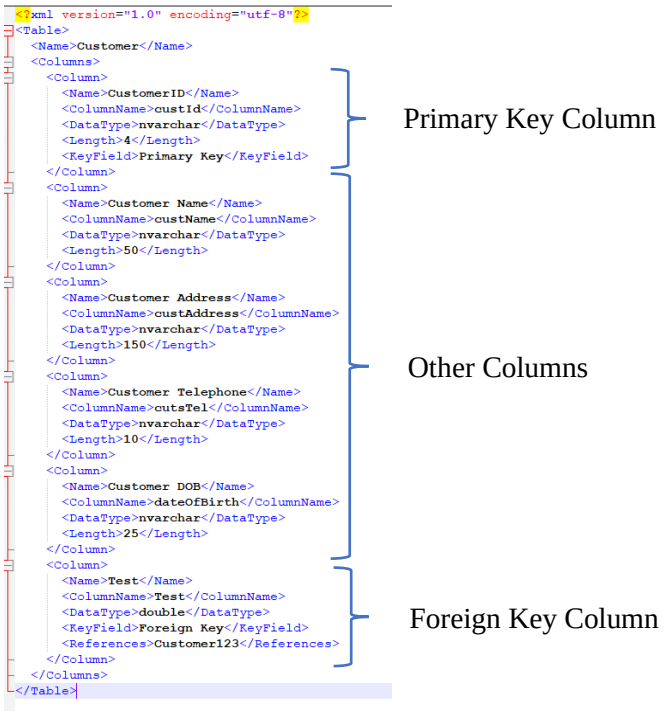


Figure 4. 11 Sample refined XML file - result 01

Result 02 – Backend Code (CRUD & REST API Code)

The backend code is the main output obtained from the component. The backend code can be in two forms, namely the CRUD or REST API code depending on developer preference. Before explaining the characteristic of the generated code and their highlights, it is needed to get an idea by looking at few sample codes generated from the component. The Table 9 shows the generated CRUD code and Table 10 shows the generated REST API Code.

An example scenario: Consider Customer related operation are needed to be generated.

Table 4.1 Generated CRUD operation code samples.

Generated "Create" Operation Code.

```
/*
********************************************************************************
*NOTE          :   This is an auto-generated Code
*@Developer    :   (Add Developer Name Here)
*@Purpose      :   Add CustomerTest to database table.
*@Created Date :   8/22/2022 8:10:57 AM
********************************************************************************
*/
public void addCustomer(String id, String custName, String custAddress, String custTel, date custDateOfBirth, String custGender, String custCountry, String custInterest)
    try{
        SqlConnection con = new SqlConnection(@Data Source=.;Initial Catalog='Test Database';Integrated Security=True);
        SqlCommand cmd;

        String sql = "INSERT INTO CustomerTest VALUES(@id,@custName,@custAddress,@custTel,@custDateOfBirth,@custGender,@custCountry,@custInterest);"
        con.Open();
        cmd = new SqlCommand(sql, con);

        cmd.Parameters.AddWithValue("@id", id);
        cmd.Parameters.AddWithValue("@custName", custName);
        cmd.Parameters.AddWithValue("@custAddress", custAddress);
        cmd.Parameters.AddWithValue("@custTel", custTel);
        cmd.Parameters.AddWithValue("@custDateOfBirth", custDateOfBirth);
        cmd.Parameters.AddWithValue("@custGender", custGender);
        cmd.Parameters.AddWithValue("@custCountry", custCountry);
        cmd.Parameters.AddWithValue("@custInterest", custInterest);

        cmd.ExecuteNonQuery();

        Console.Write('Date Added Successfully!');

    } catch(Exception ex) {
        Console.Write("Exception Occurred!");
    }

}
```

Generated "Retrieve" Operation Code.

```
*NOTE          :   This is an auto-generated Code
*@Developer    :   (Add Developer Name Here)
*@Purpose      :   Get CustomerTest from database table.
*@Created Date :   8/22/2022 8:10:57 AM
********************************************************************************
*/
public List<String> getCustomer() {
    private List<String> detailList = new List<String>();

    try{
        SqlConnection con = new SqlConnection(@Data Source=.;Initial Catalog='Test Database';Integrated Security=True);
        SqlCommand cmd;
        SqlDataReader dr;

        String sql = "SELECT id,custName,custAddress,custTel,custDateOfBirth,custGender,custCountry,custInterest FROM CustomerTest";
        con.Open();
        cmd = new SqlCommand(sql, con);
        dr = cmd.ExecuteReader();

        while(dr.Read()) {
            List<String> localDataList = new List<String>();
            for(int i = 0; i < 8; i++) {
                localDataList.Add(dr[i]);
            }

            detailList.Add(localDataList.ToString());
            return detailList;
        }


        Console.Write('Date Fetched Successfully!');
        con.Close();

    } catch(Exception ex) {
        Console.Write("Exception Occurred!");
    }
```

Generated “Update” Operation Code.

```
/*
**********************************************************************
*NOTE          :   This is an auto-generated Code
*@Developer    :   (Add Developer Name Here)
*@Purpose      :   Update CustomerTest on database table.
*@Created Date :   8/22/2022 8:10:57 AM
**********************************************************************
*/
public void updateCustomerById(String custName, String custAddress, String custTel, date custDateOfBirth, String custGender, String custCountry, String custInterest) {
    try{
        SqlConnection con = new SqlConnection(@Data Source=.;Initial Catalog='Test Database';Integrated Security=True);
        SqlCommand cmd;

        String sql = "UPDATE CustomerTest SET custName = @custName, custAddress = @custAddress, custTel = @custTel, "+
                     "custDateOfBirth = @custDateOfBirth, custGender = @custGender, custCountry = @custCountry, custInterest = @custInterest  WHERE id = "pkFieldValue";

        con.Open();
        cmd = new SqlCommand(sql, con);

        cmd.Parameters.AddWithValue("@custName", custName);
        cmd.Parameters.AddWithValue("@custAddress", custAddress);
        cmd.Parameters.AddWithValue("@custTel", custTel);
        cmd.Parameters.AddWithValue("@custDateOfBirth", custDateOfBirth);
        cmd.Parameters.AddWithValue("@custGender", custGender);
        cmd.Parameters.AddWithValue("@custCountry", custCountry);
        cmd.Parameters.AddWithValue("@custInterest", custInterest);

        cmd.ExecuteNonQuery();

        Console.Write('Data Updated Successfully!');

    } catch(Exception ex) {
        Console.Write("Exception Occurred!");
    }

}
```

Generated “Delete” Operation Code.

```
/*
**********************************************************************
*NOTE          :   This is an auto-generated Code
*@Developer    :   (Add Developer Name Here)
*@Purpose      :   Delete CustomerTest from database table.
*@Created Date :   8/22/2022 8:10:57 AM
**********************************************************************
*/
public void deleteCustomerById(String id) {
    try{
        SqlConnection con = new SqlConnection(@Data Source=.;Initial Catalog='Test Database';Integrated Security=True);
        SqlCommand cmd;

        String sql = "DELETE FROM CustomerTest WHERE id = "pkFieldValue";"
        con.Open();
        cmd = new SqlCommand(sql, con);


        cmd.ExecuteNonQuery();

        Console.Write('Data Deleted Successfully!');

    } catch(Exception ex) {
        Console.Write("Exception Occurred!");
    }

```

Table 4.2 Generated REST API Code samples.

Generated “Model class” code

```
/*
**********************************************************************************
*NOTE          :   This is an auto-generated Code
*@Developer    :   (Add Developer Name Here)
*@Scope        :   Model Class - CustomerModel
*@Created Date :   8/22/2022 8:11:24 AM
**********************************************************************************
*/
const mongoose = require('mongoose')

const CustomerTestSchema = new mongoose.Schema({
    id: {
        type: String,
        required: true,
        trim: true
    },
    custName: {
        type: String,
        required: true,
        trim: true
    },
    custAddress: {
        type: String,
        required: true,
        trim: true
    },
    custTel: {
        type: String,
        required: true,
        trim: true
    },
    custDateOfBirth: {
        type: date,
        required: true,
    },
    custGender: {
        type: String,
        required: true,
        trim: true
    },
    custCountry: {
        type: String,
        required: true,
        trim: true
    },
    custInterest: {
        type: String,
        required: true,
        trim: true
    }
});

const CustomerTest = mongoose.model("CustomerTest", CustomerTestSchema);
module.exports = CustomerTest;
```

Route Class Code

Generated “POST” HTTP Method Code.

```
const router = require('express').Router();

/**
* Imported CustomerModel Model - CustomerModel.js
*/
const CustomerModel = require('../Model/CustomerModel');

/*
**********************************************************************************
*NOTE          :   This is an auto-generated Code
*API DESC      :    Create new CustomerTest
*API           :    http://localhost:3000/customer/addcustomer
*@Developer    :   (Add Developer Name Here)
*@Created Date :   8/22/2022 8:12:17 AM
**********************************************************************************
*/
router.route('/addCustomerTest').post(async (req, res) => {
    if(req.body) {

        const CustomerTest = new CustomerModel(req.body);
        await CustomerTest.save()
            .then(data => {
                res.status(200).send({data: data});
            }).catch(error => {
                res.status(500).send({error: error});
            });
    }
});

```

Generated “GET” HTTP Method Code.

```
/*
*********************************************************************************
*NOTE           :   This is an auto-generated Code
*API DESC       :    Get all CustomerTest
*API            :    http://localhost:3000/customer/getAllcustomer
*@Developer     :   (Add Developer Name Here)
*@Created Date :   8/22/2022 8:12:17 AM
*********************************************************************************
*/
router.route('/getAllCustomerTest').get(async (req, res) => {
        await CustomerModel.find({})
            .then(data => {
                res.status(200).send({data: data});
            }).catch(error => {
                res.status(500).send({error: error});
            });
});

```

Generated “PUT” HTTP Method Code.

```
/*
*********************************************************************************
*NOTE           :   This is an auto-generated Code
*API DESC       :    Update one CustomerTest
*API            :    http://localhost:3000/customer/updatecustomer/:id
*@Developer     :   (Add Developer Name Here)
*@Created Date :   8/22/2022 8:12:17 AM
*********************************************************************************
*/
router.route('/updateCustomerTestById/:id').update(async (req, res) => {

    const custName = req.body.custName;
    const custAddress = req.body.custAddress;
    const custTel = req.body.custTel;
    const custDateOfBirth = req.body.custDateOfBirth;
    const custGender = req.body.custGender;
    const custCountry = req.body.custCountry;
    const custInterest = req.body.custInterest;

    const Id = req.param.id;

    try{
        await CustomerModel.findById(Id, updatedObj) => {
            updatedObj.custName = custName;
            updatedObj.custAddress = custAddress;
            updatedObj.custTel = custTel;
            updatedObj.custDateOfBirth = custDateOfBirth;
            updatedObj.custGender = custGender;
            updatedObj.custCountry = custCountry;
            updatedObj.custInterest = custInterest;

            updateObj.save()
                .then(data => {
                    res.status(200).send({data: data});
                }).catch(error => {
                    res.status(500).send({error: error});
                })
    } catch(error => {
        console.log(error);
    }
});

```

Generated “DELETE” HTTP Method Code.

```
/*
*********************************************************************************
*NOTE           :   This is an auto-generated Code
*API DESC       :    Delete one CustomerTest
*API            :    http://localhost:3000/customer/deletecustomer/:id
*@Developer     :   (Add Developer Name Here)
*@Created Date :   8/22/2022 8:12:17 AM
*********************************************************************************
*/
router.route('/deleteCustomerTestById/:id').delete(async (req, res) => {

    if(req.params && req.params.id){
        await CustomerModel.findByIdAndDelete(req.params.id)
        .then(data => {
            res.status(200).send({data: data});
        }).catch(error => {
            res.status(500).send({error: error});
        })
    }
});

```

Generated “Main Backend JS” code.

```
/**
 * Imports of installed packages
**/
const express = require('express');
const mongoose = require('mongoose');
const bodyparser = require('body-parser');
const cors = require('cors');
const dotenv = require('dotenv');

dotenv.config();
const app = express();

app.use(bodyparser.json());
app.use(cors());

app.get('/', async(req,res) => {
    res.send('Welcome to **Enter App Name**!!');
});

const CustomerTestRoute = require('./routes/CustomerTestRoute')

app.use('/customertest', CustomerTestRoute);

/**
 * ROUTE IMPORTED FROM - BACKEND -> ROUTES
*/
const PORT = 3000;
const MONGODB_URI = mongodb+srv://sanjay:sanjay-8330@shoppingstore.x5e6q.mongodb.net/ShoppingDatabase?retryWrites=true;

//Connection to Mongoose
mongoose.connect(MONGODB_URI, {
    useNewUrlParser: true,
    useUnifiedTopology: true,
    useCreateIndex: true,
    useFindAndModify: false
}, (error) => {
    if(error){
        console.log('Error in Connection!');
    }
});

//Check if connection is successful
mongoose.connection.once('open', () => {
    console.log('Database Synched');
})

//Running on server
app.listen(PORT, () => {
    console.log(`Server is started and running on ${PORT}`);
});

```

All above code snippets shown are generated using the backend code generator component and the use of pre-defined templates make it possible for fit to any business requirements. The following characteristics can be seen in the generated CRUD or REST API code.

- Refactor code.
- Commented code.
- Organized and structured code.
- Tested against Database connection.
- Simple code base.
- Customizable code.
- Other coding best practices (Variable naming, error handling)

4.2 Research Findings

This section focuses on the research findings made throughout the entire UI Builder implementation. The main research findings related to each component is described below.

The research findings in the UI Generator component are using CNN model to detect rough UI elements, using OpenCV's image segmentation to extract specific UI components in rough sketches, and finding automatic UI template generation algorithm. The image segmentation, model training like new approaches were founded by working with this research component. The initial phase of this research component the image segmentation and the image identification processes were complex tasks. After going through some investigation, the real concepts of the image identification were found. That was very helpful to implement this research component. As well as training data set and testing data set from the model was the other research finding of this research component.

Web frontend generator using MERN stack technologies, selecting user preferred styles and flexibility of generated code can be considered as the most promising research findings found throughout the entire implementation process of the web frontend generator component. Using MERN stack technologies is safer than HTML and has faster development. Also, another key research finding found in this web frontend component is that the XML meta file used as input is converted into a JSON file and used to execute the reactions. Implementing React is not very flexible with the XML language. Therefore, this conversion process is used. Another research finding in this web frontend generator is that the user can choose the template designs as they like. This section can be considered as another unique section of our 'UI Builder' tool. Under the literature review study, when research papers on ACG tools are studied, one of the features of the grant is to generate and deliver the implemented source code. As a result of all those ACG tools. But none of those research papers provide the source code with the resulting UI. Also, the source code can be downloaded but not modified. All these shortcomings are filled by our 'UI Builder' tool.

The research findings related to the mobile frontend generator can be described under 3 categories. They are cross flatform language for the code, styling and the flexibility of the code and keeping the coding standards. This component used Flutter as the development language. Flutter is suitable for Android, iOS, Linux, macOS, Windows, and Google Fuchsia. Most importantly all the operating systems could be accessed by a single code base. Also here allowing users to have the privilege to generate the code is important. All the users do not have the knowledge of mobile application development. Therefore, it is hard to engage them with the development procedure. Even though users do not understand the development process, they can engage with the procedure by customizing the color theme of the interface. Therefore, this system has given the options of allowing them to choose the color theme of the interface as the user's satisfaction is important for this system. Finally, this system has properly aligned code format and the comments were added for the necessary parts of the code. If the users need to customize the generated code, they can get the customization done without a confusion with a code that keeps the standards.

This section focuses on explaining the research findings from development of backend code generator component. Major research findings included here are usage of refined XML file to capture table structure, customizable generated code, and common codebase to build complex business requirements. First, focusing on the technical/functional aspects of research findings, the use of refined XML file can be highlighted. Recording and storing the table structure helps to suit changing business requirements. The data schema tends to change with business processes and requires lots of time and effort to respond to the change. Secondly, the process of obtaining the generated code from the component needs to be emphasized. Unlike other available code generators, UI Builder tools strictly follow the folder structure. The ease of use and accessibility are equally considered like the functional aspects of the component. Finally, the research also proved the strength of generating code for a business organization. The component mainly focused on generating a simple code base for the business on which they can customize or add complex business logic depending on various business requirements.

4.3 Discussions

This section explains in depth about the accuracy of results obtained from UI Builder tool components. Below are the details of the various evaluations used to check the accuracy of each component.

1.3.1 UI Generator

The resulting standard UI from the component is directly obtained from the UI elements identified from the rough sketch. Unlike other UI generator tools our tool "UI Builder" has the feature to display the resulting UI soon after the rough sketch is uploaded. This makes the tool stand unique in a way developers or designers could observe how the actual UI look like when the frontend code is really implemented. Another underlying feature is getting the user inputs just before generating the standard UI. This provides the ability for designers or developers to determine or control how the actual UI should look like.

Obtaining basic parameters such as labels, headings, and parameters related to position of UI elements make it easy for the designers to try different templates for the uploaded UI sketch. The generated standard UI from UI generator component is simple which makes it easier to build or to redesign it into a complex user interface. Currently the UI generator components supports proper mapping for basic UI elements. (Labels, headings, checkbox, button, hyperlink, textbox). The component is designed in a way more UI components can be easily added and mapped to the generated standard UI.

The other major outcome from the research component is the documented UI specification in the form of XML. The generated XML file has a direct dependency on the major outcome which is the standard UI. The component extends its ability to construct an XML file which clearly describes the generated standard UI. Another important advantage of documenting the UI lies on the future scope of the tool. The generated XML file which acts as the primary input for the code generator component establishes the proper flow throughout the UI builder tool which makes the code generation and UI generation pipeline syncs together and achieve the desired outcome.

By referring to the evaluation metric charts we can come up to a solution that the accuracy level of each UI component is high.

4.3.2 Web Frontend Generator

A discussion related to the obtained results of the web front generator is done in this section. Here, the concept of evaluation metrics is used for this process, and the accuracy and efficiency of the generated web-frontend code can be checked. A brief testing was conducted here. By comparing automatically generated code and manually executed code, a comparative analysis of the test results is performed using the previously mentioned evaluation metrics.

The various descriptive sample forms used for this testing process are given below.

1. Sample login form for web application.
2. Sample vehicle registration form.
3. Sample Payment form for online shopping platform.
4. Sample Restaurant Order Form.
5. Sample Employee Information Form.
6. Sample Loan Application Form.

Below is the result obtained from the brief testing related to the web frontend generator. This makes it clear that automatic implementation is more accurate and efficient than manual implementation in relation to every field of evaluation metrics.

Table 4. 3 Evaluation Metric Results - component 02

	Time taken to develop		The number of Bugs in the code		Count of logic tested		Number of code lines	
Sample Forms	Development Time (mins)		Number of Bugs		Test Run		Lines of Codes (LOC)	
	Manual	Auto	Manual	Auto	Manual	Auto	Manual	Auto
Login Form	350	4	2	0	10	2	234	215
Vehicle Registration Form	400	4	8	0	18	2	315	300
Payment Form for Online Shopping	650	9	10	0	20	4	378	250
Restaurant Order Form	520	6	12	0	13	2	297	190
Employee Information Form	450	5	15	0	11	2	431	278
Loan Application Form	600	8	11	1	17	3	450	385

4.3.3 Mobile Frontend Generator

This section discusses the accuracy of the results that were obtained. An evaluation was conducted for the purpose of calculating the accuracy of the mobile frontend code generator. The results of the evaluation were obtained from comparing the manual development and the automated development. There were five user interfaces were used for the evaluation.

1. Login form
2. Registration form for E-commerce site
3. Vehicle booking form
4. Food delivery form
5. Hotel reservation form

The evaluation metrics and the data that gathered for each UI template are displayed in table 5. The column Man. represents the date related to manual development and the column Auto. represents the data related to automated development.

Table 4. 4 Evaluation Metric Results - component 03

	Development time		Bug Count		Test Run		Code Size (LOC)	
	Man.	Auto.	Man.	Auto.	Man.	Auto.	Man.	Auto.
Login form	30 min	2 min	5	0	100%	100%	182	134
Registration form for E-commerce site	60 min	2 min	6	0	100%	100%	225	187
Vehicle booking form	90 min	3 min	6	2	50%	100%	424	328
Food delivery form	112 min	3 min	5	1	75%	100%	396	220
Hotel reservation form	230 min	3 min	8	2	80%	100%	685	489

4.3.4 Backend & Database Generator

This section explains deeper on the results obtained from the backend code generator component with the help of evaluation metric. The evaluation metric is used to measure the accuracy of generated backend or frontend code. The evaluation metric used in our research shows more resemblance towards the evaluation metric which Akbulut and Toprak [24], Possatto and Lucrédio [25] used in their study related to the code generator framework for smart TV platform and automatically propagating changes from reference implementations to code generator templates respectively. The code generation components in UI Builder tool used the same evaluation metric to measure the accuracy of generated code.

An experiment was conducted to determine the functionality and performance of backend code generator component. An approach which laid upon comparing the auto-generated code and manually implemented code gave better explanation to comment on the accuracy of component. The following procedure was followed in the experiment conducted.

1. A developer was asked to implement selected modules, both manually and using the backend code generator component in UI Builder tool.
2. Data was obtained and recorded under each attribute used in evaluation metric.
3. Results from evaluation metric were compared and analyzed.

The following Figure 20 shows the results obtained from the experiment which focuses on the backend code generator component.

Table 4. 5 Evaluation Metric Results - component 04

	Development time (minutes)		Bug Count		Test Run		Code Size (LOC)	
	Man.	Auto.	Man.	Auto.	Man.	Auto.	Man.	Auto.
Employee Registration Form	350	5	20	0	15	3	320	210
Customer Signup in E-commerce site	480	4	12	0	25	2	423	324
IT Asset Request	620	8	16	2	17	2	210	84
Vehicle Registration for reputed Automobile Company	500	7	11	0	27	2	353	294
Login Form	400	4	3	0	20	3	242	159
Homepage of E-commerce site (List of products displayed).	650	8	14	0	32	2	510	415

5. SUMMARY OF EACH STUDENT'S CONTRIBUTION

Table 5. 1 Individual Contribution

Student ID	Student Name	Individual Component	Contribution
IT19150758	Ekanayake K.L.W	UI Generator	✓ Collect the user requirements from selected user group using background survey. ✓ Identify the UI elements. ✓ Train the CNN model by the dataset. ✓ Test the dataset. ✓ Process the UI elements using image processing techniques and develops an XML document. ✓ Design the standard UI. ✓ Test the component using Unit Testing & Integration Testing.
IT19158228	Sakthivel.S	Backend & Database Generator	✓ Collect the user requirements from selected user group using background survey. ✓ Design UI and UX for the individual component. ✓ Convert XML to refined XML. ✓ Connect with SQL database. ✓ Implement the backend generator using forward engineering systems.

			✓ Test the component using Unit Testing & Integration Testing.
IT19144986	H.M. Kasuni Navodya	Web Frontend Generator	✓ Collect the user requirements from selected user group using background survey. ✓ Design UI and UX for the individual component. ✓ Convert XML to JSON and create template styles for interfaces. ✓ Implement the web frontend generator using MERN Stack technologies. ✓ Test the component using Unit Testing & Integration Testing.
IT19150826	D. P. Kavindi Gimshani	Mobile Frontend Generator	✓ Collect the user requirements from selected user group using background survey. ✓ Design UI and UX for the individual component. ✓ Read UI elements and create color themes for the interfaces. ✓ Implement the mobile frontend generator. ✓Test the component using Unit Testing & Integration Testing.

6. CONCLUSION

There are ACG tools and template generators are being used currently. But there are certain expectations of users were omitted by those existing solutions. Through the conducted survey, we were able to identify the usage and the expectations of the users. UI Builder was developed to meet those expectations and fulfill the user requirements which were not able to touch via the existing solutions. UI Builder is developed to find the solutions for the identified gap under four main components. They could be listed as below.

1. Convert hand drawn UI sketches to standard UI templates
2. Web frontend code generator
3. Mobile frontend code generator
4. Backend code generator with database connectivity

UI generator component is developed to convert hand drawn UI sketches to standard UI templates. Style selection, generated code can be downloaded and modified can be highlighted as the main features of the web frontend code generator. Mobile frontend code generator component is capable of providing a single codebase for all the devices and platforms despite the usage of varies mobile devices all over the world. The uniqueness of UI Builder could be introduced with backend code generator with database connectivity which does not exist in any other ACG tools. With the help of the above components, UI Builder was designed and developed to benefits the developers as well as anyone who is interested in developing a system. Users can develop a system from a hand drawn UI to a system that meets the expectations of users by few clicks. It has reduced ethe cost of systems, allowed code reuse, and refactor as well as customizable code provided for the users to develop a system with minimum effort and time.

REFERENCE LIST.

[1]. Ambika Choudhury, “10 AI Applications that can generate code themselves.” https://analyticsindiamag.com/10-ai-applications-that-can-generate-code-themselves/ (accessed Jan. 03, 2022)

[2]. H. Liao, J. Jiang and Y. Zhang, "A Study of Automatic Code Generation," 2010 International Conference on Computational and Information Sciences, 2010, pp. 689-691, doi: 10.1109/ICCIS.2010.171.

[3]. Omar EL BEGGAR, Brahim BOUSETTA and Taoufiq GADI, "Automatic code generation by model transformation from sequence diagram of system's internal behavior", *International Journal of Computer and Information Technology*, vol. 01, no. 02, November 2012

[4]. Prajkta R. Pawde and Vikrant Chole, "Generation of Java Code Structure from UML Class Diagram", *International Journal of Innovative Science and Modern Engineering (IJISME)*, vol. 2, no. 7, June 2014.

[5]. Harshal D. Gurad and V. S. Mahalle, "Transformation Of Uml Sequence Diagram To Java Code", *International Journal of Pure and Applied Research in Engineering and Technology*, vol. 2, no. 8, pp. 703-710, 2014.

[6]. A. Desai and P. Patadia, "Drag and Drop: A Better Approach to CAPTCHA," 2009 Annual IEEE India Conference, 2009, pp. 1-4, doi: 10.1109/INDCON.2009.5409359.

[7]. Pavel Bernshtam, “Code generation for backend and frontend.When, why and how?” https://itnext.io/code-generation-for-backend-and-frontend-when-why-and-how-a14d1cbbc47b (accessed Jan. 12, 2022)

[8]. A Divya and V. Renuka, "A Novel Approach for Business Logic Evaluation Model in Web Service Environment", *IJISET - International Journal of Innovative Science Engineering & Technology*, vol. 1, no. 9, November 2014.

[9]. Sun, Zhengxing, and Jing Liu. "Informal user interface for graphical computing." International Conference on Affective Computing and Intelligent Interaction. Springer, Berlin, Heidelberg, 2005.

[10]. M. Usman, A. Nadeem and T. -h. Kim, "UJECTOR: A Tool for Executable Code Generation from UML Models," 2008 Advanced Software Engineering and Its Applications, 2008, pp. 165-170, doi: 10.1109/ASEA.2008.39.

[11]. M. Tubishat, I. Alsmadi and M. Al-Kabi, "Using XML files to document the user interfaces of applications," 2009 5th IEEE GCC Conference & Exhibition, 2009, pp. 1-4, doi: 10.1109/IEEEGCC.2009.5734242.

[12] Kinzel, J. "A model driven approach to build high effective and ergonomic mobile business applications." IEEE International Symposium on Consumer Electronics (ISCE 2010). IEEE, 2010.

[13]. S. D. Rathod, "Automatic code generation with business logic by capturing attributes from user interface via XML," 2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT), 2016, pp. 1480-1484, doi: 10.1109/ICEEOT.2016.7754929.

[14]. A. Desai and P. Patadia, "Drag and Drop: A Better Approach to CAPTCHA," 2009 Annual IEEE India Conference, 2009, pp. 1-4, doi: 10.1109/INDCON.2009.5409359.

[15]. M. Ko and Y. B. Park, "Automatic Generation of UML Model-Based Image Processing Source Code in Hadoop Platform," 2017 International Conference on Platform Technology and Service (PlatCon), 2017, pp. 1-4, doi: 10.1109/PlatCon.2017.7883690.

[16]. Chowdhury, Anirban. "Design and development of a stencil for mobile user interface (UI) design." Research into Design for a Connected World. Springer, Singapore, 2019. 629-639.

[17]. B. Aşıroğlu et al., "Automatic HTML Code Generation from Mock-Up Images Using Machine Learning Techniques," 2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT), 2019, pp. 1-4, doi: 10.1109/EBBT.2019.8741736.

[18]. Sermuga Pandian, Vinoth Pandian, et al. "MetaMorph: AI Assistance to Transform Lo-Fi Sketches to Higher Fidelities." 32nd Australian Conference on Human-Computer Interaction. 2020.

[19] Pereira, Carlos Eduardo, Fábio Fagundes Lima, and João Pedro Granja. "Code Generation based on Uml Model Transformation."

[20]. Elec, Turk & Sci, Comp & Uyanik, Burak & Şahin, Veysel. (2020). A template-based code generator for web applications. Turkish Journal of Electrical Engineering and Computer Sciences. 28. 1747 – 1762. 10.3906/elk-1910-44.

[21]. M. d'Amorim, R. Abreu and C. Mello, "Visual Sketching: From Image Sketches to Code," 2020 IEEE/ACM 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), 2020, pp. 101-104.

[22]. JonnadulaNarasimha Rao, GajulaHarish, “PRE-PROGRAMMED WEB PAGE IMPLEMENTATION FOR MOCKUP IMAGE” http://www.joics.org/gallery/ics-2878%20-%201.pdf [Accessed: 20 – Jan - 2022]

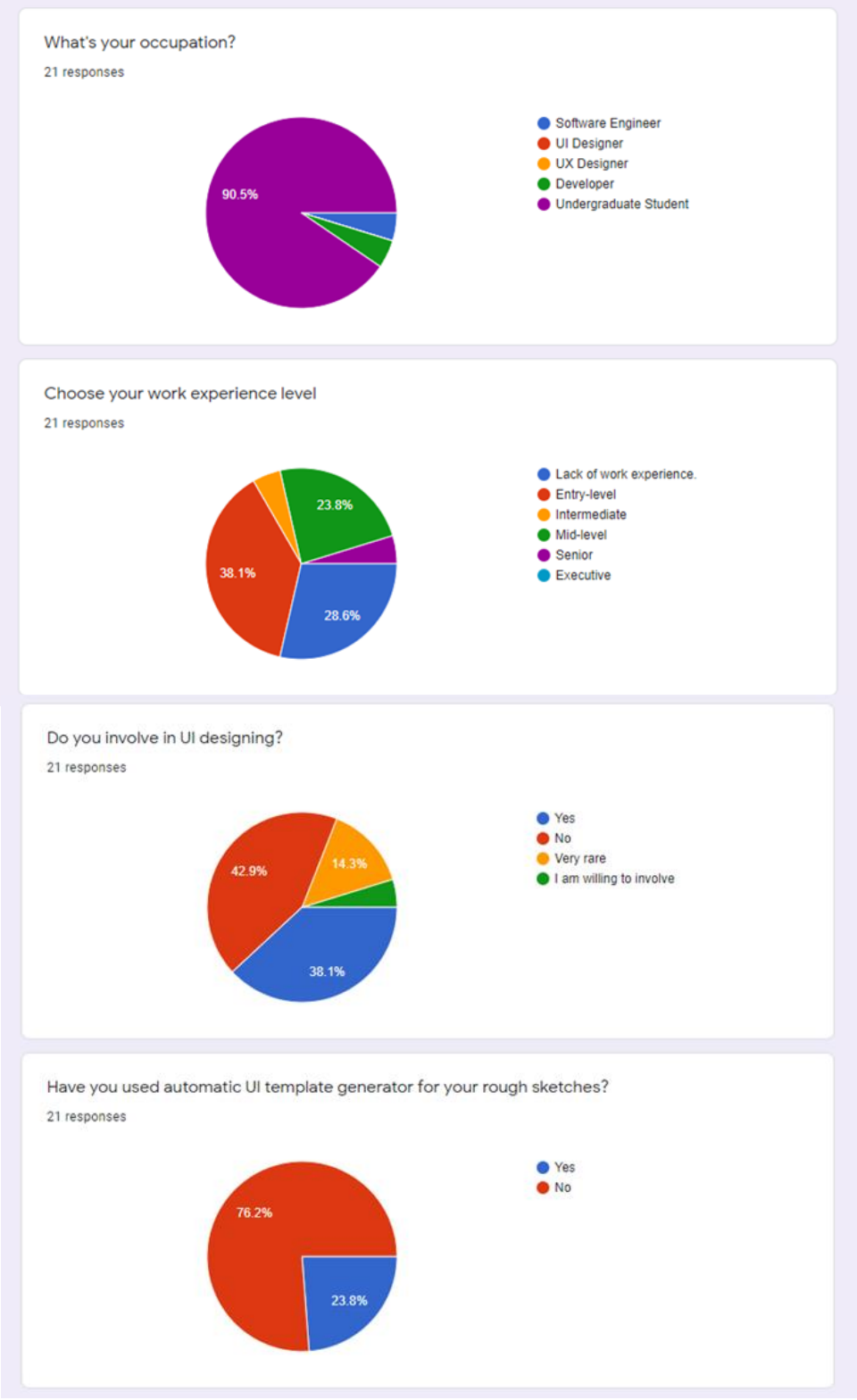
[23]. T. Beltramelli, “pix2code: Generating code from a graphical user interface screenshot,” CoRR, vol. abs/1705.07962, 2017. [Online]. Available: http://arxiv.org/abs/1705.07962

[24]. Akbulut A, Toprak S. Code generator framework for smart TV platforms. IET Software 2019; 13 (4): 268–279. doi: 10.1049/iet-sen.2018.5157

[25]. Possatto MA, Lucrédio D. Automatically propagating changes from reference implementations to code generation templates. Information and Software Technology 2015; 67: 65–78. doi: 10.1016/j.infsof.2015.06.009

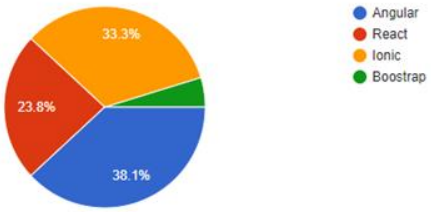
APPENDICES

Appendix - A: Results from selected user group (UI Generator)



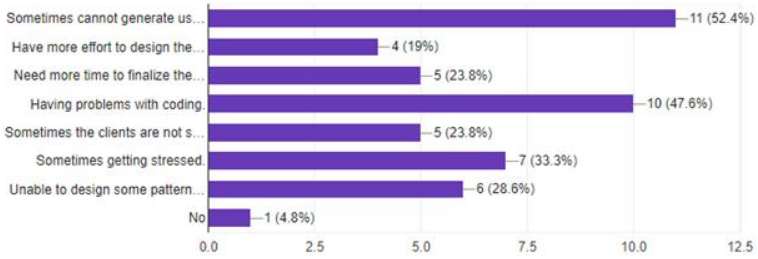
What type of framework you mostly work?

21 responses



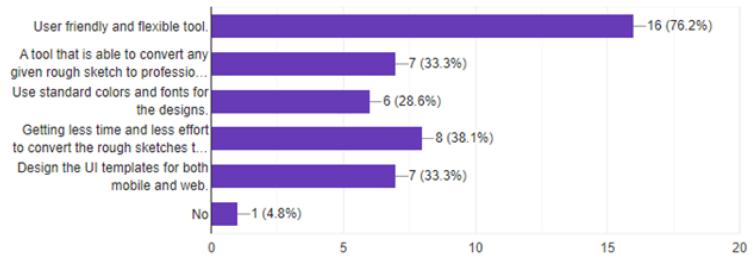
Select the challenges you face with the UI templates designing according to the rough sketches.

21 responses



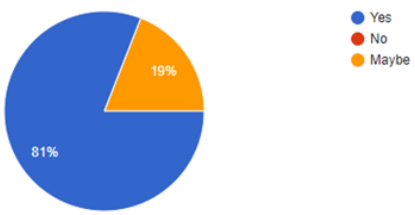
Select your expectations for the UI template generator that you may use in your future projects.

21 responses



Are you willing to use automatic UI template generator in your future projects?

21 responses



Other than these needs provide your other expectations or requirements you expect to be fulfilled by automatic UI template generator.

21 responses

- No
- The tool should be an efficient one
- Above points cover all my expectations
- Eye-catching
- user friendly
- Yes
- Generated UI templates must fulfill the given standards such as it must be able to identify exact requirements and have a borad touch on resources identify those said requirements
- The tool should be able to suggest design standards by its own
- A user-friendly one

Other than these needs provide your other expectations or requirements you expect to be fulfilled by automatic UI template generator.

21 responses

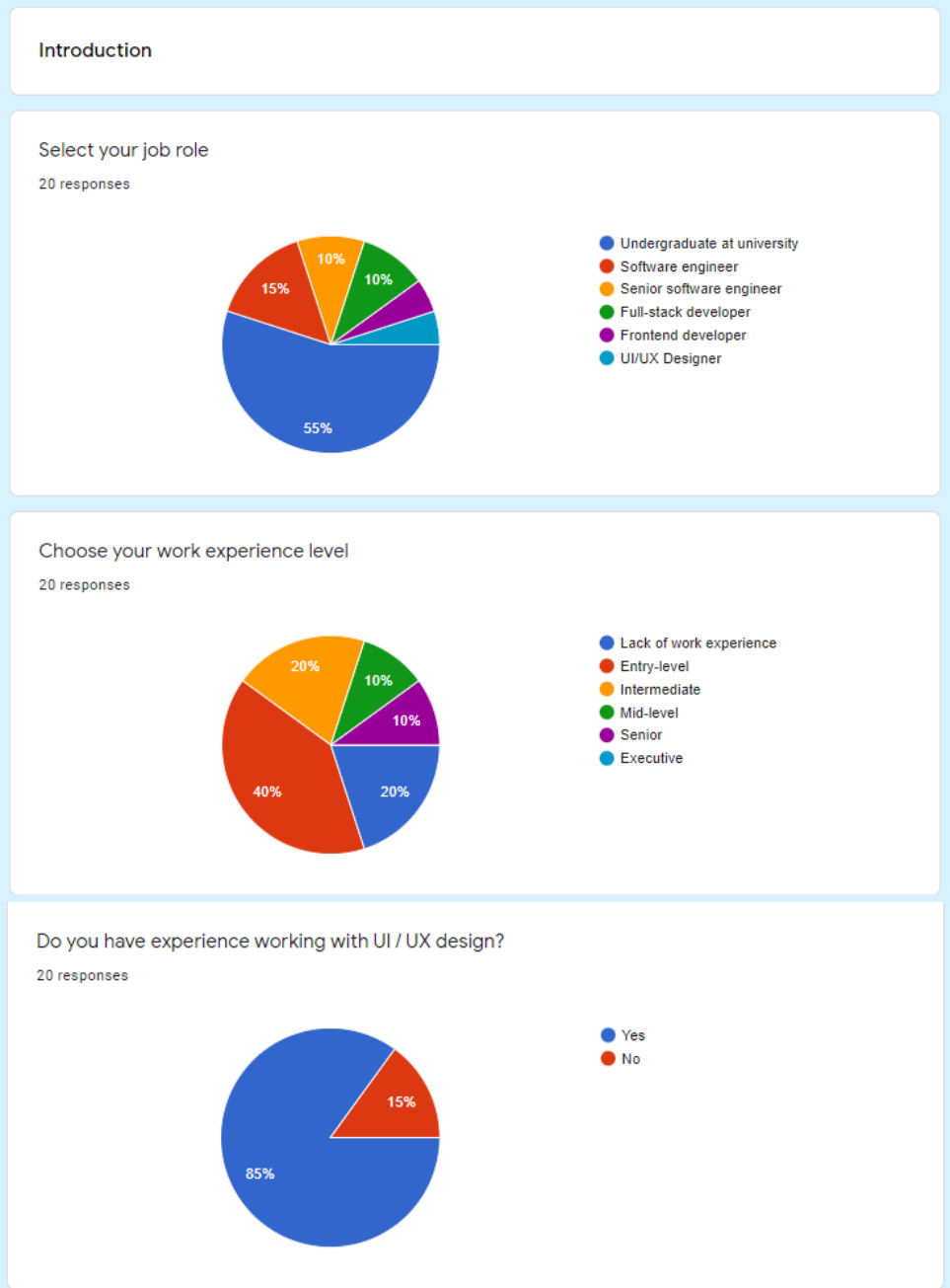
- Design the UI templates for web
- User friendly
- Ability to do a task within very short period of time
- Nothing
- lots of customization ability
- Please arrange any workshops for UI design
- Should be able to satisfy client
- Building a user friendnly UI
- -

Other than these needs provide your other expectations or requirements you expect to be fulfilled by automatic UI template generator.

21 responses

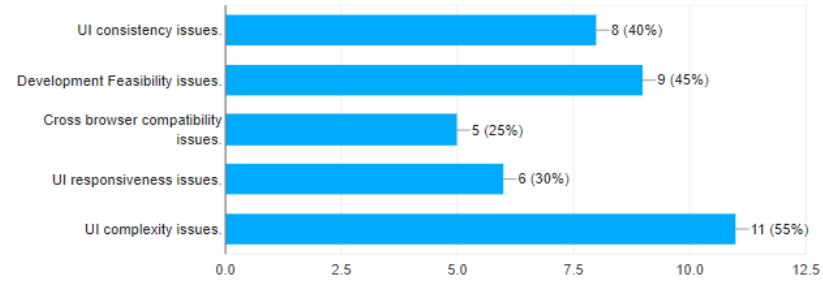
- Ability to do a task within very short period of time
- Nothing
- lots of customization ability
- Please arrange any workshops for UI design
- Should be able to satisfy client
- Building a user friendnly UI
- -
- Make templates according to people's mind
- Should be user friendly and attractive.

Appendix - B: Results from selected user group (Web Frontend Generator)



What difficulties do you face when designing a new UI?

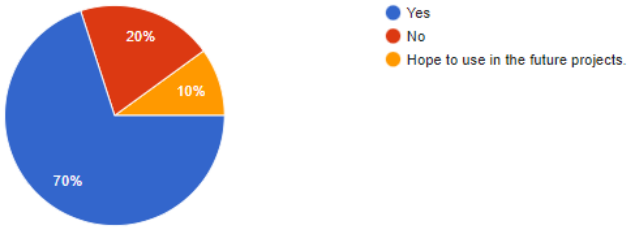
20 responses



WEB FRONTEND CODE GENERATOR

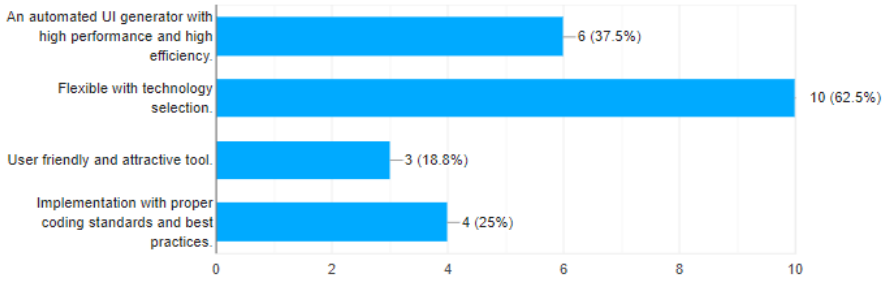
Have you used automated code generation tools in any past projects?

20 responses



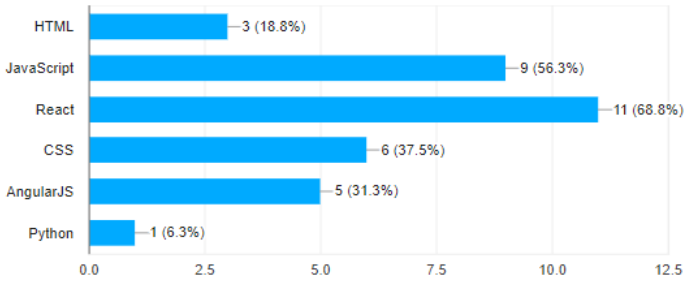
Select your expectations on automated frontend UI generator?

16 responses



What languages would you most like to use for web front development?

16 responses



In addition to the above, give us your other ideas and requirements about automated UI generator.

4 responses

Frontend code should be able to download & modify

Should be more user friendly

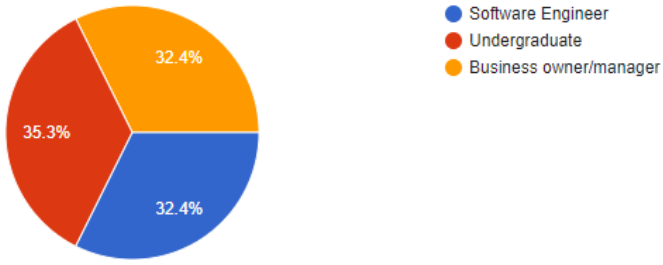
Code should be accurate & simple. Because sometimes codes are very complex and cannot understand.

Should be able to download both frontend code and CSS file.

Appendix - C: Results from selected user group (Mobile Frontend Generator)

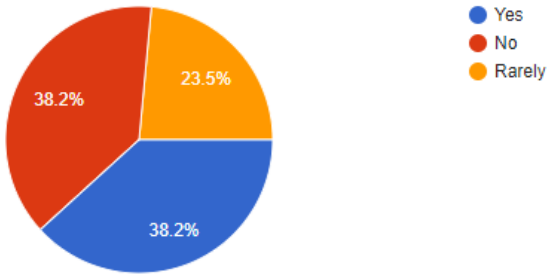
What is your occupation?

34 responses



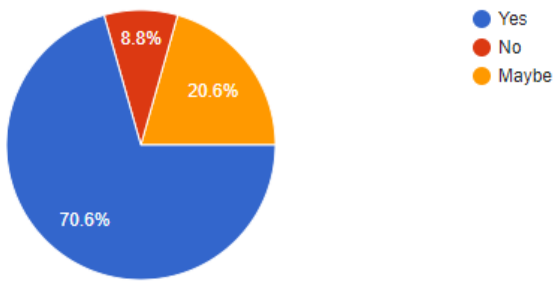
Do you have any experience on mobile application development?

34 responses



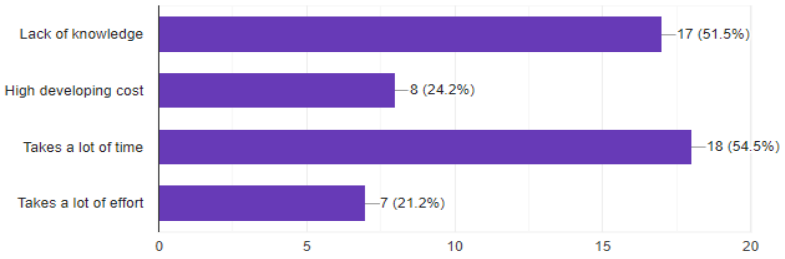
Are you interested in developing your own mobile application?

34 responses



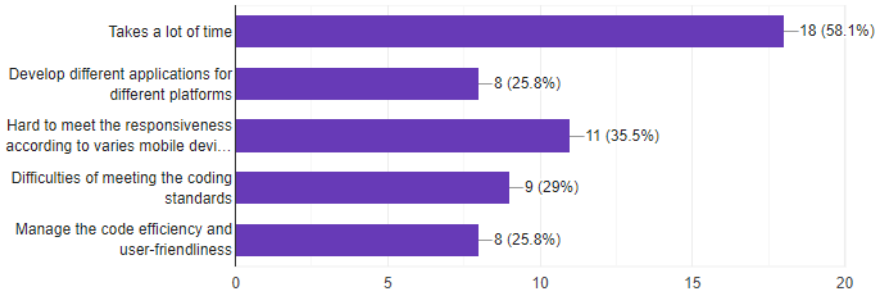
If you were interested in developing a mobile application what made you avoid developing an application?

33 responses



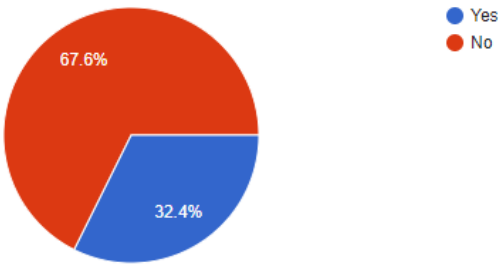
If you have the experience of developing, what are the challenges that you go through?

31 responses



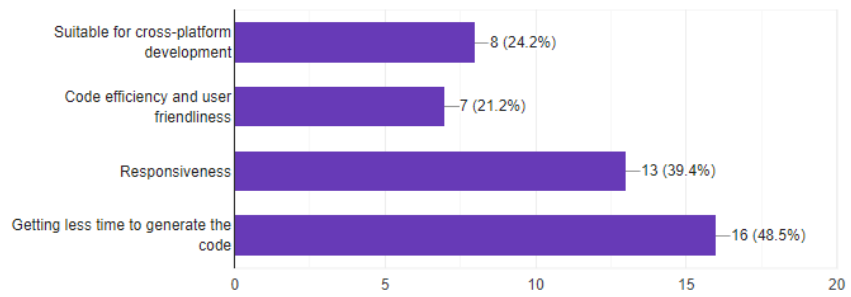
Have you used a frontend mobile code generator?

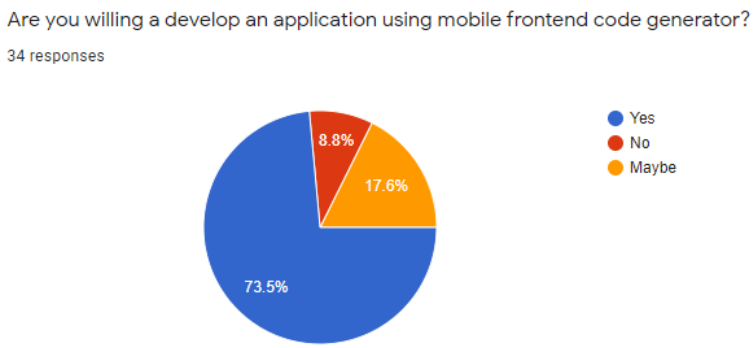
34 responses



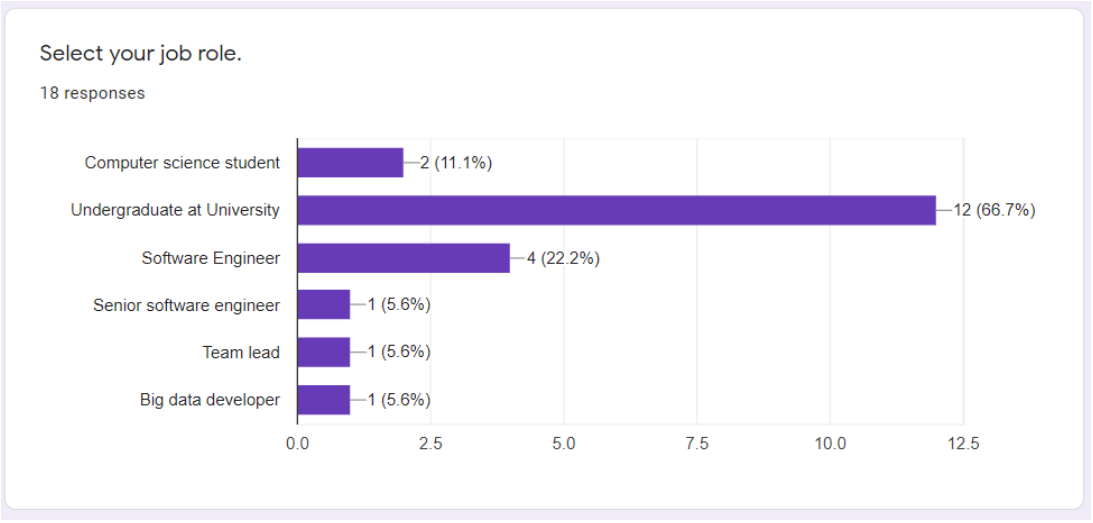
What are the expectations for frontend mobile code generator?

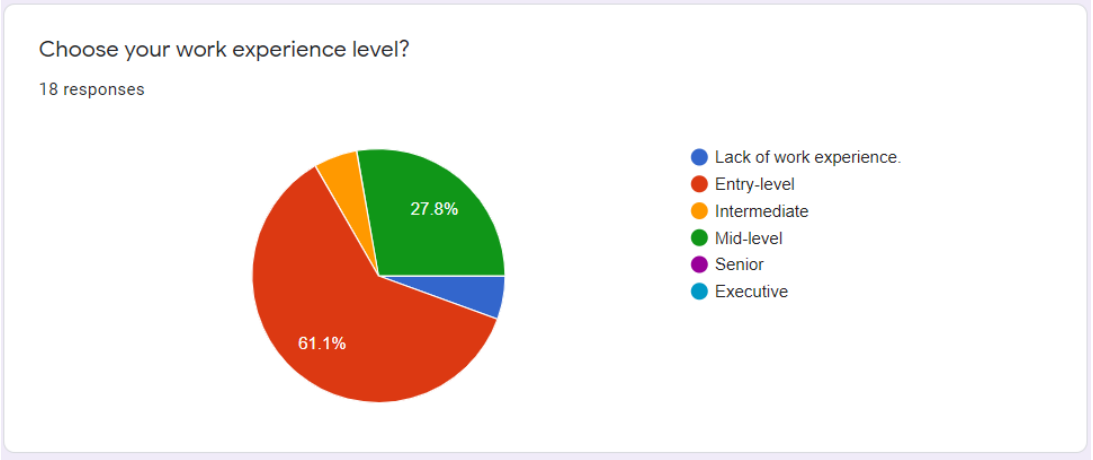
33 responses





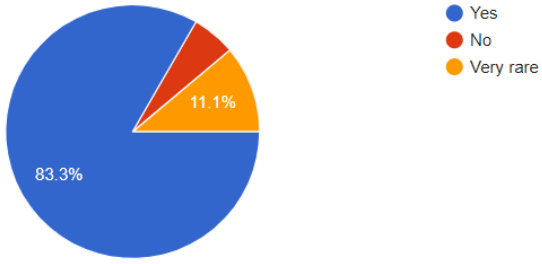
Appendix - D: Results from selected user group (Backend Generator)





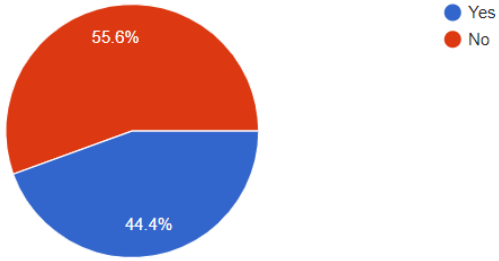
Do you involve in backend development and database implementation?

18 responses



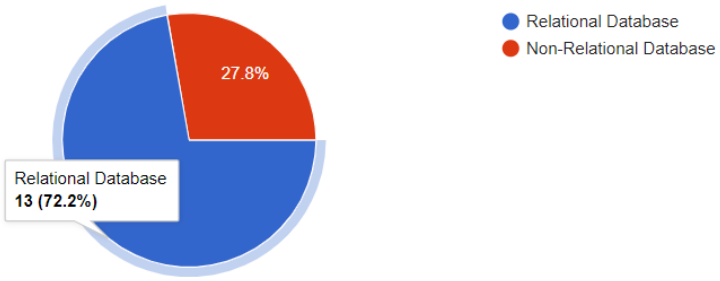
Have you used automatic code generation tools in any of your projects.

18 responses



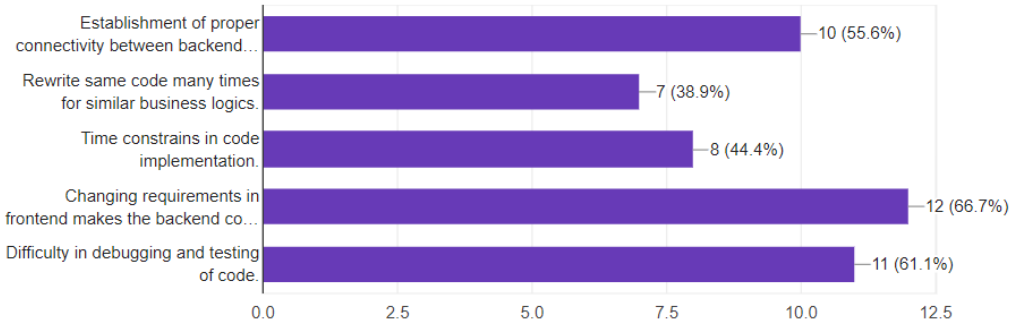
What type of database you mostly work?

18 responses



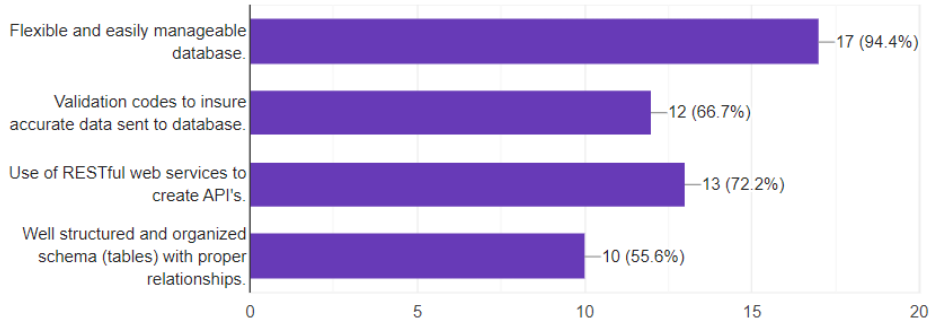
Select the challenges you face with the backend code implementation.

18 responses



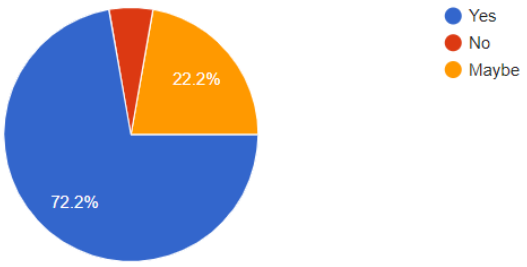
Select your expectations on database used for your project.

18 responses

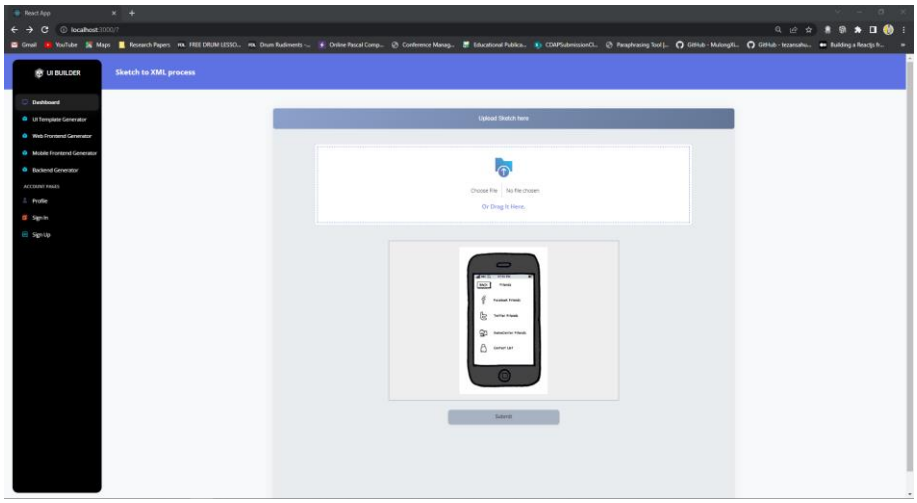


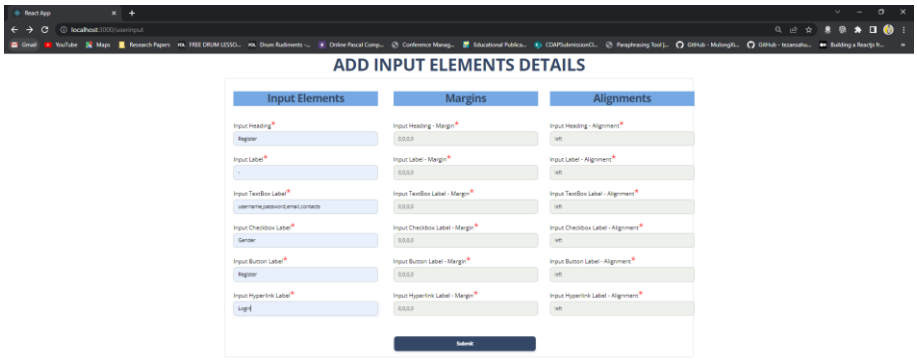
Are you willing to use automatic backend code and database generators in your projects.

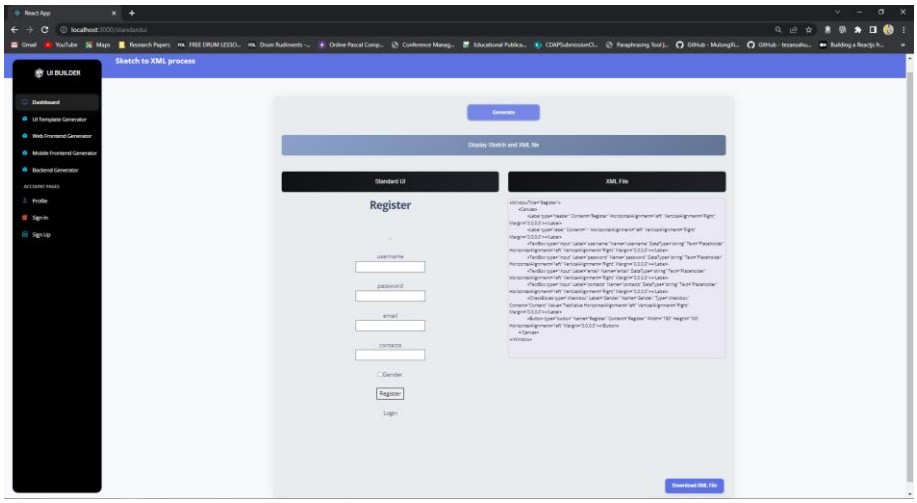
18 responses



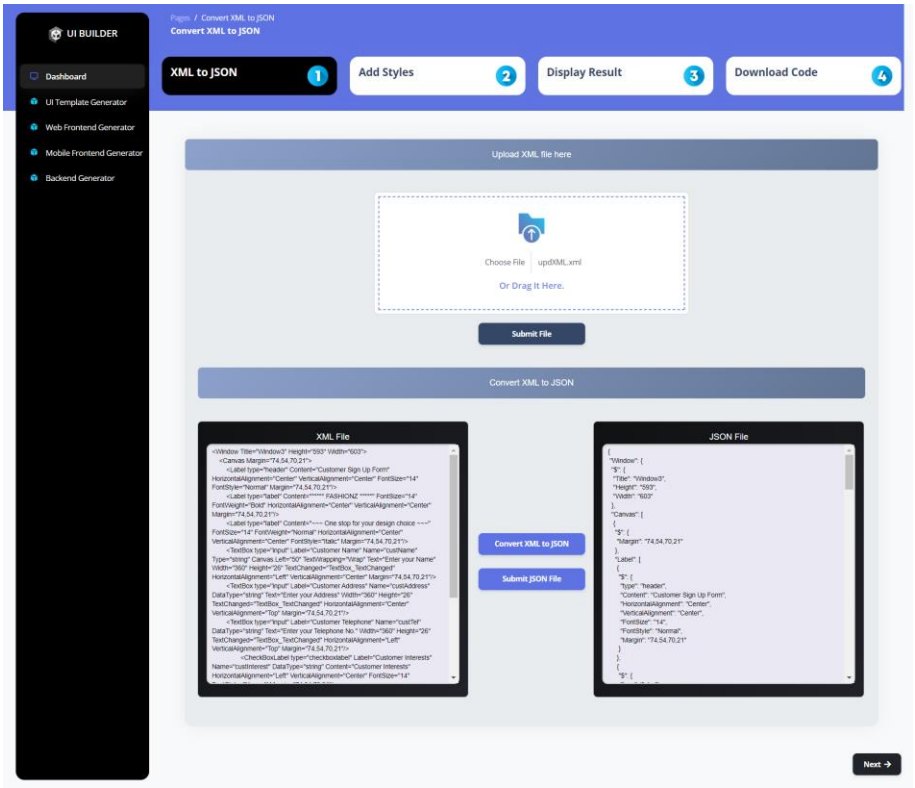
Appendix - E: UI Generator Interfaces

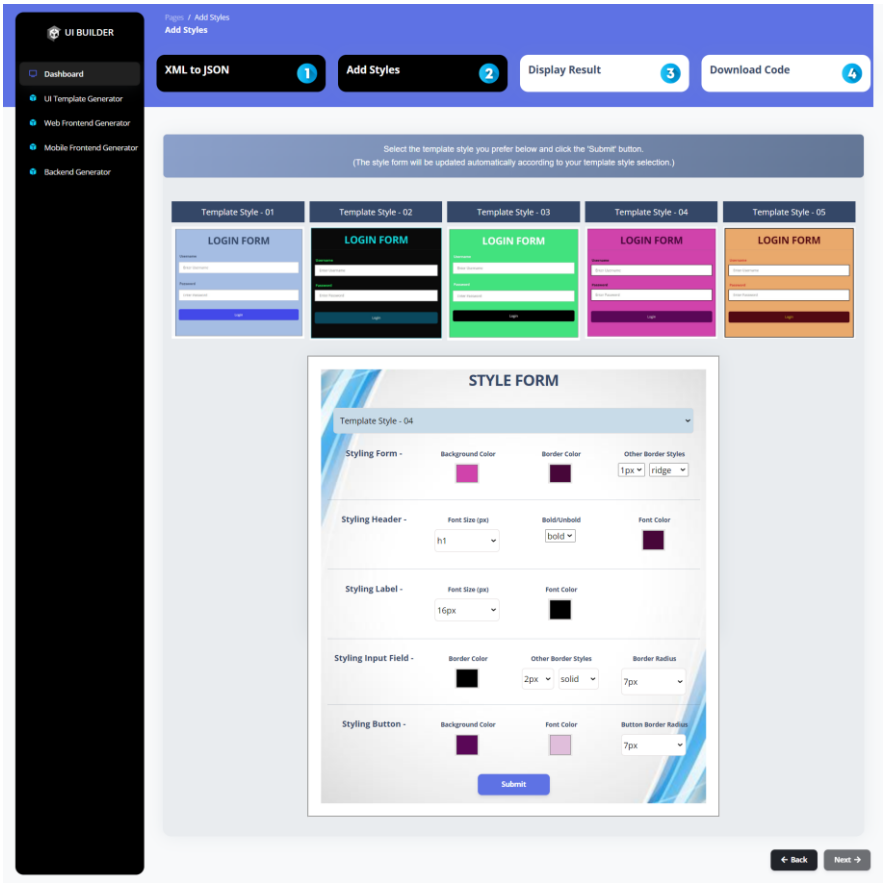


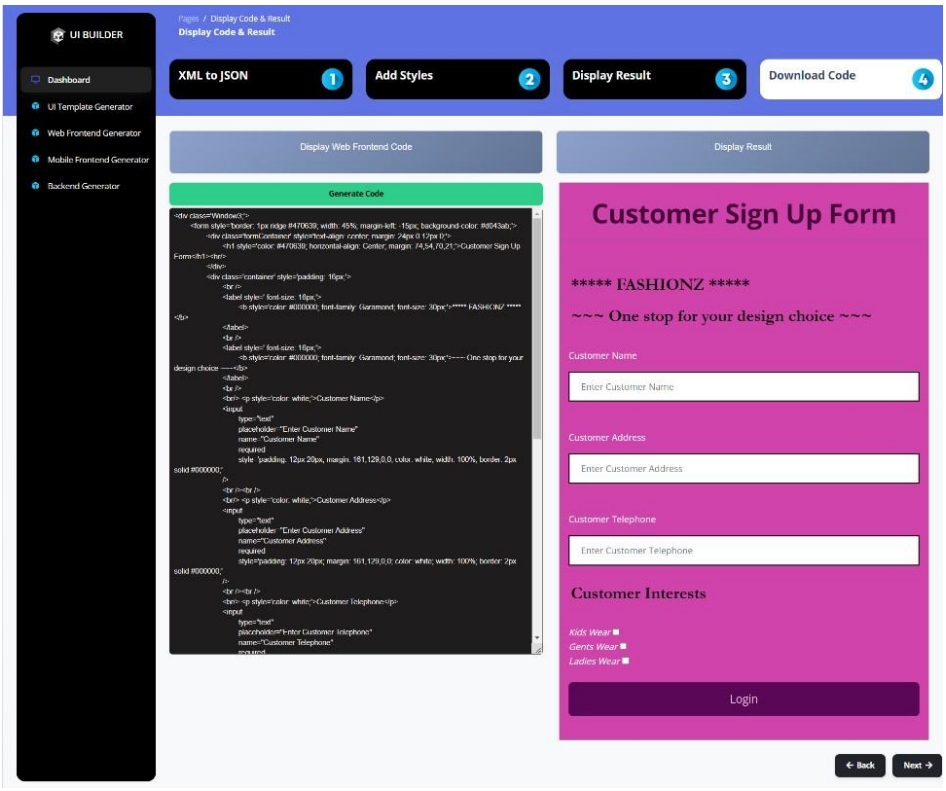


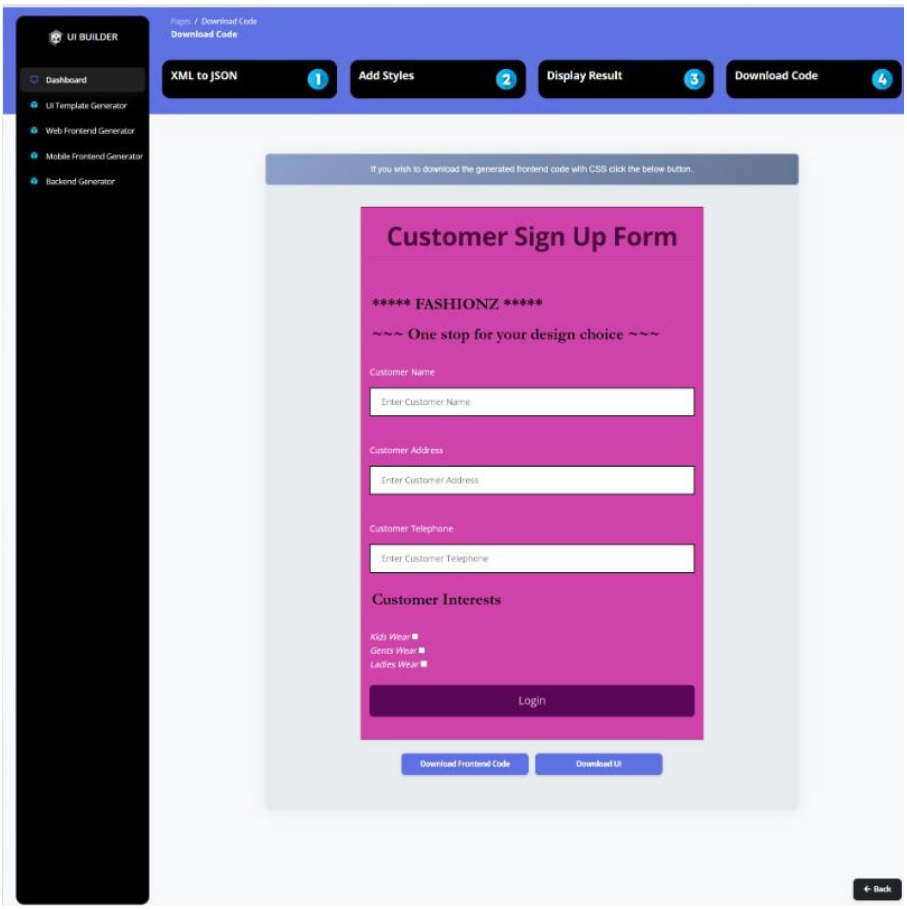


Appendix - F: Web Frontend Generator Interfaces

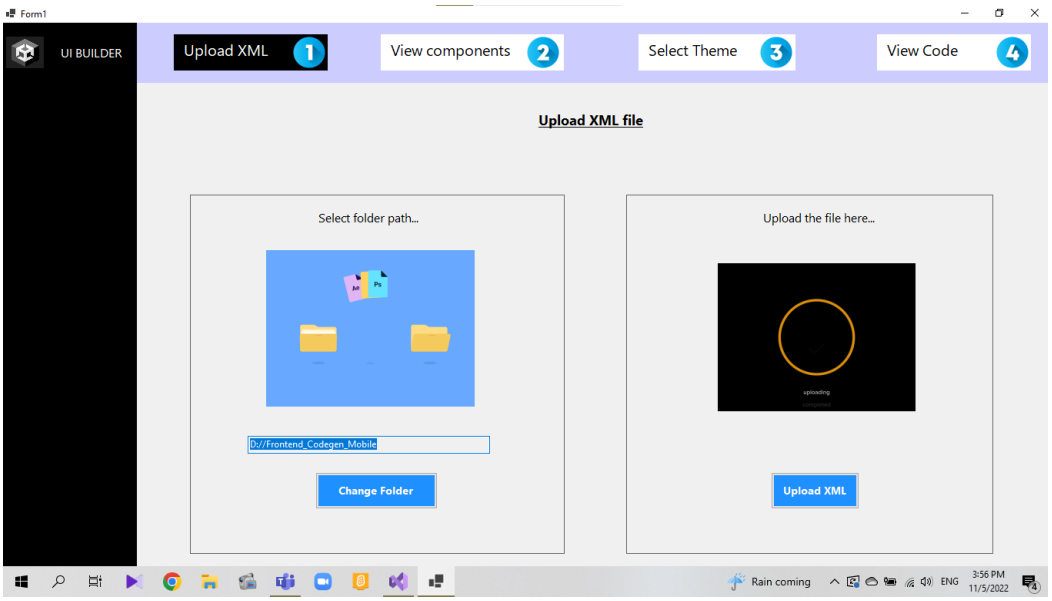


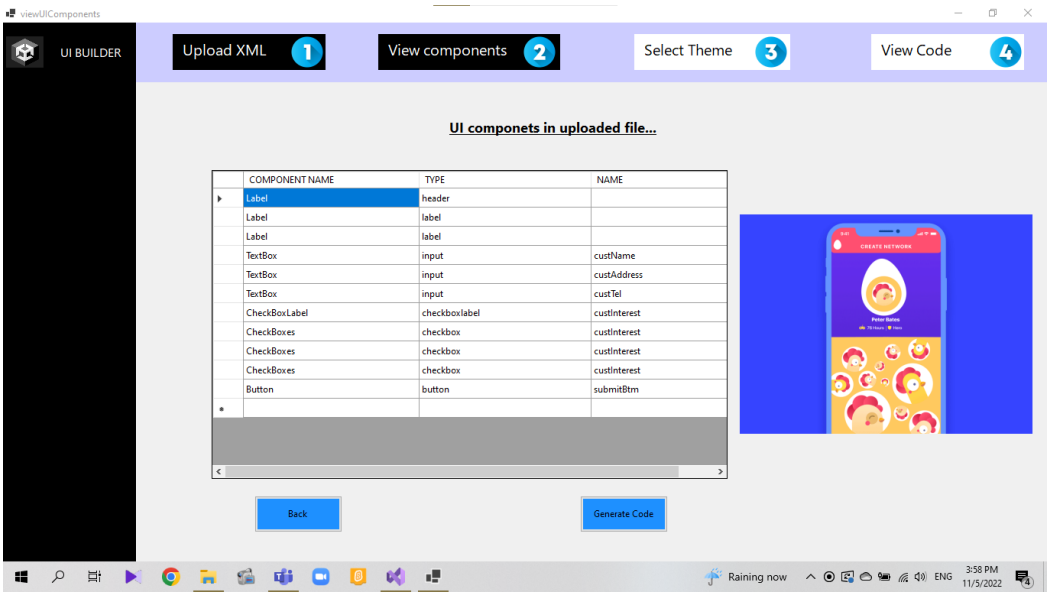


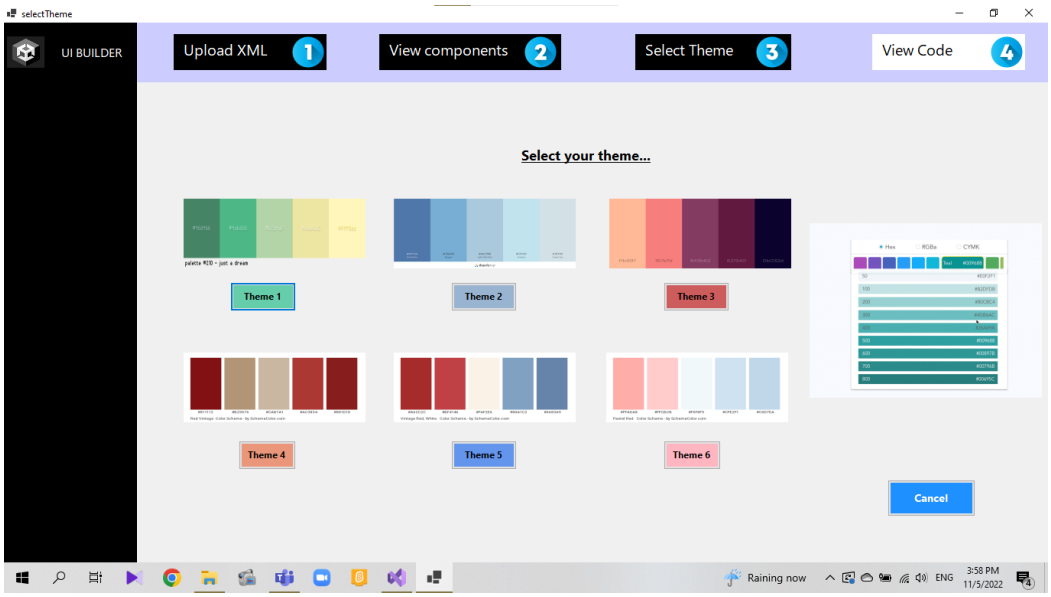

UI BUILDER
Dashboard
UI Template Generator
Web Frontend Generator
Mobile Frontend Generator
Backend Generator
Pages / Display Code & Result
Display Code & Result
XML to JSON
1
Add Styles
2
Display Result
3
Download Code
4
Display Web Frontend Code
Display Result
Generate Code
Customer Sign Up Form
***** FASHIONZ *****
~~~ One stop for your design choice ~~~
Customer Name
Enter Customer Name
Customer Address
Enter Customer Address
Customer Telephone
Enter Customer Telephone
Customer Interests
Kids Wear
Gents Wear
Ladies Wear
Login
← Back
Next →

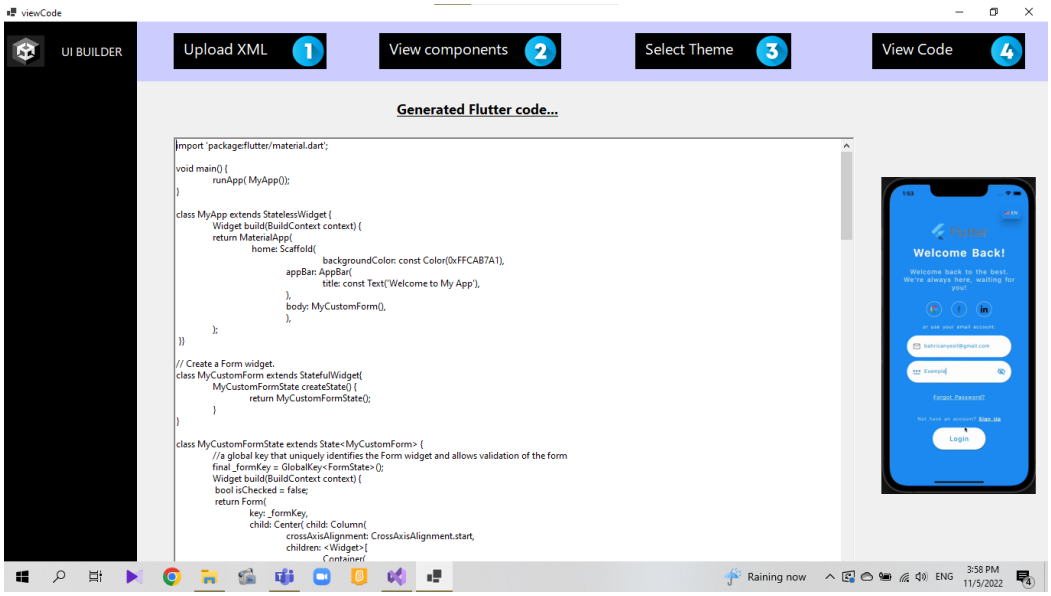

UI BUILDER
Dashboard
UI Template Generator
Web Frontend Generator
Mobile Frontend Generator
Backend Generator
Pages / Download Code
Download Code
XML to JSON
1
Add Styles
2
Display Result
3
Download Code
4
If you wish to download the generated frontend code with CSS click the below button.
Customer Sign Up Form
***** FASHIONZ *****
~~~ One stop for your design choice ~~~
Customer Name
Enter Customer Name
Customer Address
Enter Customer Address
Customer Telephone
Enter Customer Telephone
Customer Interests
Kids Wear
Gents Wear
Ladies Wear
Login
Download Frontend Code
Download UI
← Back

Appendix - G: Mobile Frontend Generator Interfaces

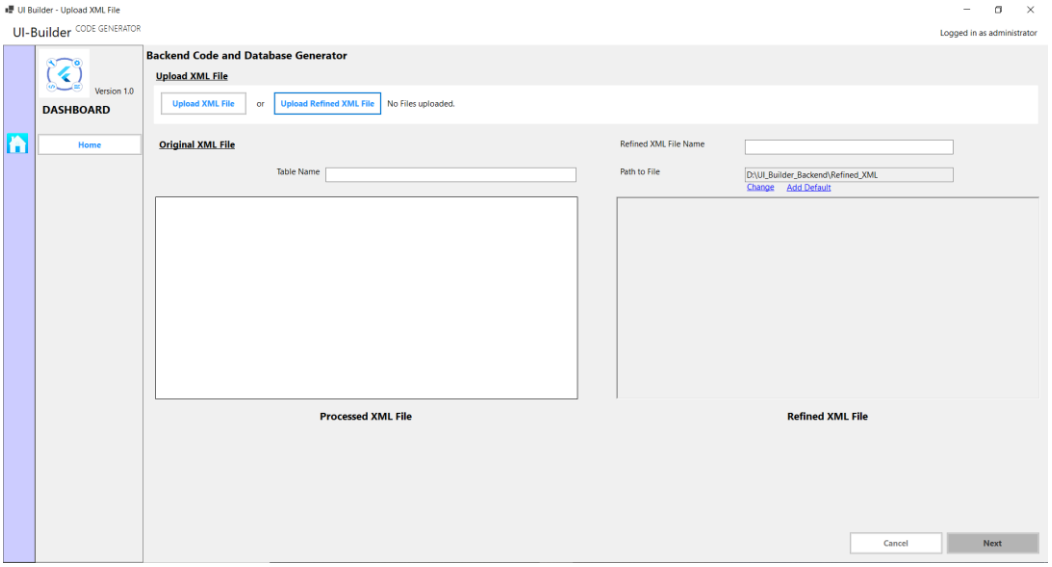


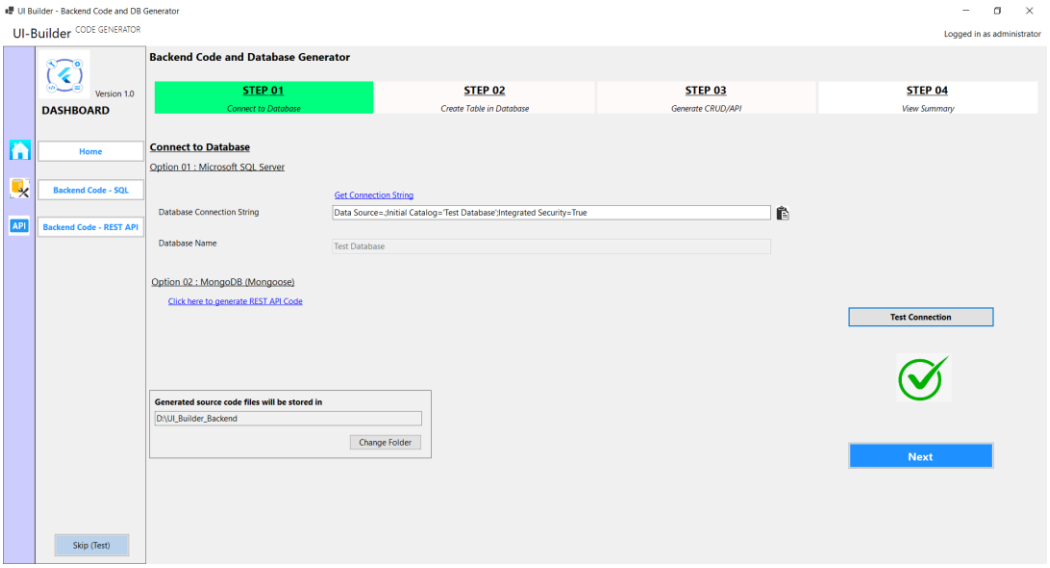


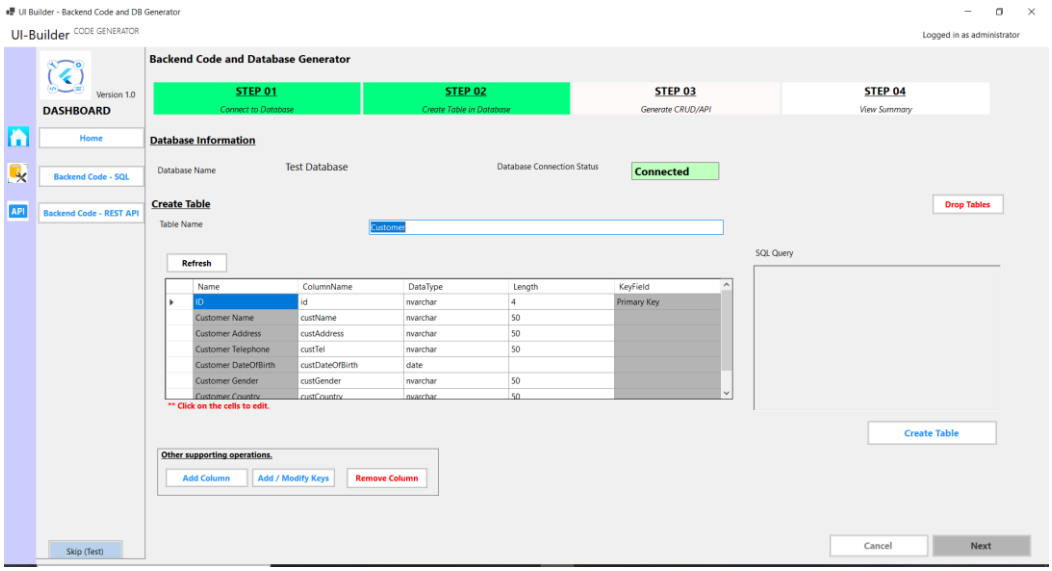

selectTheme
UI BUILDER
Upload XML 1
View components 2
Select Theme 3
View Code 4
Select your theme...
Theme 1
Theme 2
Theme 3
Theme 4
Theme 5
Theme 6
Cancel
Raining now
ENG
3:58 PM
11/5/2022

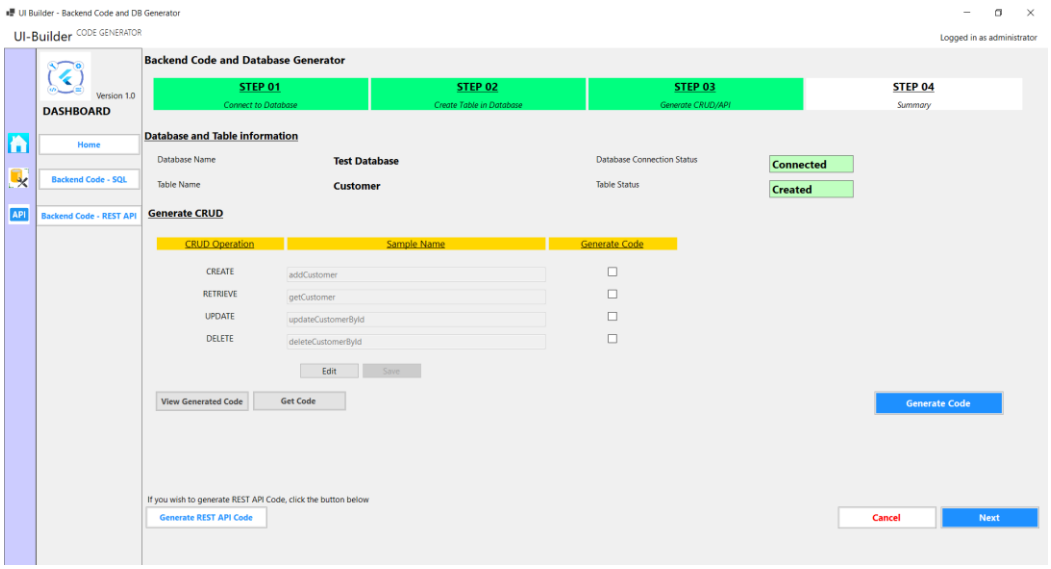

viewCode
UI BUILDER
Upload XML 1
View components 2
Select Theme 3
View Code 4
Generated Flutter code...
import 'package:flutter/material.dart';
void main() {
runApp(MyApp());
}
class MyApp extends StatelessWidget {
Widget build(BuildContext context) {
return MaterialApp(
home: Scaffold(
backgroundColor: const Color(0xFFCAB7A1),
appBar: AppBar(
title: const Text('Welcome to My App'),
),
body: MyCustomForm(),
),
);
}}
// Create a Form widget.
class MyCustomForm extends StatefulWidget{
MyCustomFormState createState() {
return MyCustomFormState();
}
}
class MyCustomFormState extends State<MyCustomForm> {
//a global key that uniquely identifies the Form widget and allows validation of the form
final _formKey = GlobalKey<FormState>();
Widget build(BuildContext context) {
bool isChecked = false;
return Form(
key: _formKey,
child: Center(child: Column(
crossAxisAlignment: CrossAxisAlignment.start,
children: <Widget>[
Container(
Welcome Back!
Login
Raining now
ENG
3:58 PM
11/5/2022

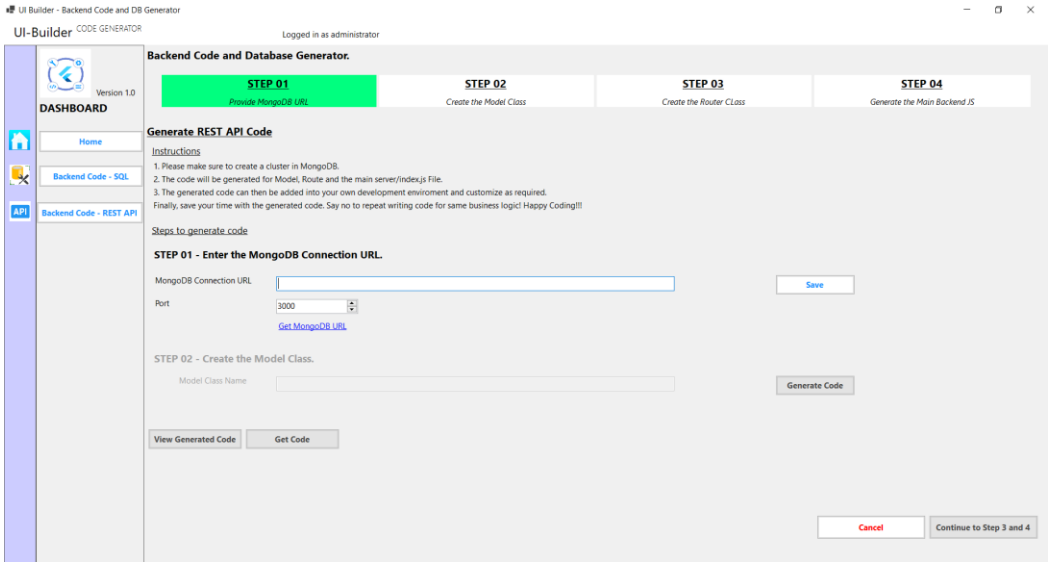
Appendix - H: Backend & Database Generator Interfaces

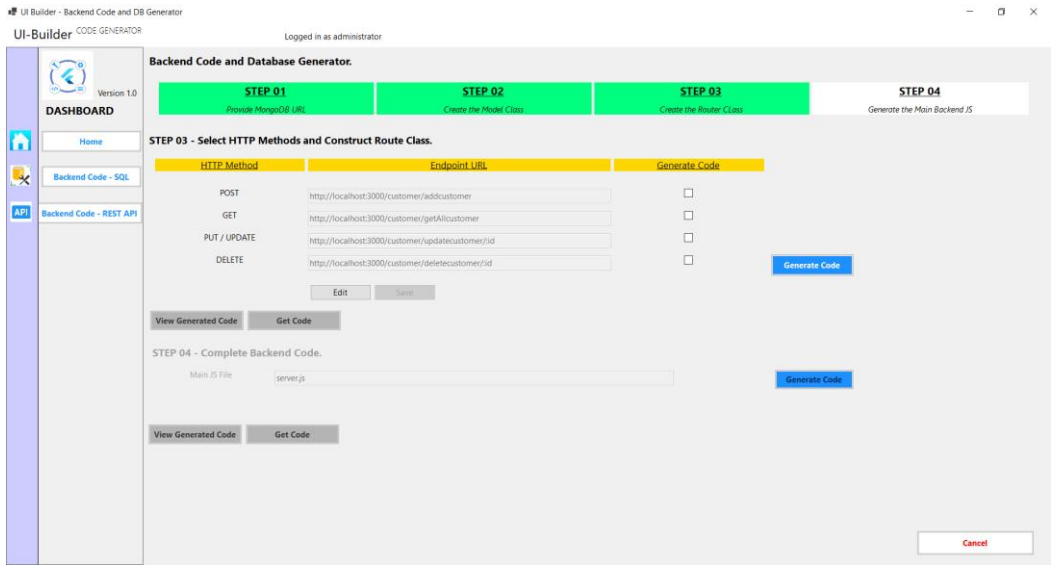







UI Builder - Backend Code and DB Generator
UI-Builder CODE GENERATOR
Logged in as administrator
Version 1.0
DASHBOARD
Home
Backend Code - SQL
Backend Code - REST API
Backend Code and Database Generator
STEP 01
Connect to Database
STEP 02
Create Table in Database
STEP 03
Generate CRUD/API
STEP 04
Summary
Database and Table information
Database Name
Test Database
Database Connection Status
Connected
Table Name
Customer
Table Status
Created
Generate CRUD
CRUD Operation
Sample Name
Generate Code
CREATE
addCustomer
RETRIEVE
getCustomer
UPDATE
updateCustomerById
DELETE
deleteCustomerById
Edit
Save
View Generated Code
Get Code
Generate Code
If you wish to generate REST API Code, click the button below
Generate REST API Code
Cancel
Next


UI Builder - Backend Code and DB Generator
UI-Builder CODE GENERATOR
Logged in as administrator
Version 1.0
DASHBOARD
Home
Backend Code - SQL
Backend Code - REST API
Backend Code and Database Generator.
STEP 01
Provide MongoDB URL
STEP 02
Create the Model Class
STEP 03
Create the Router CLass
STEP 04
Generate the Main Backend JS
Generate REST API Code
Instructions
1. Please make sure to create a cluster in MongoDB.
2. The code will be generated for Model, Route and the main server/index.js File.
3. The generated code can then be added into your own development enviroment and customize as required.
Finally, save your time with the generated code. Say no to repeat writing code for same business logic! Happy Coding!!!
Steps to generate code
STEP 01 - Enter the MongoDB Connection URL.
MongoDB Connection URL
Save
Port
3000
Get MongoDB URL
STEP 02 - Create the Model Class.
Model Class Name
Generate Code
View Generated Code
Get Code
Cancel
Continue to Step 3 and 4


UI Builder - Backend Code and DB Generator
UI-Builder CODE GENERATOR
Logged in as administrator
Version 1.0
DASHBOARD
Home
Backend Code - SQL
Backend Code - REST API
Backend Code and Database Generator.
STEP 01
Provide MongoDB URL
STEP 02
Create the Model Class
STEP 03
Create the Router CLass
STEP 04
Generate the Main Backend JS
STEP 03 - Select HTTP Methods and Construct Route Class.
HTTP Method
Endpoint URL
Generate Code
POST
http://localhost:3000/customer/addcustomer
GET
http://localhost:3000/customer/getAllcustomer
PUT / UPDATE
http://localhost:3000/customer/updatecustomer/:id
DELETE
http://localhost:3000/customer/deletecustomer/:id
Edit
Save
Generate Code
View Generated Code
Get Code
STEP 04 - Complete Backend Code.
Main JS File
server.js
Generate Code
View Generated Code
Get Code
Cancel

Appendix - I: Plagiarism Report

